Deep Bayesian Inference for Control: Normalizing Flow – Based Approaches to System Identification, State Estimation, and Optimal Control

genehmigte Dissertation
zur Erlangung des akademischen Grades der Doktor-Ingenieurin (Dr.-Ing.).
an der
Fakultät für Angewandte Informatik
der
Universität Augsburg

vorgelegt von
Yi Zhang
aus
Tangshan, China

2026

Dekan: Prof. Dr. Jörg Hähner
Doktorvater: Prof. Dr.-Ing. Lars Mikelsons

I dedicate this thesis to my family.

Contents

Acronyms **i**

List of Figures **vii**

List of Tables **ix**

I Introduction **1**
1 Motivation 1
2 Objectives and Contributions 6
3 Outline 7

II Background and Preliminaries **9**
1 Notations 9
2 Bayesian Inference 10
2.1 Bayes' Rule 10
2.2 Markov Chain Monte Carlo 11
2.2.1 Metropolis-Hastings Algorithm 12
2.2.2 Metropolis-Adjusted Langevin Algorithm 12
2.3 Variational Inference 15
3 Deep Probabilistic Methods 17
3.1 Deep Gaussian Process 17
3.1.1 Gaussian Process Regression 18
3.1.2 Parameter Identification using GPR 19
3.1.3 Deep Gaussian Process 19
3.2 Bayesian Neural Network 20
3.3 Variational Autoencoder 22

4 Normalizing Flow . . . 24
4.1 Basics . . . 24
4.1.1 Change of Variables . . . 24
4.1.2 Using Normalizing Flows for Modeling and Inference . . . 28
4.2 Coupling Flows . . . 29
4.3 Residual Flows . . . 31
4.4 Continuous Flows . . . 32
4.4.1 Neural Ordinary Differential Equations . . . 33
4.4.2 Continuous Normalizing Flows . . . 34
4.5 Flow Matching . . . 35
4.6 Conditional Normalizing Flows . . . 36
5 From Bayesian Inference to Control . . . 38
5.1 Bayesian System Identification . . . 38
5.1.1 Bayesian Model Selection . . . 39
5.1.2 Bayesian Parameter Identification . . . 40
5.2 Bayesian State Estimation . . . 41
5.2.1 Bayesian Filters . . . 41
5.2.2 State-Space Models . . . 43
5.2.3 Stochastic Differential Equations . . . 44
5.3 Model Predictive Control . . . 44
5.3.1 Nominal Model Predictive Control . . . 45
5.3.2 Stochastic Model Predictive Control . . . 46
5.4 Reinforcement Learning for Control . . . 47

III Literature Review **49**
1 Bayesian Parameter Identification . . . 49
1.1 Deep Gaussian Processes . . . 49
1.2 Bayesian Neural Networks . . . 50
1.3 Conditional Variational Autoencoders . . . 50
1.4 Conditional Normalizing Flows . . . 51
1.5 Conditional Diffusion Models . . . 51
2 Bayesian State Estimation . . . 51
2.1 Data-Assisted Bayesian Filters . . . 52
2.2 Deep State-Space Models . . . 52
3 Deep Probabilistic Models and Time Series Modeling . . . 53
3.1 Integration of Deep Probabilistic Models with Time Series Modeling Architectures . . . 53
3.2 Normalizing Flows and Stochastic Processes . . . 54

4 Deep Generative Models as Control Policy 55

IV Methodology and Numerical Validation **59**

1 Parameter Identification 59

1.1 Universal Framework with Deep Generative Models for Amortized Bayesian Inference 59

1.2 Normalizing Flows for Parameter Identification 60

1.2.1 Overall Settings 61

1.2.2 Performance Comparison (Winner: RealNVP) 62

1.2.3 Sample Efficiency (Winner: RealNVP) 64

1.2.4 Linear Correlation (Winners: SBM, RealNVP, NSF) 66

1.2.5 Multimodal Distribution (Winner: NSF) 68

1.3 Relationship Between Bayesian Inference and Sensitivity Analysis . 69

1.3.1 Sobol' index 70

1.3.2 Mutual Information 73

1.4 When Does the Framework Fail? 74

1.5 Stochastic Normalizing Flows 79

1.5.1 Theory of Stochastic Normalizing Flows 79

1.5.2 Experiments on Stochastic Normalizing Flows 82

2 State Estimation 85

2.1 Amortized State Flow 85

2.2 Latent State Space Model Flow 86

2.3 NICE Flow State Space Model 88

2.4 Stochastic Interpolant 89

3 Optimal Control 90

V Experiments and Results **91**

1 Overview 91

2 Parameter Identification 91

3 State Estimation 91

4 Optimal Control 91

VI Conclusion and Future Work **93**

Abstract **95**

Zusammenfassung **97**

Bibliography **98**

Own Contributions and Publications **99**
Own share in data acquisition and data analysis 99
Own Publications 99

Appendix **121**

Acknowledgement **123**

Eidesstattliche Versicherung (Affidavit) in German **125**

Angabe zu verwendeten KI-basierter Elektronischer Hilfsmittel **127**

Acronyms

DGM Deep Generative Model

VAE Variational Autoencoder

CVAE Conditional Variational Autoencoder

NF Normalizing Flow

CNF Conditional Normalizing Flow

SNF Stochastic Normalizing Flow

FM Flow Matching

DDPM Denoising Diffusion Probabilistic Model

SBM Score-based Diffusion Model

BNN Bayesian Neural Network

RNN Recurrent Neural Network

LSTM Long Short-Term Memory

NARX Nonlinear Autoregressive eXogenous

MPC Model Predictive Control

DGP Deep Gaussian Process

PDF Probability Density Function

CDF Cumulative Distribution Function

MCMC Markov Chain Monte Carlo

MH Metropolis-Hastings

MALA Metropolis-adjusted Langevin algorithm

RMALA Riemannian Manifold MALA

HMC Hamiltonian Monte Carlo

ELBO Evidence Lower Bound

KL Kullback-Leibler

VI Variational Inference

GPR Gaussian process regression

SE squared exponential

RBF Radial Basis Function

GP Gaussian Process

DGP Deep Gaussian Process

SGD Stochastic Gradient Descent

CAVI Coordinate Ascent Variational Inference

BbB Bayes by Backprop

CNN Convolutional Neural Network

RNN Recurrent Neural Network

ODE ordinary differential equation

SDE stochastic differential equation

NICE Non-linear Independent Component Estimation

RealNVP Real-Valued Non-Volume Preserving

MAF Masked Autoregressive Flow

NSF Neural Spline Flow

i.i.d. independent and identically distributed

INN Invertible Neural Network

cINN conditional Invertible Neural Network

HINT Hierarchical Invertible Neural Transport

MPC Model Predictive Control

DIC Deviance Information Criterion

MAP maximum a posteriori

KF Kalman Filter

EKF Extended Kalman Filter

UKF Unscented Kalman Filter

PF Particle Filter

PID Proportional-Integral-Derivative

RL Reinforcement Learning

MDP Markov Decision Process

PINN Physics-Informed Neural Network

PDE Partial Differential Equation

SSM State-Space Model

EM Expectation-Maximization

SDE Stochastic Differential Equation

SCNF Stochastic Continuous Normalizing Flow

CTFP Continuous Time Flow Process

ELU Exponential Linear Unit

RMSE Root Mean Squared Error

MCIW Mean Confidence Interval Width

CICP Confidence Interval Coverage Probabiltiy

SA Sensitivity Analysis

OOD Out-of-Distribution

MI Mutual Information

MLE Maximum Likelihood Estimation

GAN Generative Adversarial Network

List of Figures

II.1 Knowledge Map 9
II.2 MH MALA Comparison 14
II.3 Variational Autoencoder Architecture 23
II.4 The Clay Volume Analogy for Change of Variables 25
II.5 Coupling Flow Architecture 30
II.6 Illustration of three Flows 32
II.7 BayesFlow Architecture 38
II.8 Model Predictive Control (MPC) and Reinforcement Learning (RL). 45
II.9 MPC time dependency 45

IV.1 Universal Framework of Using Deep Generative Models for Amortized Bayesian Inference 60
IV.2 Detailed Comparison of Deep Bayesian Inference Methods 63
IV.3 Performance Comparison w.r.t. Sample Efficiency 64
IV.4 Linear Correlation Comparison 67
IV.5 Multimodal Comparison 68
IV.6 Convergence Speed & Sensitivity Analysis 71
IV.7 Global Sensitivity Analysis of Free-Vibration and Symmetric Coupled Mass-Spring-Models 72
IV.8 Comparison of Sensitivity and Identifiability Under Noisy Observations . . 74
IV.9 Stress Test of Normalizing Flows under Noise 75
IV.10 Evaluation of Stress Test under Misspecification and OOD 76
IV.11 Micro View of Distribution under Misspecification 77
IV.12 Global Residual Analysis under OOD 77
IV.13 SNF Architecture 80
IV.14 Multimodal Modeling Comparison between NF and SNF 82

IV.15Improved Parameter Identification Under Noisy Observations with SNF . . 83
IV.16Architecture of amortized state estimation with normalizing flows. 86
IV.17Architecture of Latent State Space Model Flow. 87

List of Tables

II.1 Notation Summary 10
II.2 Alignment between MH Algorithm and Autonomous Rover Mapping 13
II.3 Two Strategies for Parameter Identification Using GPR 20

III.1 Normalizing Flows in Reinforcement Learning 55

IV.1 Training paradigms and inference across Deep Generative Models (DGMs). 60
IV.2 Performance Benchmarking of Deep Bayesian Inference Methods 62
IV.3 Multimodal Modeling Comparison 82

Chapter I

Introduction

1 Motivation

In this seemingly certain world, uncertainty lingers like a shadow, yet from the uncertain, we often find certainty.

The paradoxical relationship between certainty and uncertainty manifests across disciplines, demonstrating how truth often arises from navigating ambiguity. Quantum mechanics redefines predictability under randomness in the subatomic world, where Newtonian deterministic physics collapses. This uncertainty led to groundbreaking laws (e.g. Heisenberg's Uncertainty Principle), which now underpin modern technologies like quantum computing and all forms of spectroscopy. An example of medical diagnosis is that a patient presents vague symptoms, such as fatigue or pain, which could indicate anything from stress to cancer. Doctors converge on the cause through iterative hypothesis-testing approaches. Climate science also epitomizes how we manage uncertainty. Climate models incorporate a wide range of sources of variability and uncertainty. A common way to address it is to combine the results of a collection of different simulation models. By running multi-model ensembles, scientists can identify consistent patterns across the uncertainty.

From quantum randomness to medical diagnosis to climate models, uncertainty does not destroy the deterministic systems, but provides us accessibility to identify the systems. This perspective supports the use of probabilistic inference methods, which offer a rigorous framework for representing uncertainty and reasoning from uncertainty. In statistics, modeling uncertainty of variables from given data samples, using a probability distribution rather than just a point estimate is known as inference *. The estimated distribution facilitates uncertainty quantification and provides a measure of reliability of the results, making it particularly well-suited for real-world applications.

Probabilistic inference grounded in Bayes' rule is known as ***Bayesian inference***.

*We note that "inference" is an overloaded term in this context. We distinguish between: (i) Bayesian Inference, the statistical goal of recovering the posterior distribution; (ii) Variational Inference, the mechanism to achieve Bayesian inference; (iii) Inference: the process that maps data to latent space in generative models, which is in constrast to generation; and (iv) Computational Inference, the deep learning engineering practice of executing the generation pass (for generative models meaning from latent to data) of a trained network on test data.

Bayes' rule interprets unknown random variables with beliefs (the prior). The beliefs are updated using observed data (likelihood), yielding the posterior distribution. A simplified formulation is:

$$\text{posterior} = \text{prior} \times \text{likelihood} \,/\, \text{total evidence (or marginal likelihood)}.$$

For example, we can illustrate Bayes' rule using the medical diagnosis scenario: the objective is to obtain the probability of a disease given a positive test result:

$$P(\text{Disease}|\text{Positive}) = \frac{P(\text{Positive}|\text{Disease}) \times P(\text{Disease})}{P(\text{Positive})},$$
$$P(\text{Positive}) = P(\text{Positive}|\text{Disease}) \times P(\text{Disease}) + P(\text{Positive}|\overline{\text{Disease}}) \times P(\overline{\text{Disease}}),$$

where $\overline{\text{Disease}}$ denotes "No Disease".

In a more general Bayesian context, inference is the process of obtaining the posterior distribution of target variables from observed data. This perspective extends naturally to modern machine learning, where unknown values, structures, and policies can all be understood as random variables and probability distributions (Murphy, 2022).

Especially in Deep Generative Models (DGMs), Bayesian inference plays an essential role (Murphy, 2023). Through introducing random variables to represent complex data in latent space, DGMs enable data synthesis from latent variables (generation) based on the recognition of underlying latent representation from data (inference). However, directly inferring the posterior distribution over the latent variables using Bayes' rule is generally intractable, since all possible values of the latent variables are rarely obtainable in complex and high-dimensional models. Addressing this challenge is essential to fully unfold the potential of DGMs. Variational Autoencoders (VAEs) (Kingma and Welling, 2019) tackle this problem by introducing an inference network as a tractable approximation to the posterior. Normalizing Flows (NFs) (Rezende and Mohamed, 2016) construct bijective transformations between observed data and latent variables, achieving exact computation of posteriors. Diffusion models (Ho et al., 2020) sidestep calculating the posterior distribution over latent variables by designing it as a simple Gaussian, and instead frame both inference and generation as an iterative denoising process from noise through learned transformations.

To sum up, DGMs are inherently probabilistic; through probabilistic modeling and inference, they are powerful in extracting latent information and representing uncertainty in complex data. These capabilities are proven in numerous applications in images, text, and speech. The same principles extend naturally to scientific and engineering domains. One prominent example is: how can DGMs empower identification and modeling of complex, stochastic, nonlinear dynamical systems? In control theory, the process of building mathematical models of dynamical systems from observed input-output data is

called ***system identification***. Interpreting system identification in probabilistic terms, ***dynamics modeling*** is inferring the posterior of the next states based on the previous states and system inputs, while ***parameter identification*** is estimating the posterior over system parameters given observations. Identifying systems in a probabilistic way enables uncertainty quantification, which is crucial for reliable and robust decision-making in real-world applications.

Classical methods for system identification rely on linearized models or pre-specified nonlinear structures. Model parameters are then estimated from observed data using estimation techniques. Subsequently, a validation process is employed to assess and select the most appropriate model structure. Although these methods offer strong physical interpretability and are often data-efficient, they exhibit several limitations: parameter estimation can be computationally inefficient with large amounts of data; results are sensitive to the choice of model structures and dependent on expert knowledge; uncertainty is difficult to handle and quantify; and they are generally inapplicable to partially observable systems and high-dimensional and raw data.

Despite the demand for uncertainty quantification, deep learning methods are suitable for system identification thanks to their strength in representing information from complex data, flexibility of function approximation, and scalability in high-dimensional data. Furthermore, their compatibility with gradient-based optimization enables efficient navigation of high-dimensional landscapes, allowing optimizers to find solutions nearly equivalent to the global optima. This provides a distinct advantage over classical methods, where low-dimensional parameter spaces are more prone to being trapped in poor local minima that fail to capture the underlying dynamics. Taking uncertainty quantification into account, a simple yet effective technique is Gaussian Process Regression (Särkkä, 2019), which involves assigning a Gaussian process prior to the latent regressor and performing statistical conditioning based on the available observations. This method has gained widespread adoption due to its non-parametric characteristic and adaptability. Nonetheless, its significant computational and memory requirements limit its feasibility when dealing with large datasets. Another straightforward approach to combining the advantages of deep learning methods and the possibility of uncertainty modeling is the Bayesian Neural Network (BNN) (Goan and Fookes, 2020). Unlike standard neural networks that produce point estimates, BNNs assign probability distributions over the network parameters, thereby inducing distributions for outputs. However, BNNs reveal expressiveness limitations because the approximate inference may not reliably capture true posterior behavior (Foong et al., 2020). Therefore, the adaptation and application of DGMs are essential for achieving rich representational capacity, flexible modeling frameworks, and the ability to effectively handle large-scale data.

Different DGMs have been investigated in system identification applications. Here, we

evaluate briefly three prominent frameworks: VAEs, NFs, and Diffusion Models. VAEs work by compressing complex data into a lower-dimensional latent space and then learning to reconstruct original data from compressed latent variables. This makes VAEs flexible to cooperate with time series modeling architectures such as Recurrent Neural Network (RNN) (Girin et al., 2021) and Nonlinear Autoregressive eXogenous (NARX) (Paniagua and López, 2024), which is beneficial in uncovering the time-dependent features from data. However, VAEs suffers from a well-known problem of shrinkage of variance (Asperti, 2020) and tends to converge to a degenerate local optimum, which is also known as posterior collapse (Wang et al., 2023). Unlike VAEs, NFs take a different approach by using a series of invertible transformations to turn a simple probability distribution into a complex one that matches the data. The requirement of invertibility restricts their choices of neural network architectures, which limits their direct applicability to time series modeling. On the other side, NFs provide exact and expressive density estimation capabilities. This means they can precisely calculate the mathematical probability of a specific outcome, making them exceptionally strong at quantifying uncertainty and enhancing the accuracy of probabilistic forecasts. Diffusion models have recently gained popularity by learning to generate high-quality and coherent data through a process of gradually reversing noise. They release the architectural constraints and manifest the strength in generating predictions. Nevertheless, their iterative data generation procedure results in substantial computational overhead. Balancing expressiveness, computational efficiency, and sample quality, ***normalizing flows*** are chosen in this work as the generative backbone of probabilistic system identification.

While parameter identification focuses on determining the time-invariant system parameters, ***state estimation*** is the process of inferring hidden, time-varying state variables from a history of noisy and potentially partial observations and known control inputs. Because of the noise in the sensors (measurement noise) and unmodeled disturbances in the system itself (process noise), state estimation is naturally probabilistic. Thus, estimating the target states is computing the posterior distribution of target states. The traditional method for state estimation is the Luenberger State Observer, which assumes the systems are deterministic and both the system dynamics and measurement models are linear and time-invariant. Taking stochasticity into account, Kalman Filter is the most widely applied method. Beyond system linearity, a Kalman Filter assumes the process and measurement noises are additive Gaussian noise with zero mean. In order to handle nonlinear systems, Extended Kalman Filter linearizes the nonlinear functions around the state estimate. However, the linearization introduces errors, leading to wrong estimation or even divergence. An improved modification is the Unscented Kalman Filter, which carefully selects the means and variances and propagates them through the nonlinear functions. However, it is still only compatible with Gaussian noises.

Through relaxing the requirement for rigid physical modeling and avoiding restrictive

assumptions about uncertainty—-while maintaining high expressiveness, modeling flexibility, and scalability to large datasets, the selected NF architecture establishes a foundation for both probabilistic system identification and state estimation. From a data-centric perspective, these tasks represent different mapping challenges: parameter identification maps time series to a set of static data points; state estimation involves inferring data points or reconstructing temporal trajectories from noisy observations. In order to address these challenges, it is necessary to extend the standard normalizing flows, given the limited research currently available in this area. Bridging this gap not only broadens the applicability of NFs but also contributes to advancing Bayesian inference techniques for complex, time-varying systems.

Having explored system identification and state estimation through Bayesian inference, we can advance toward ***optimal control***. By leveraging data as the primary source of insight into system behavior, we enable control strategies that are more adaptive, scalable, and robust. This is especially advantageous in situations where analytical models are incomplete, uncertain, or impractical to derive. One direct application is model-free control, which obtains control inputs directly from observations and reference trajectories without requiring an explicit system model. This architecture is especially effective for real-time path planning and tracking. By bypassing complex analytical derivations, it allows for the rapid evaluation of numerous candidate trajectories, enabling the system to select the best path under time-sensitive constraints. In recent literature, diffusion models have attracted widespread attention for representing control policies due to their ability to generate high-quality, trajectories under multimodal observations. However, while diffusion models excel in sample quality, normalizing flows are comparably more efficient and rigorous because of their capacity for exact density estimation. Despite these advantages, the use of NFs as a generative backbone for control remains largely underexplored. Building upon model-free principles, Reinforcement Learning (RL) extends this paradigm by optimizing these stochastic policies through continuous interaction with the environment. RL agents learn to maximize long-term rewards while adapting to system uncertainties, effectively enhancing the capabilities of traditional Model Predictive Control (MPC). Integrating RL with NFs offers a powerful framework for achieving uncertainty-aware, robust, and self-improving control in dynamic environments.

In summary, this work is motivated by three key considerations:

- Bayesian inference provides a principled framework for modeling uncertainty and extracting knowledge from uncertainty, enabling system identification and state estimation based solely on observations. This is particularly valuable in scenarios where first-principles models are unavailable, incomplete, or prohibitively complex.

- While Normalizing Flows (NFs) offer the expressiveness, exact density estimation,

and scalability required for complex dynamical systems, their application to system identification and state estimation remains significantly underexplored. Current NF architectures are often not natively designed for temporal dependencies; therefore, they must be specifically adapted to handle sequential data and hidden state trajectories to be effective for these tasks.

- Leveraging Normalizing Flows (NFs) to learn stochastic control policies and refining them through reinforcement learning enables adaptive, uncertainty-aware decision-making in complex, unpredictable environments. This approach has not been fully investigated and offers significant potential for advancing control strategies.

2 Objectives and Contributions

The center aim of this dissertation is to leverage the expressiveness, computational efficiency, and mathematical rigor of normalizing flows to develop a unified, data-driven framework for probabilistic system identification, state estimation, and optimal control. The primary objectives of this dissertation are as follows:

- To investigate the efficacy and limitation of normalizing flows in identifying parameters under system observations.
- To adapt and extend normalizing flow architectures for sequential data processing, enabling the estimation of hidden, time-varying states in stochastic dynamical systems.
- To formulate and evaluate normalizing flows as generative control policies, exploring their potential to provide uncertainty-aware decision-making within optimal control and reinforcement learning frameworks.

Building upon these objectives, the key contributions of this dissertation are:

- Real-World Performance Assessment: A comprehensive examination of a normalizing flow-based Bayesian inference framework in real-world applications, evaluating its robustness, generalizability, and computational advantages over classical estimation techniques.
- Sensitivity-Guided Inference: The integration of sensitivity analysis with normalizing flows to enable reliable parameter identification in systems where parameters exhibit varying levels of identifiability.
- Stochastic BayesFlow: The development of a hybrid framework that combines stochastic components with normalizing flows to achieve robust parameter identification under observational and system uncertainty.

- FlowSSM: The proposal of a novel state-space modeling method that leverages normalizing flows to capture complex, non-Gaussian uncertainty trajectories in stochastic dynamical systems.
- IMFlow for Robust Control: The extension of normalizing flows to represent stochastic control policies, introducing a framework for integrating generative models directly into the optimal control and reinforcement learning loop.

3 Outline

The remainder of this dissertation is organized as follows:

- Chapter II (Backgrounds and Preliminaries) establishes the foundations required for this work. It introduces the principles of Bayesian inference, the mechanics of deep generative models, particularly the normalizing flows, and the connection from Bayesian inference to control methods.
- Chapter III (Literature Review) provides a comprehensive survey of the current state of the art about applying deep generative models in parameter identification, state estimation, and control technologies. Through this review, the specific research gaps addressed by this dissertation are identified.
- Chapter IV (Methodology and Numerical Validation) details the core technical frameworks developed in this work. These methodologies are categorized into three primary research pillars: parameter identification, state estimation, and optimal control. Each method is accompanied by numerical experiments and synthetic benchmarks to provide theoretical proof of concept and demonstrate the efficacy of the proposed algorithms.
- Chapter V (Experiments and Results) presents a series of benchmarks and real-world case studies designed to evaluate the proposed frameworks. This chapter validates the performance of the developed methods in terms of accuracy, computational efficiency, and robustness to noise, following the same three-pillar structure as the methodology.
- Chapter VI (Conclusion and Future Work) synthesizes the key findings of this research, reflects on the contributions made to the fields of system identification, state estimation, and optimal control, and outlines potential directions for future research.

Chapter II

Background and Preliminaries

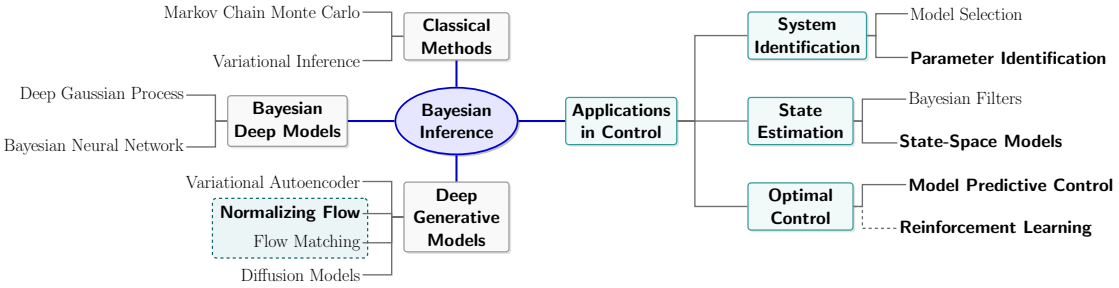


Figure II.1: Conceptual map illustrating the integration of Bayesian inference with deep generative modeling and its applications in control engineering. The left side categorizes the modeling paradigms employed, moving from classical statistical methods to modern deep generative architectures. Notably, Normalizing Flows and Flow Matching are grouped together due to their shared theoretical foundation in continuous-time differential transformations and exact density estimation. On the right, the framework is mapped to core control tasks: system identification, state estimation, and optimal control. A dashed line connects Reinforcement Learning and Optimal Control, signifying the modern methodological convergence of these two disciplines where data-driven learning is increasingly viewed as an adaptive realization of classical optimal control theory.

This section introduces the necessary foundational concepts and methodologies for understanding the rest of this work. Before the main contents, we explain the unified notations applied in this whole work. We first review classical Bayesian inference methods. Then we introduce deep probabilistic methods, including Deep Gaussian Process (DGP), Bayesian Neural Network (BNN), and Variational Autoencoder (VAE). Next, we present the core methods of this work, Normalizing Flows (NFs), a powerful class of generative models for flexible density estimation. Finally, we bridge Bayesian inference to control methods, covering system identification, state estimation, and model predictive control. The overview for this chapter is shown in Figure II.1.

1 Notations

To maintain clarity throughout this document, we adopt the following notation:

Category	Description
Basic Symbols	Scalars: x; Vectors: $\boldsymbol{x}$; Matrices: $\boldsymbol{X}$; Sets: $\mathcal{D}$
Random Variables	Multidimensional random variables: $\boldsymbol{x} = [x_1, \ldots, x_D]^T$; Sample index: $\boldsymbol{x}^{(i)}$; Stochastic processes: $X(t)$
Time Series	Time series of length T: $\boldsymbol{x}_{(1:T)}$; Time-dependent variables: $\boldsymbol{x}(t)$ or $x(t)$; Dimension index: $\boldsymbol{x}_i$
Variable Modifications	Estimated: $\hat{\boldsymbol{x}}$; Transformed: $\tilde{\boldsymbol{x}}$; Time derivative: $\dot{\boldsymbol{x}}$
Probability and Statistics	PDF: $p(\cdot)$; Joint PDF: $p(\cdot, \cdot)$; Conditional PDF: $p(\cdot \mid \cdot)$; Expectation: $\mathbb{E}_{p(x)}[f(x)]$; Covariance: $\mathrm{Cov}[\boldsymbol{x}, \boldsymbol{y}]$; Variance: $\mathrm{Var}[\boldsymbol{x}]$
Matrix Operations	Identity: $\boldsymbol{I}$; Jacobian: $\boldsymbol{J}$; Determinant: $\det \lvert\boldsymbol{J}\rvert$; Trace: $\mathrm{Tr}(\boldsymbol{X})$; Transpose: $\boldsymbol{X}^\top$

Table II.1: Notation Summary

2 Bayesian Inference

Bayesian inference gets its name from using Bayes' rule to perform statistical inference of probability distributions. This chapter refers to *Bayesian Data Analysis* (Gelman et al., 2013), *Handbook of MCMC* (Allen, 2007), and *Variational Inference* (Blei et al., 2017).

2.1 Bayes' Rule

Bayes' rule is derived from the chain rule of conditional probability. For target variables $\boldsymbol{\theta}$ and the corresponding observed data $\mathcal{D}$, the chain rule states that:

$$p(\boldsymbol{\theta}, \mathcal{D}) = p(\boldsymbol{\theta}|\mathcal{D})p(\mathcal{D}) = p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}). \tag{II.1}$$

Rearranging the terms gives Bayes' rule or Bayes' theorem:

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}. \tag{II.2}$$

In Bayes' theorem, $p(\boldsymbol{\theta})$ is the initial distribution of the variables before any data samples are considered, which is called the *prior distribution*. $p(\boldsymbol{\theta}|\mathcal{D})$ is the posterior distribution of the target variables after being updated by the observed data. $p(\mathcal{D}|\boldsymbol{\theta})$ is the likelihood of expected data given the setting of parameters $\boldsymbol{\theta}$. $p(\mathcal{D})$ is called the *marginal likelihood* or *evidence* of expected data across all possible variable settings:

$$p(\mathcal{D}) = \Sigma_{\boldsymbol{\theta}'} p(D|\boldsymbol{\theta}')p(\boldsymbol{\theta}')\Delta\boldsymbol{\theta}', \tag{II.3a}$$

$$p(\mathcal{D}) = \int p(D|\boldsymbol{\theta}')p(\boldsymbol{\theta}')d\boldsymbol{\theta}'. \tag{II.3b}$$

In practice, it is often treated as a normalizing constant and ignored. The unnormalized posterior is then given by:

$$p_u(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}). \tag{II.4}$$

Normally, the posterior distribution does not have a closed-form expression. Instead, we approximate it using numerical methods. Computational algorithms that rely on repeated random sampling to obtain numerical results refers to "Monte Carlo" methods. Monte Carlo methods provide ways to approximate the posterior distribution by expectations:

$$E = \mathbb{E}_{p(\boldsymbol{\theta}|\mathcal{D})}[f(\boldsymbol{\theta})] = \int f(\boldsymbol{\theta})p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta}, \tag{II.5}$$

where $f(\boldsymbol{\theta})$ is any function of interest, for example, $f(\boldsymbol{\theta}) = \boldsymbol{\theta}$. As the simplest way, we can approximate it using standard numerical techniques such as a Riemann sum over a discrete grid of points:

$$E \approx \Sigma_{\boldsymbol{\theta}'} f(\boldsymbol{\theta}')\Delta\boldsymbol{\theta}'. \tag{II.6}$$

However, this approach is only feasible for low-dimensional $\boldsymbol{\theta}$ due to the curse of dimensionality. Besides, for complex distributions, the discretization is highly inefficient. Therefore, for high-dimensional parameters and complex distributions, we often resort to sampling-based or optimization-based methods to approximate the posterior distribution, which will be discussed in the following sections.

2.2 Markov Chain Monte Carlo

The "Markov Chain" aspect indicates that these algorithms generate a sequence of samples where each sample depends only on the previous one. In contrast to the basic Monte Carlo methods, Markov Chain Monte Carlo (MCMC) methods produce auto-correlated samples, so that each new sample adds less new information than an independent draw would. Different sequences updating algorithms lead to different MCMC methods. The goal of MCMC is to construct a Markov chain that has the desired posterior distribution

as its *equilibrium* distribution. After running the chain for a sufficient number of iterations, the samples generated from the chain can be treated as samples from the target posterior distribution.

This concept is analogous to the steady-state behavior of a dynamical system. Typically, a dynamical system undergoes a transient phase before settling into an equilibrium where the states remain constant. In MCMC, while the variables continue to move, their probability distribution reaches a steady state. If we interpret the probability density as the energy landscape of a physical system, equilibrium is reached when the system has settled into a stable statistical configuration. At this point, although individual samples continue to transition between different regions of the parameter space, the net movement between these regions is zero. This balance ensures that the density of samples remains constant over time, allowing the chain to accurately represent the invariant target posterior distribution.

2.2.1 Metropolis-Hastings Algorithm

The Metropolis-Hastings (MH) algorithm, which is named after Metropolis (1953) and Hastings (1970), is a widely used MCMC method. To understand its logic, consider the "Autonomous Rover Mapping" analogy: Imagine an autonomous rover equipped with a noisy sensor is deployed into a dark and unmapped crater. Its task is to create a high-resolution density map of water-ice concentrations across the crater floor. It doesn't have a satellite map; it only knows the signal strength of ice at its current coordinates using a neutron spectrometer. It cannot teleport and thus has to move from point to point in order to take readings. The density map is essentially a confidence map of where the ice is located ($\boldsymbol{\theta}$) given the sensor observations ($\mathcal{D}$). A alignment between the MH algorithm and its analogy is shown in Table II.2.

Although this procedure allows flexible approximation of complex posterior distribution, it suffers from slow convergence rate due to its random exploration nature. Thus, gradient-based MCMC methods are proposed to incorporate the structure of the target density, leading to faster sampling.

2.2.2 Metropolis-Adjusted Langevin Algorithm

The Metropolis-adjusted Langevin algorithm (MALA) (Besag, 2004) combines the MH algorithm with Langevin diffusion, whose stationary distribution is the target density. In this framework, the rover no longer jumps blindly; instead, it uses its onboard sensors to detect the local slope of the resource distribution. Its movement is then governed by a first-order control law consisting of two parts: a deterministic drift that pushes the rover uphill toward higher density, and a stochastic diffusion (Gaussian noise) that ensures the rover continues to explore rather than getting stuck at a single peak.

MH Algorithm (Random Walk)	**Autonomous Rover Mapping**
1. Initialization Set starting point $\boldsymbol{\theta}_{(0)}$. Define proposal $q(\boldsymbol{\theta}'\|\boldsymbol{\theta})$, which is usually a Gaussian $\mathcal{N}(\boldsymbol{\theta}, \sigma^2)$.	**1. Power-On & Config** The rover starts at GPS coordinate $\boldsymbol{\theta}_{(0)}$. Set the Jogging Logic (q): a random-walk jump distance based on motor constraints.
2. Iteration Loop (for each i):	**2. Navigation Duty Cycle**:
(a) Propose a new state: Generate a candidate $\boldsymbol{\theta}'_{(i)}$ using the proposal distribution $q(\boldsymbol{\theta}'_{(i)}\|\boldsymbol{\theta}_{(i-1)})$.	*(a) Candidate Path Planning:* A phantom coordinate $\boldsymbol{\theta}'_{(i)}$ nearby is calculated. This is a potential move, but the actuators are not yet engaged.
(b) Calculate Transition Probability $T(\boldsymbol{\theta}'_{(i)}\|\boldsymbol{\theta}_{(i-1)})$*:* $T(\boldsymbol{\theta}'_{(i)}\|\boldsymbol{\theta}_{(i-1)}) = \min\left(1, \frac{p(\boldsymbol{\theta}'_{(i)} \mid \mathcal{D})\, q(\boldsymbol{\theta}_{(i-1)} \mid \boldsymbol{\theta}'_{(i)})}{p(\boldsymbol{\theta}_{(i-1)} \mid \mathcal{D})\, q(\boldsymbol{\theta}'_{(i)} \mid \boldsymbol{\theta}_{(i-1)})}\right)$ (II.7)	*(b) Sensor Evaluation based on Data* $\mathcal{D}$*:* The rover asks: "Based on our sensor history ($\mathcal{D}$), is the ice likely denser at the phantom spot $\boldsymbol{\theta}'$ than my current spot $\boldsymbol{\theta}$?" It also checks for mechanical bias in q.
(c) Update via Accept/Reject: $\boldsymbol{\theta}_{(i)} = \begin{cases} \boldsymbol{\theta}'_{(i)}, & \text{if } u < T \\ \boldsymbol{\theta}_{(i-1)}, & \text{otherwise} \end{cases}$ (II.8)	*(c) Actuator Actuation:* If the spot is "better," the motors drive there. If "worse," it rolls a die: it might still move there to explore, or stay put and record the current spot again.
3. Accumulation Repeat for N iterations to obtain a sequence of samples $\{\boldsymbol{\theta}_{(1)}, \ldots, \boldsymbol{\theta}_{(N)}\}$.	**3. Mission Duration** The rover repeats this loop N times. Every cycle, it writes its final (x, y) coordinate for that minute into the mission logbook.
4. Estimator $p(\boldsymbol{\theta}\|\mathcal{D}) \approx \frac{1}{N}\sum_{n=1}^{N} \delta_{\boldsymbol{\theta}_{(n)}}(\boldsymbol{\theta})$ $\delta_{\boldsymbol{\theta}_{(n)}}(\boldsymbol{\theta})$ denotes a dirac function of the query $\boldsymbol{\theta}$ at the target $\boldsymbol{\theta}_{(n)}$.	**4. Resource Density Map** Download the log and plot the coordinates. The resulting point cloud density shows the ice distribution $p(\boldsymbol{\theta}\|\mathcal{D})$ across the crater.

Table II.2: Alignment between the Metropolis-Hastings (MH) algorithm and an autonomous rover mapping.

In general, for a stochastic process $\Theta(t)$ for variables $\boldsymbol{\theta}$, the Langevin diffusion is described by the following Stochastic Differential Equation (SDE):

$$d\Theta(t) = \nabla \log p(\Theta(t))dt + \sqrt{2}dW(t), \tag{II.9}$$

where $W(t)$ denotes a standard Wiener process (Gabbiani and Cox, 2010). The Wiener process is a continuous-time stochastic process with independent, normally distributed increments, which can be interpreted as Brownian motion. It has the following properties:

(1) $W(0) = \mathbf{0}$;

(2) for $t > s \geq 0$, $W(t) - W(s) \sim \sqrt{t-s}\,\mathcal{N}(\mathbf{0}, \boldsymbol{I})$;

(3) for $t_1 > s_1 \geq 0$, and $t_2 > s_2 \geq 0$, $W(t_1) - W(s_1)$ and $W(t_2) - W(s_2)$ are independent.

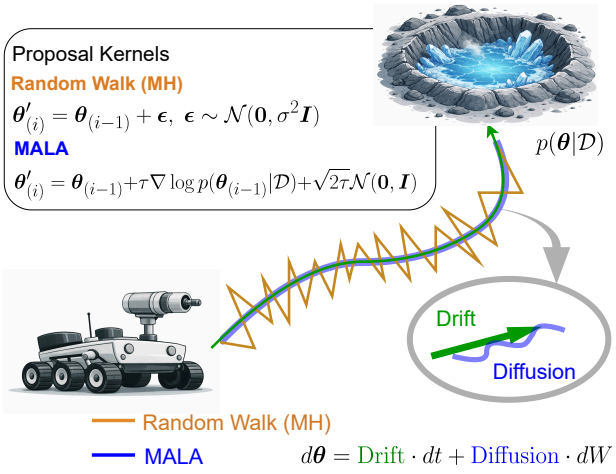


Figure II.2: An illustration of MH algorithm and MALA using the autonomous rover as a conceptual sampler. The crater representing $p(\boldsymbol{\theta}|\boldsymbol{\mathcal{D}})$ serves as the objective landscape where the sampler aims to concentrate its states. The Random Walk (MH) is shown as the jagged orange path, while the MALA is represented by a smoother green path (drift) along with a blue band (diffusion).

A comparison between the MH algorithm and MALA is illustrated in Figure II.2. The MALA uses the first order Euler-Maruyama discretization of the SDE, resulting in the following proposal distribution:

$$\Theta(t) = \Theta(i-1) + \tau \nabla \log p(\Theta(i-1)) + \sqrt{2\tau}\,\mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{II.10}$$

where τ is a step size parameter. Because of the discretization, convergence to the target density is no longer guaranteed. Therefore, the MH algorithm is applied to correct the bias introduced by the discretization. For each iteration i, the MALA updates the state as follows (corresponding to the iteration part in Table II.2):

(a) **Guidance with Gradient:** Unlike the uninformed MH, which only considers the current position, MALA first computes the gradient of the log-posterior: $\nabla \log p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D})$. This determines the drift direction toward regions of higher probability.

(b) **Informed Proposal:** Propose a new state $\boldsymbol{\theta}'_{(i)}$ using the Langevin dynamics (II.10): $\boldsymbol{\theta}'_{(i)} = \boldsymbol{\theta}_{(i-1)} + \tau \nabla \log p(\boldsymbol{\theta}_{(i-1)}) + \sqrt{2\tau}\, \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.

(c) **Transition Probability:** Calculate the transition probability according to Equation (II.7). Specifically, the proposal distribution is derived from the Langevin dynamics:

$$q(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)}) \propto \exp(-\frac{1}{4\tau} \left\| \boldsymbol{\theta}'_{(i)} - \boldsymbol{\theta}_{(i-1)} - \tau \nabla \log p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D}) \right\|_2^2). \tag{II.11}$$

(d) **Correction (Accept/Reject):** Using the same mechanism as MH, a uniform random number $u \sim U(0,1)$ is generated. Then, the state $\boldsymbol{\theta}_{(i)}$ is updated by the accepting ratio according to Equation (II.8).

Compared to the MH, MALA converges faster and scales better in higher dimensions, at the cost of requiring gradients and tuning. As a more advanced MCMC method, Hamiltonian Monte Carlo (HMC) (Duane et al., 1987) further improves sampling efficiency by simulating the whole sampling trajectory as Hamiltonian dynamics.

2.3 Variational Inference

Different from MCMC, Variational Inference (VI) (Jordan et al., 1998; Wainwright and Jordan, 2008) is an optimization-based method that approximates the target posterior distribution $p(\boldsymbol{\theta}|\mathcal{D})$ with a simpler, parameterized distribution $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$, where $\boldsymbol{\phi}$ are the variational factors. The goal of VI is to find the optimal variational factors $\boldsymbol{\phi}^*$ that minimize the distance between the approximate distribution and the true posterior. The distance is normally measured by the Kullback-Leibler (KL)-divergence (Kullback and Leibler, 1951), which is defined as:

$$D_{\mathrm{KL}}(q(z)||p(z)) = \mathbb{E}_{q(z)}[\log \frac{q(z)}{p(z)}] = \int q(z) \log \frac{q(z)}{p(z)} dz. \tag{II.12}$$

Thus, the optimization problem of VI can be formulated as:

$$\boldsymbol{\phi}^* = \arg\min_{\boldsymbol{\phi}} D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})). \tag{II.13}$$

Substituting Bayes' rule (II.2) into KL-divergence, we have:

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) \,||\, \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}). \tag{II.14}$$

The marginal likelihood $p(\mathcal{D})$ is usually intractable, as it involves integrating over all possible parameter settings. Therefore, the divergence cannot be computed directly.

Expanding the above equation with the definition of KL-divergence, we have:

$$\begin{aligned} D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})}[\log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \log p(\mathcal{D}|\boldsymbol{\theta}) - \log p(\boldsymbol{\theta}) + \log p(\mathcal{D})] \\ &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}) + \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}). \end{aligned} \tag{II.15}$$

Since $p(\mathcal{D})$ is independent of the variational factors $\boldsymbol{\phi}$, the last term is reduced to $\log p(\mathcal{D})$. Combining the terms $p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) = p(\boldsymbol{\theta}, \mathcal{D})$ gives

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) + \log p(\mathcal{D}) \tag{II.16}$$

Or combining the terms $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$ and $p(\boldsymbol{\theta})$ via KL-divergence gives:

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta})) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\mathcal{D}) \tag{II.17}$$

Rearranging the terms in Equation (II.16) gives:

$$\log p(\mathcal{D}) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) + \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}). \tag{II.18}$$

Since the KL-divergence is always non-negative, we have:

$$\log p(\mathcal{D}) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}). \tag{II.19}$$

Because of this inequality, the right hand side is called the Evidence Lower Bound (ELBO), also known as the variational lower bound. Another way to formulate the ELBO using Equation (II.17) is:

$$\mathrm{ELBO} = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) - D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta})). \tag{II.20}$$

Then, minimizing the KL-divergence is equivalent to maximizing the ELBO (II.19), leading to the equivalent optimization problem:

$$\boldsymbol{\phi}^* = \arg\max_{\boldsymbol{\phi}} \mathrm{ELBO} = \arg\min_{\boldsymbol{\phi}} [\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D})]. \tag{II.21}$$

To complete the optimization, we need to choose a suitable family of variational distributions $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$. A common choice is the mean-field family (Bishop, 2006), which assumes that the variables $\boldsymbol{\theta}$ are mutually independent and each governed by a distinct factor under q:

$$q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) = \Pi_i q_{\phi_i}(\theta_i), \tag{II.22}$$

where each factor ϕ_i governs a variable θ_i. Then, the ELBO (II.19) is iteratively optimized by updating each factor while holding the others fixed. The solution of the optimal distribution $q_{\phi_i}(\theta_i)$ is proportional to the exponential expected log of the complete condition:

$$\log q^*_{\phi_i}(\theta_i) \propto \mathbb{E}_{\Pi_{j\neq i} q_{\phi_i}(\theta_i)}[\log p(\theta_i|\boldsymbol{\theta}_{j\neq i}, \mathcal{D})], \tag{II.23}$$

where the expectation is taken with respect to the distributions of all other groups $j \neq i$. This algorithm is called Coordinate Ascent Variational Inference (CAVI) (Bishop, 2006).

The assumption of the mean-field VI simplifies the optimization problem, and thus makes it scalable for large dimensional variables and also enables parallelizable computation. However, it may also limit the expressiveness of the variational distribution, especially when the posterior correlations are crucial, leading to suboptimal approximations of the true posterior. Summarizing, VI is generally faster than MCMC and scales better to large datasets, but it may provide less accurate approximations of the posterior distribution (Blei et al., 2017).

3 Deep Probabilistic Methods

The classical methods, MCMC and VI, although provide general solution for Bayesian inference, they are limited by their "approximation-per-point" behavior. Deep probabilistic methods, on the contrary, enable amortized Bayesian inference and unlock both high expressiveness and scalability to large datasets. In the following section, we briefly review three prominent deep probabilistic methods: the Deep Gaussian Process, the Bayesian Neural Network, and the Variational Autoencoder. The former two methods are direct extensions of classical Bayesian inference with deep architectures, while the last one is a fundamental deep generative model that leverages variational inference.

3.1 Deep Gaussian Process

A Gaussian process can be interpreted as an infinite-dimensional Gaussian distribution. It is fully specified by a mean function $m(\cdot)$ and a covariance (kernel) function $k(\cdot, \cdot)$. For simplification, we consider a single-variable case where the variable is x and the corresponding Gaussian process is $X(t)$. We collect M observations at discrete input

points $\boldsymbol{x} = [x_1, \ldots, x_M]^\top$, the associated random variables follow a multivariate normal distribution:

$$\begin{bmatrix} X(t_1) \\ \vdots \\ X(t_M) \end{bmatrix} \sim \mathcal{N}\left(\boldsymbol{\mu} = \begin{bmatrix} \mu(t_1) \\ \vdots \\ \mu(t_M) \end{bmatrix}, \boldsymbol{K} = \begin{bmatrix} k(t_1,t_1) & \ldots & k(t_1,t_M) \\ \vdots & \ddots & \vdots \\ k(t_M,t_1) & \ldots & k(t_M,t_M) \end{bmatrix} \right). \tag{II.24}$$

Note that the time here is not really temporal but indicates the coordinates of variables in the parameter domain. For example, the t_i-th observation in the dataset is x_i. Thus, in the following, we call t_i an index, instead of a time point. Using a Gaussian process to infer variables from observed data based on prior distributions leads to Gaussian process regression (GPR) (Rasmussen and Williams, 2008). In comparison to MCMC, GPR aims to directly approximate the target distribution function with a Gaussian-based surrogate model instead of sampling them iteratively.

3.1.1 Gaussian Process Regression

We assume that we observe noisy measurements $\boldsymbol{y} = [y_1, \ldots, y_M]^\top$ at inputs $\boldsymbol{x}$. The observation model relates the latent function $f(\cdot)$ to the observations via additive Gaussian noise:

$$y_i = f(x_i) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2). \tag{II.25}$$

The likelihood $p(y_i|f(x_i))$ is thus given by:

$$p(y_i|f(x_i)) = \mathcal{N}(y_i|f(x_i), \sigma_\epsilon^2). \tag{II.26}$$

The prior distribution over functions $f(\cdot)$ is assumed to be a zero-mean Gaussian process:

$$f(\cdot) \sim \mathcal{GP}(0, k(\cdot,\cdot)). \tag{II.27}$$

The kernel function $k(\cdot,\cdot)$ is usually chosen to be the squared exponential (SE) kernel function, or as known as the Radial Basis Function (RBF), defined as:

$$k(x_i, x_j) = \sigma_{SE}^2 \exp(-\frac{1}{2\ell^2} \|x_i - x_j\|^2), \tag{II.28}$$

where σ_{SE} and ℓ are vertical and horizontal scale parameters, which control the vertical span and smoothness of the function, respectively.

To obtain the optimal regression performance, the hyperparameters $\varphi = \{\sigma_{SE}, \ell, \sigma_\epsilon\}$ must be optimized. In GPR, this is achieved by maximizing the log marginal likelihood. By integrating out the latent function values f, the marginal distribution of the data $\boldsymbol{y}$

depends only on the inputs $\boldsymbol{x}$ and hyperparameters φ:

$$p(\boldsymbol{y}|\boldsymbol{x},\varphi) = \mathcal{N}(\boldsymbol{y}|\boldsymbol{0}, \boldsymbol{K} + \sigma_\epsilon^2 \boldsymbol{I}). \tag{II.29}$$

The optimization objective $\mathcal{J}(\varphi)$ is the logarithm of this marginal likelihood:

$$\mathcal{J}(\varphi) = \log p(\boldsymbol{y}|\boldsymbol{x},\varphi) = \underbrace{-\frac{1}{2}\boldsymbol{y}^\top (\boldsymbol{K} + \sigma_\epsilon^2 \boldsymbol{I})^{-1}\boldsymbol{y}}_{\text{Data Fit}} \underbrace{-\frac{1}{2}\log|\boldsymbol{K} + \sigma_\epsilon^2 \boldsymbol{I}|}_{\text{Complexity Penalty}} - \frac{M}{2}\log 2\pi. \tag{II.30}$$

The optimal hyperparameters $\hat{\varphi}$ are found by maximizing this function, typically using gradient-based optimizers.

The GPR is widely used for regression tasks owing to its non-parametric nature and its capacity to quantify uncertainty. However, its computational complexity scales cubically with the number of training points, i.e., $\mathcal{O}(D^3)$ for an D-dimensional input. Additionally, calculating the Gaussian processes requires the entire storage of training datasets, incurring a memory cost of $\mathcal{O}(M^2)$, which grows quadratically with the dataset size M. These limitations hinder its scalability to high-dimensional inputs and large datasets. In addition, the expressiveness of GPR is constrained by the choice of kernel function and its incapacity to perform hierarchical feature extraction.

3.1.2 Parameter Identification using GPR

In the context of system identification, we aim to estimate the posterior distribution of system parameters given observed system data, denoted as $p(\boldsymbol{\theta}|\mathcal{D})$. There are two primary strategies to achieve this, which are compared in Table II.3.

3.1.3 Deep Gaussian Process

To improve expressiveness, Deep Gaussian Process (DGP) (Damianou and Lawrence, 2013) is proposed as multilayer hierarchical generalizations of Gaussian processes. The input $\boldsymbol{x}$ is passed through multiple layers of Gaussian processes to obtain the output $\boldsymbol{y}$. For an L-layer DGP, the model extends the Gaussian process (II.25) by:

$$y_i = f_L(f_{L-1}(\dots f_1(x_i) + \epsilon_{i,1} \dots)) + \epsilon_L, \quad f_l(\cdot) \sim \mathcal{GP}(0, k_l(\cdot,\cdot)), \\ l = 1,\dots,L,\ i = 1,\dots,M. \tag{II.31}$$

The deep architecture enables Gaussian processes to learn more complex data patterns and hierarchical representations. It can be seen as a deep neural network with infinitely wide layers (Neal, 1996), where each layer applies a Gaussian process, instead of nonlinear

Feature	**Direct Inverse Regression** (Li et al., 2025a)	**Surrogate-Based Inference (Forward)** (Takhtaganov and Müller, 2018)
Core Concept	The GPR learns the inverse mapping directly from observation space to parameter space, acting as amortized Bayesian inference.	The GPR acts as a surrogate for the physical simulator to approximate the likelihood function, accelerating the simulation process. The posterior distribution of the parameters is obtained by MCMC.
GP Input ($\boldsymbol{x}$)	The observed system data $\mathcal{D}$ (or extracted feature vector).	The system parameter $\boldsymbol{\theta}$.
GP Output ($\boldsymbol{y}$)	The estimated system parameter $\boldsymbol{\theta}$.	The simulated system response $\hat{\boldsymbol{y}} = \mathrm{Sim}(\boldsymbol{\theta})$.
Posterior $p(\boldsymbol{\theta}\|\mathcal{D})$	Obtained immediately from the GP's predictive distribution.	Obtained via Bayes' rule using the GP as the likelihood term: $p(\boldsymbol{\theta}\|\mathcal{D}) \propto p(\mathcal{D}\|\boldsymbol{\theta})p(\boldsymbol{\theta})$.
Pros & Cons	**Pros:** Computationally fast at inference time. **Cons:** Ill-posed if the mapping is non-unique (multiple $\boldsymbol{\theta}$ produce same $\mathcal{D}$).	**Pros:** Scientifically rigorous; correctly handles multi-modal distributions and uncertainty. **Cons:** Requires iterative sampling (MCMC).

Table II.3: Comparison of Gaussian Process strategies for parameter identification.

matrix transformation. However, due to the computational cost and storage consumption the DGP is still cumbersome to apply in practice.

3.2 Bayesian Neural Network

Another approach of deep probabilistic methods involves integrating Bayesian inference with deep neural networks. A Bayesian Neural Network (BNN) (Neal, 1996; MacKay, 1995) places prior distributions over the network's weights and biases, allowing uncertainty to be modeled directly within the architecture.

The goal of training a BNN is to infer the posterior distribution over the parameters $\boldsymbol{\theta} = [\boldsymbol{w}, \boldsymbol{b}]$ given the training data $\mathcal{D}$, i.e., $p(\boldsymbol{\theta}|\mathcal{D})$. VI is commonly employed to approximate the posterior by optimizing the ELBO (II.20). Here, we inherit the notation from VI. The parameters $\boldsymbol{\theta}$ are not the to be identified system parameters.

To facilitate gradient-based optimization via backpropagation—similar to standard neural networks—the reparameterization trick (Kingma and Welling, 2022) is used. This technique expresses random variables as deterministic variables of noise, enabling efficient

and differentiable sampling during training. For example, for a Gaussian variational distribution $q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}|\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$, we can reparameterize $\boldsymbol{\theta}$ as:

$$\boldsymbol{\theta} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{II.32}$$

where $\odot$ represents the Hadamard product, i.e., the element-wise product, and the variational parameters here are $\boldsymbol{\phi} = [\boldsymbol{\mu}, \boldsymbol{\sigma}]$. To train a BNN on large datasets, Stochastic Gradient Descent (SGD) (Bottou, 1999) is typically employed to optimize the ELBO. Unlike standard gradient descent, SGD estimates the gradient at each iteration using a randomly selected subset of the data. Then, the variational objective of a BNN using the ELBO (II.20) becomes:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \frac{1}{M} \sum_{i=1}^{M} [D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta})) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}_i|\boldsymbol{\theta})], \tag{II.33}$$

where each mini-batch $\mathcal{D}_i \subset \mathcal{D}$ is drawn from the equally sized subsets of the training data $\mathcal{D} = \{\mathcal{D}_1, \ldots, \mathcal{D}_M\}$. During each optimization step, the auxiliary noise $\boldsymbol{\epsilon}$ is sampled multiple times from $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ to perform Monte Carlo estimation of the expectation terms in the loss function $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi})$. The gradient $\nabla_{\boldsymbol{\phi}} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi})$ is then computed via backpropagation and averaged across all samples. Finally, the variational parameter $\boldsymbol{\phi}$ is updated by

$$\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} - \alpha \nabla_{\boldsymbol{\phi}} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}), \tag{II.34}$$

where α is the learning rate. This procedure is called Bayes by Backprop (BbB) (Blundell et al., 2015).

In comparison to DGP, the BNN demonstrates better scalability to large datasets and improved representational capacity. Moreover, the BbB method facilitates the compatibility of BNN with deep learning techniques and thus enables flexible implementation in different neural network architectures, such as Convolutional Neural Network (CNN) (Shridhar et al., 2019), Recurrent Neural Network (RNN) (Fortunato et al., 2019), and Transformer (Müller et al., 2024).

However, a fundamental challenge remains. Neither DGPs (typically employed as forward maps) nor BNNs (which can operate in either mapping direction) inherently model the posterior uncertainty of parameters in a direct, closed-form manner. Instead, both act as functional mappings that require an external probabilistic framework to characterize the distribution of interest, such as marginalization over latent functions or variational optimization over weights.

Consequently, several works raise concerns with BNNs, including wrong uncertainty estimation in between separated regions of observations (Foong et al., 2019, 2020), poor

generalization ability under domain shift (Izmailov et al., 2021), and a high sensitivity to the choice of prior distributions (Silvestro and Andermann, 2020). These limitations, arising from the difficulty of capturing complex, multi-modal posterior landscapes using standard approximate inference, motivate the applications of more powerful generative models, such as variational autoencoders and NFs, which are discussed in the following sections.

3.3 Variational Autoencoder

In classical VI methods, the parameters of interest $\boldsymbol{\theta}$ are random variables, and the approximate posterior distribution over the parameters is parameterized by variational parameters $\boldsymbol{\phi}$. In contrast, a Variational Autoencoder (VAE) is constructed with deterministic neural networks (parameterized by $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$) and introduces variability by latent variables. For a dataset $\mathcal{D} = \{\boldsymbol{x}^{(i)}\}_{i=1}^{M}$, the samples are assumed to be independent and identically distributed (i.i.d.) and drawn from an unchanging underlying distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x})$. A VAE extracts information from input data $\boldsymbol{x}$ to latent variables $\boldsymbol{z}$ by a parameterized inference model $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$, which is also called an *encoder* or *recognition model*. Then, the VAE generates data $\boldsymbol{x}$ from the latent variables $\boldsymbol{z}$ by another parameterized model $p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$, which is also called a *decoder*. The decoder is often mentioned as a "generative" model. However, according to the formal definition (Kingma and Welling, 2019), a generative model learns a joint distribution $p_{\boldsymbol{\theta}}(\boldsymbol{z}, \boldsymbol{x}) = p_{\boldsymbol{\theta}}(\boldsymbol{z})p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$, including sampling $\boldsymbol{z} \sim p_{\boldsymbol{\theta}}(\boldsymbol{z})$ and sampling $\boldsymbol{x} \sim p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$.

The latent variables connect the inference model and the generative model. From the encoder side, the inference model $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ approximates the true posterior distribution $p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ of the generative model, which is intractable. From the decoder side, the distribution of latent variables is a prior distribution, which is usually chosen to be a standard normal distribution, i.e., $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\mathbf{0}, \boldsymbol{I})$. The whole architecture is illustrated in Figure II.3.

Same reparameterization trick as in the BNN is used to enable gradient-based optimization. The latent variables $\boldsymbol{z}$ becomes:

$$\boldsymbol{z} = \boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}) + \boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{II.35}$$

where $\boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x})$ and $\boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x})$ are also neural networks. Different from the mini-batch optimization in the BNN, we use one single encoder neural network to perform posterior inference over all samples in the dataset. This strategy of sharing variational parameters across datapoints is called *amortized variational inference* (Gershman and Goodman, 2014; Margossian and Blei, 2024), which improves the optimization efficiency.

The objective of the inference model is to minimize the KL-divergence between the inferred and true posterior $D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}))$. Recall the ELBO in VI (II.19, II.20),

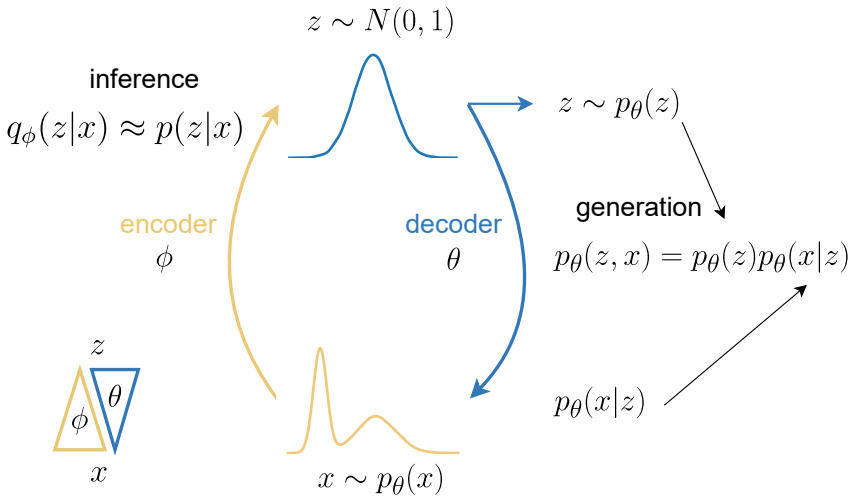


Figure II.3: The architecture of a Variational Autoencoder. The encoder (with neural network parameters $\boldsymbol{\phi}$) maps the input data $\boldsymbol{x}$ to latent variables $\boldsymbol{z}$, and the decoder (with neural network parameters $\boldsymbol{\theta}$) reconstructs the input data from the latent variables.

the ELBO of a VAE is given by:

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) - D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p_{\boldsymbol{\theta}}(\boldsymbol{z})). \tag{II.36}$$

Maximizing the ELBO optimizes concurrently both the inference model and the generative model. The marginal likelihood $p_{\boldsymbol{\theta}}(\boldsymbol{\theta})$ is maximized, meaning the generated data maximally resembles the input data. Meanwhile, the distance between the approximated and the true posterior is minimized. Changing the prior distribution in the ELBO, $p_{\boldsymbol{\theta}}(\boldsymbol{z}) := p(\boldsymbol{z})$, the overall loss function of a VAE is given by:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \underbrace{-\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})}_{\text{reconstruction}} + \underbrace{D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p(\boldsymbol{z}))}_{\text{regularization}}. \tag{II.37}$$

The first term is a reconstruction loss, which encourages the decoder to reconstruct the input data from the latent variables. The second term is a regularization term, which forces the approximate posterior to be close to the prior distribution.

VAEs have proven to be versatile and powerful in diverse applications, such as representation learning, anomaly detection, and generative modeling (Mienye and Swart, 2025). To enable VAEs to perform Bayesian inference of variables $\boldsymbol{x}$ given observations $\boldsymbol{y}$, the observations should be inserted into both the encoder and decoder as conditions. This method is named as conditional VAE (Sohn et al., 2015). The inference model becomes $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{y})$,

the prior distribution becomes $p(\boldsymbol{z}|\boldsymbol{y})$, and the generative model becomes $p_{\boldsymbol{\theta}}(\boldsymbol{z}, \boldsymbol{x}|\boldsymbol{y})$. The conditions are often constrained to be low-dimensional variables, such as class labels (Sohn et al., 2015) or physical parameters (Martínez-Palomera et al., 2020). For high-dimensional and complex conditions, a powerful decoder is required to capture the relationship between the variables and observations. In this case, the latent variables may be ignored and thus the variability losses. This issue is known as posterior collapse (Bowman et al., 2016; McCarthy et al., 2020), which limits the application of VAEs in conditional inference tasks.

4 Normalizing Flow

Another deep probabilistic model is the Normalizing Flows (NFs). Rezende and Mohamed (2016) proposed NF as a method for approximating posterior distributions by transforming a simple base distribution through a sequence of invertible mappings into complex target distributions.

4.1 Basics

We simplify a NF as an invertible and differentiable function $f : \mathbb{R}^D \to \mathbb{R}^D$ (as known as a diffeomorphism) that maps a random variable $\boldsymbol{z}$ in latent space to another random variable $\boldsymbol{x}$ in data space:

$$\boldsymbol{x} = f(\boldsymbol{z}), \tag{II.38a}$$

$$\boldsymbol{z} = f^{-1}(\boldsymbol{x}). \tag{II.38b}$$

The variable $\boldsymbol{x}$ follows an underlying distribution of data $p_X(\boldsymbol{x})$. The variable $\boldsymbol{z}$ is drawn from a simple base distribution $p_Z(\boldsymbol{z})$, such as a standard normal distribution. Understanding the transformation between the two density functions is essential for comprehending NFs.

4.1.1 Change of Variables

According to the change of variables formula (Schwartz, 1954), when both the latent and data space are smooth and the function f is differentiable, we have:

$$\int \Psi(\boldsymbol{x}) d\boldsymbol{x} = \int \Psi(\boldsymbol{x} = f(\boldsymbol{z})) \left|\det \boldsymbol{J}_f(\boldsymbol{z})\right| d\boldsymbol{z}, \tag{II.39}$$

where Ψ is a continuous function, $\boldsymbol{J}_f(\boldsymbol{z})$ is the Jacobian of the mapping f and $\det|\cdot|$ stands for determinant:

$$\left|\det \boldsymbol{J}_f(\boldsymbol{z})\right| = \left|\det \frac{\partial f(\boldsymbol{z})}{\partial \boldsymbol{z}}\right|. \tag{II.40}$$

This Equation (II.39) implies probability conservation under transformation.

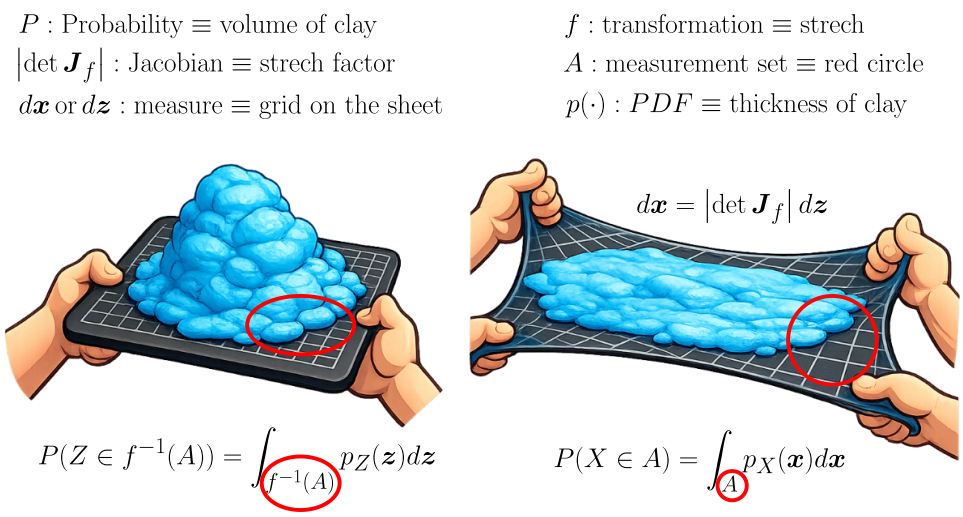


Figure II.4: The Clay Volume Analogy for Change of Variables. Probability mass is represented as clay on a rubber sheet. Transforming the variable ($\boldsymbol{z} \to \boldsymbol{x}$) stretches the sheet. While the total amount of clay (probability P) remains constant, the local thickness (density $p(\cdot)$) decreases as the area increases.

In probability theory, the *Radon–Nikodym derivative* (Durrett, 1996) provides a rigorous method to express how one probability measure changes with respect to another. In the context of continuous random variables, the Probability Density Function (PDF) is essentially the Radon–Nikodym derivative of the probability measure (probability mass of an event) with respect to the Lebesgue measure (standard geometric volume). This relationship generalizes the familiar link between a Cumulative Distribution Function (CDF) and a PDF. To provide a physical intuition for this abstract concept, we introduce the "Clay Volume" Analogy illustrated in Figure II.4. The Radon–Nikodym derivative corresponds to the local thickness of the clay at any given point.

For or a random variable $\boldsymbol{x}$ and a measurement set A, the probability measure is defined by the integral of the density:

$$P(X \in A) = \int_A p_X(\boldsymbol{x}) d\boldsymbol{x}. \tag{II.41}$$

To relate this to a transformed variable $\boldsymbol{z}$ via an invertible mapping (Equation (II.38b)), we rely on the invariance of probability. An event occurs in A for $\boldsymbol{x}$ if and only if it occurs

in the pre-image $f^{-1}(A)$ for $\boldsymbol{z}$:

$$P(X \in A) = P(Z \in f^{-1}(A)), \tag{II.42a}$$

$$\int_A p_X(\boldsymbol{x}) d\boldsymbol{x} = \int_{f^{-1}(A)} p_Z(\boldsymbol{z}) d\boldsymbol{z}. \tag{II.42b}$$

We apply the multivariate change of variables formula (Equation II.39) to the right-hand integral in Equation (II.42b). This transforms the domain of integration from $f^{-1}(A)$ back to A, introducing the Jacobian determinant to account for local volume distortion:

$$\int_{f^{-1}(A)} p_Z(\boldsymbol{z}) d\boldsymbol{z} = \int_A p_Z(f^{-1}(\boldsymbol{x})) \left|\det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x})\right| d\boldsymbol{x}. \tag{II.43}$$

Here, the Jacobian determinant of the inverse mapping f^{-1} is related to the forward mapping by:

$$\left|\det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x})\right| = \left|\det \frac{\partial f^{-1}(\boldsymbol{x})}{\partial \boldsymbol{x}}\right| = \left|\det \frac{\partial \boldsymbol{z}}{\partial f(\boldsymbol{z})}\right| = \frac{1}{\left|\det \boldsymbol{J}_f(\boldsymbol{z})\right|}. \tag{II.44}$$

By substituting (II.43) into (II.42b), we obtain an equality of two integrals defined over the same domain A::

$$\int_A p_X(\boldsymbol{x}) d\boldsymbol{x} = \int_A p_Z(f^{-1}(\boldsymbol{x})) \left|\det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x})\right| d\boldsymbol{x}. \tag{II.45}$$

Crucially, Equation (II.45) holds for any arbitrary measurable set A. In measure theory, if two continuous functions integrate to the same value over every possible subset of the domain, the functions themselves must be identical almost everywhere. Therefore, we can equate the integrands directly:

$$p_X(\boldsymbol{x}) = p_Z(f^{-1}(\boldsymbol{x})) \left|\det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x})\right|. \tag{II.46}$$

Substituting $\boldsymbol{x}$ and $\boldsymbol{z}$ with equations II.38a and II.38b respectively, it can also be formulated using the forward mapping f:

$$p_Z(\boldsymbol{z}) = p_X(f(\boldsymbol{z})) \left|\det \boldsymbol{J}_f(\boldsymbol{z})\right|. \tag{II.47}$$

These two equations are declared as the **change of variables** applied for NFs. They are often formulated in logarithm function:

$$\log p_X(\boldsymbol{x}) = \log p_Z(f^{-1}(\boldsymbol{x})) + \log \left| \det \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right|, \tag{II.48}$$

$$\log p_Z(\boldsymbol{z}) = \log p_X(f(\boldsymbol{z})) + \log \left| \det \boldsymbol{J}_f(\boldsymbol{z}) \right|. \tag{II.49}$$

The derivation of the probability density in Equation (II.47) relies fundamentally on the Change of Variables theorem. Consequently, for a Normalizing Flow to be both mathematically valid and computationally tractable, the transformation f cannot be arbitrary; it must satisfy three strict structural constraints:

1. **Invertibility (Bijectivity):** The mapping $f : Z \to X$ must be bijective. This ensures that the inverse f^{-1} exists and is unique, allowing distinct data points $\boldsymbol{x}$ to be uniquely mapped to latent representations $\boldsymbol{z}$ (inference) and vice versa (sampling).

2. **Differentiability:** The function f and its inverse must be differentiable. This is a prerequisite for the existence of the Jacobian matrix $\boldsymbol{J}_f$. In differential geometry, a map satisfying conditions (1) and (2) is termed a *diffeomorphism*.

3. **Computational Efficiency of the Jacobian:** While the first two properties ensure mathematical validity, this third property ensures practical feasibility. Training an NF involves maximizing the log-likelihood, which requires calculating the log-determinant of the Jacobian, $\log |\det \boldsymbol{J}_f(\boldsymbol{z})|$. For an arbitrary high-dimensional function with a dimension D, this calculation scales with $\mathcal{O}(D^3)$. To make training efficient, Normalizing Flows are explicitly designed such that this term can be computed in linear time, $\mathcal{O}(D)$, often by restricting the Jacobian to be triangular or diagonal.

Beyond the structural constraints of individual mappings, a powerful feature of NFs is composability (Papamakarios et al., 2021). Given a series of transformations $f_L \circ \cdots \circ f_1$, the following equations hold:

$$\boldsymbol{x} = f_L(f_{L-1}(\cdots f_1(\boldsymbol{z}) \cdots)), \tag{II.50a}$$

$$f^{-1} = (f_L \circ \cdots \circ f_1)^{-1} = f_1^{-1} \circ \cdots \circ f_L^{-1}, \tag{II.50b}$$

$$\boldsymbol{J}_f(\boldsymbol{z}) = \boldsymbol{J}_{f_L}(\boldsymbol{z}_{L-1}) \cdot \boldsymbol{J}_{f_{L-1}}(\boldsymbol{z}_{L-2}) \cdots \boldsymbol{J}_{f_1}(\boldsymbol{z}_0). \tag{II.50c}$$

Equation (II.50c) applies the multivariate **chain rule**. A crucial consequence for training is that the log-determinant of the Jacobian decomposes into a sum of terms:

$$\log |\det \boldsymbol{J}_f(\boldsymbol{z})| = \sum_{k=1}^{L} \log |\det \boldsymbol{J}_{f_k}(\boldsymbol{z}_{k-1})|. \tag{II.51}$$

This allows the total change in density to be computed by simply summing the log-determinants of each layer, enabling the construction of deep, expressive flows while

maintaining linear computational complexity.

4.1.2 Using Normalizing Flows for Modeling and Inference

The invertibility also provides us easy way to fit NF models by minimizing the divergence between the true data distribution and the estimated one. For a NF with parameters $\boldsymbol{\theta}$, the KL-divergence is given by:

$$D_{\mathrm{KL}}(p_X(\boldsymbol{x})\|p_{\boldsymbol{\theta}}(\boldsymbol{x})) = \mathbb{E}_{\boldsymbol{x}\sim p_X}[\log p_X(\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{x}\sim p_X}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x})]. \tag{II.52}$$

In this objective function, the first term (the entropy of the data) depends only on the true distribution and is constant with respect to $\boldsymbol{\theta}$. Consequently, minimizing the KL divergence is equivalent to maximizing the second term, which corresponds to the expected log-likelihood. Given a dataset $\mathcal{D} = \{\boldsymbol{x}^{(m)}\}_{m=1}^{M}$ drawn from $p_X(\boldsymbol{x})$, we can approximate this expectation using the sample mean (Monte Carlo simulation):

$$\mathbb{E}_{\boldsymbol{x}\sim p_X}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x})] \approx \frac{1}{M}\sum_{m=1}^{M} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}^{(m)}). \tag{II.53}$$

This confirms that minimizing the KL divergence is equivalent to Maximum Likelihood Estimation (MLE). By substituting the change of variables formula (Equation (II.46)) into the log-likelihood, we obtain the final loss function to be minimized:

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= -\frac{1}{M}\sum_{m=1}^{M}\left[\log p_Z(f_{\boldsymbol{\theta}}^{-1}(\boldsymbol{x}^{(m)})) + \log\left|\det \boldsymbol{J}_{f_{\boldsymbol{\theta}}^{-1}}(\boldsymbol{x}^{(m)})\right|\right], \\ &= \frac{1}{M}\sum_{m=1}^{M}\left[\underbrace{\frac{1}{2}\left\|f_{\boldsymbol{\theta}}^{-1}(\boldsymbol{x}^{(m)})\right\|_2^2}_{\text{inference}} \underbrace{-\log\left|\det \boldsymbol{J}_{f_{\boldsymbol{\theta}}^{-1}}(\boldsymbol{x}^{(m)})\right|}_{\text{preparation for generation}}\right], \quad \text{when } p_Z(\boldsymbol{z}) = \mathcal{N}(\mathbf{0}, \boldsymbol{I}).\end{aligned} \tag{II.54}$$

The parameters $\boldsymbol{\theta}$ are then optimized using stochastic gradient-based methods.

The loss function of NFs suggests that during training, the learned latent representation is compared with the true latent distribution, while the term of Jacobian determinant ensures this comparison to be transformed back to the data space. Recalling the VAE, the first term can be seen as inference, while the second term can be implicitly regarded as "preparation for generation". After the NF model is well-trained, new data can be generated by sampling the latent variables and transforming them via the forward mapping. It is important to emphasize that, NFs are conventionally trained in one direction at a

time. Simultaneously bidirectional training would necessitate computing gradients through the composition of a function and its inverse, leading to problematic gradient flows of the form $\partial\mathcal{L}/\partial\boldsymbol{\theta}_f(\boldsymbol{\theta}_{\boldsymbol{f}^{-1}})$, which creates numerical instabilities and computational inefficiencies.

A variety of architectures have been developed to realize NFs. A GitHub website (Riebesell et al., 2023) summarizes the key publications and implementations in this domain. As outlined in the comprehensive review by Kobyzev et al. (2021), successful architectures include coupling flows, autoregressive flows, residual flows, and infinitesimal (continuous) flows. Throughout this work, we focus primarily on coupling flows, which offer structural invertibility and have been extensively utilized in NFs. Infinitesimal flows characterize distributional transformations through continuous dynamics, establishing deep theoretical connections with other probabilistic frameworks such as diffusion models. Thus, we also introduce continuous NFs following the description of coupling flows.

4.2 Coupling Flows

The basic idea of the coupling flow is partitioning the input variables into two subsets, one subset of which is processed by neural networks and also concurrently directly passed through to the output variables, while the other subset is combined with the processed subset through affine functions.

The first proposed coupling flow was NICE (Dinh et al., 2015). The input variables $\boldsymbol{u}$ are split into two parts $[\boldsymbol{u}_1, \boldsymbol{u}_2]$, and the corresponding output variables are $\boldsymbol{v} = [\boldsymbol{v}_1, \boldsymbol{v}_2]$. The architecture in the forward direction is formulated by the following functions:

$$\boldsymbol{v}_1 = \boldsymbol{u}_1, \quad \boldsymbol{v}_2 = g(\boldsymbol{u}_2; s(\boldsymbol{u}_1)), \tag{II.55}$$

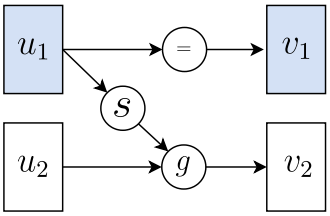
where s is a neural network and g is an affine function $g(a; b) = a + b$ or $a \odot b$ (element-wise multiplication). The forward architecture is illustrated in Figure II.5a. Then, the inverse functions are easily calculated:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1, \quad \boldsymbol{u}_2 = g^{-1}(\boldsymbol{v}_2; s(\boldsymbol{u}_1)). \tag{II.56}$$

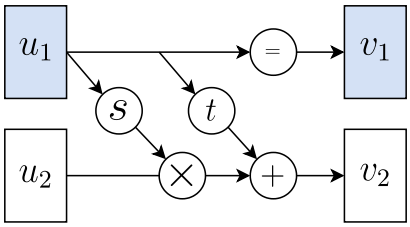
From the above functions, the Jacobian matrix is:

$$\frac{\partial \boldsymbol{v}}{\partial \boldsymbol{u}} = \begin{bmatrix} \partial \boldsymbol{v}_1/\partial \boldsymbol{u}_1 & \partial \boldsymbol{v}_1/\partial \boldsymbol{u}_2 \\ \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_1 & \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_2 \end{bmatrix} = \begin{bmatrix} \boldsymbol{I} & \boldsymbol{0} \\ \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_1 & \partial g(\boldsymbol{u}_2; s(\boldsymbol{u}_1))/\partial \boldsymbol{u}_2 \end{bmatrix} \tag{II.57}$$

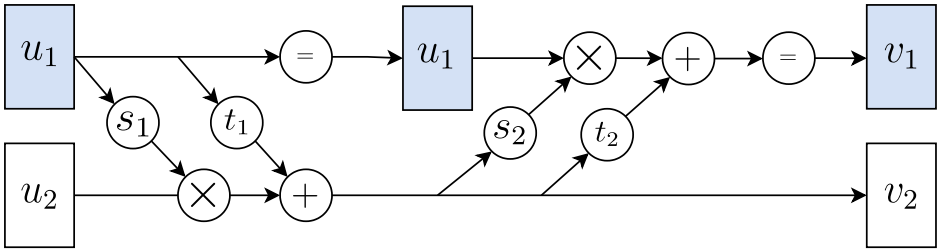
Because of the triangle matrix, the determinant is $\partial g(\boldsymbol{u}_2; s(\boldsymbol{u}_1))/\partial \boldsymbol{u}_2$. By composing multiple coupling layers, we can construct increasingly complex hierarchical transformations. Since each coupling layer preserves a subset of its input unchanged, it is essential to alternate the roles of the partitioned subsets across successive layers. This ensures that all input



(a) The architecture of a Non-linear Independent Component Estimation (NICE) layer. s is a neural network, and g is an affine function.



(b) The architecture of a Real-Valued Non-Volume Preserving (RealNVP) layer. s and t are neural networks. The symbol × stands for element-wise product by the exponential function over input.



(c) The architecture of an Invertible Neural Network (INN) layer. s_1, s_2, t_1, and t_2 are neural networks. The symbol × stands for element-wise product by the exponential function over input.

Figure II.5: The architectures of coupling flows. The input of the layer is $\boldsymbol{u}$ and the output is $\boldsymbol{v}$.

dimensions are eventually processed. Analyzing the Jacobian reveals that a minimum of three coupling layers is required for full inter-dimensional interaction. In practice, we typically employ four layers to achieve sufficient expressiveness.

An improved architecture is RealNVP, where one part of the variables is processed by two neural networks. The forward transformation is:

$$\boldsymbol{v}_1 = \boldsymbol{u}_1, \quad \boldsymbol{v}_2 = \boldsymbol{u}_2 \odot \exp(s(\boldsymbol{u}_1)) + t(\boldsymbol{u}_1). \tag{II.58}$$

And the inverse transformation is:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1, \quad \boldsymbol{u}_2 = (\boldsymbol{v}_2 - t(\boldsymbol{u}_1)) \odot \exp(-s(\boldsymbol{u}_1)). \tag{II.59}$$

The Jacobian determinant $\det|\partial \boldsymbol{v}/\boldsymbol{u}|$ is then $\exp(\Sigma_i s(\boldsymbol{u}_{1,i}))$. The architecture is shown in Figure II.5b. To interact all variables in one block, INN (Ardizzone et al., 2019a) extends RealNVP by another same coupling block, as shown in Figure II.5c.

A further enhancement of RealNVP is Glow (Kingma and Dhariwal, 2018), which inserts an invertible normalization layer and an invertible permutation layer before affine transformation. Hierarchical Invertible Neural Transport (HINT) (Kruse et al., 2021) partitions the inputs of coupling blocks recursively to construct a dense, triangular Jacobian. Another method to increase interaction among variables is Masked Autoregressive Flow (MAF) (Papamakarios et al., 2018), which can be considered as a generalized RealNVP that leverages autoregressive transformations between variables. Apart from affine coupling, Neural Spline Flow (NSF) (Durkan et al., 2019) applies monotonic rational-quadratic spline functions, enabling more expressive and smooth transformations, however, at the cost of higher complexity.

4.3 Residual Flows

Such flows build step-wise invertible transformations between distributions, enabling tractable Jacobian calculation and density estimation. A notable drawback lies in the strict architectural constraint on coupling blocks. To release this limitation, residual flows (Behrmann et al., 2019; Chen et al., 2020) introduce invertible mapping through specified residual blocks. A residual block (He et al., 2015) takes the form $f(x) = x + F(x)$, where F is a neural network. When F is constrained to be Lipschitz continuous and the Lipschitz constant is smaller than 1, the function $f(x)$ is invertible.

The definition of a Lipschitz continuous function is referred to the work by Khalil and Grizzle (2002).

Definition 1. *(**Lipschitz continuous function**) A function $f : \mathbb{R}^d \to \mathbb{R}^K$, defined on some domain $dom(f) \subseteq \mathbb{R}^d$, is called C-Lipschitz continuous w.r.t. some α-norm, if $\forall \boldsymbol{x}, \boldsymbol{y} \in dom(f) : \|f(\boldsymbol{x}) - f(\boldsymbol{y})\|_\alpha \leq C \|\boldsymbol{x} - \boldsymbol{y}\|_\alpha$. C is called Lipschitz constant of f and $0 < C < \inf$.*

Lemma 1. *(**Invertibility of Residual Units**) Let $f(\mathbf{x}) = \mathbf{x} + F(\mathbf{x})$. If F is C-Lipschitz continuous with $C < 1$, then f is a bijection.*

Proof. To show f is invertible, we must show that for any $\mathbf{y}$, there exists a unique $\mathbf{x}$ such that $\mathbf{y} = \mathbf{x} + F(\mathbf{x})$. This can be rewritten as a fixed-point problem:

$$\mathbf{x} = \mathbf{y} - F(\mathbf{x}) = G(\mathbf{x}). \tag{II.60}$$

By the definition of G, for any $\mathbf{x}_1, \mathbf{x}_2$:

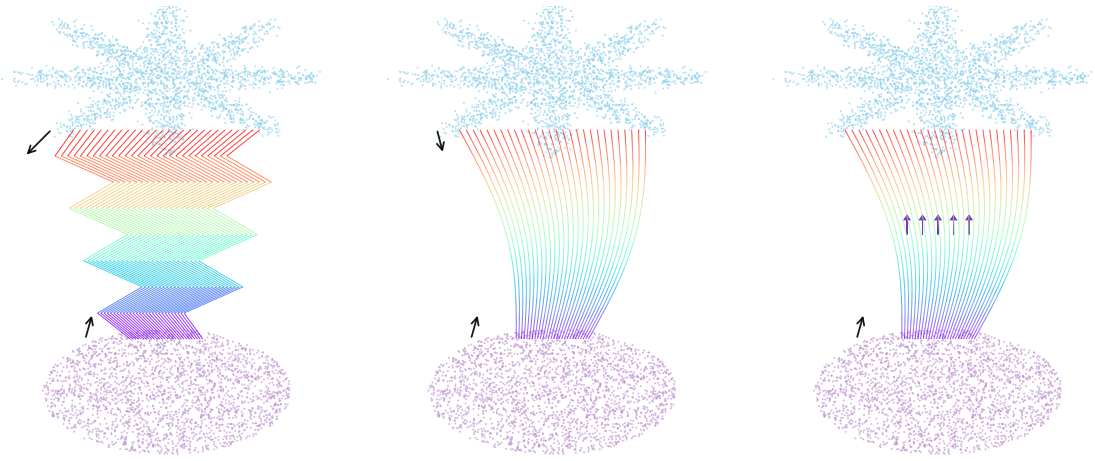
$$\|G(\mathbf{x}_1) - G(\mathbf{x}_2)\| = \|(\mathbf{y} - F(\mathbf{x}_1)) - (\mathbf{y} - F(\mathbf{x}_2))\| \tag{II.61}$$

$$= \|F(\mathbf{x}_2) - F(\mathbf{x}_1)\| \tag{II.62}$$

$$\leq C \|\mathbf{x}_2 - \mathbf{x}_1\| . \tag{II.63}$$

Since $C < 1$, G is a contraction mapping. According to the *Banach Fixed-Point Theorem* (Shukla et al., 2016), G has a unique fixed point $\mathbf{x}^*$ for every $\mathbf{y}$, implying f is bijective. □

The residual block can be seen as an Euler discretization of an ordinary differential equation (ODE) (Lu et al., 2020; Ruthotto and Haber, 2018; Haber and Ruthotto, 2017), which is the foundation of continuous NFs. A simplified comparison between the residual flow, continuous flow and flow matching is illustrated in Figure II.6.



(a) Residual Flow **(b)** Continuous Flow **(c)** Flow Matching

Figure II.6: Illustration of the Residual Flow, Continuous Flow, and Flow Matching. The bottom point cloud is the base distribution in the latent space, while the top one in the star shape is the target distribution in the data space. The residual flow and continuous flow are trained from the data to latent space and sampled from latent to the data space, while the Flow Matching is trained by learning the vector field and sampled from the latent to the data space. The other direction is not applied by Flow Matching.

4.4 Continuous Flows

When the transformation from latent space to data space is modeled by a continuous dynamical system, the NF is called a *continuous NF*. The continuous dynamical system

can be formulated by a Neural ODE, which is introduced as follows.

4.4.1 Neural Ordinary Differential Equations

For a dynamical system with states $\boldsymbol{h}$, an ODE is:

$$\frac{d\boldsymbol{h}(t)}{dt} = f(\boldsymbol{h}(t), t; \boldsymbol{\theta}), \tag{II.64}$$

where $\boldsymbol{\theta}$ denotes system parameters. When the ODE is modeled by neural networks, it becomes a Neural ODE (Chen et al., 2019). Given the initial state $\boldsymbol{h}(0)$, the final state $\boldsymbol{h}(T)$ is the solution of the initial value problem at the time step T using a numerical solver. The loss function of training a Neural ODE is:

$$\mathcal{L}(\boldsymbol{h}(T)) = \mathcal{L}(\int_0^T f(\boldsymbol{h}(t), t; \boldsymbol{\theta})dt), \tag{II.65}$$

where f is a neural network parameterized by $\boldsymbol{\theta}$. A straightforward application of Neural ODEs is learning continuous time-series models. Moreover, when the time step is interpreted as architectural depth, the distributional transformation in NFs can also be constructed with a Neural ODE. The Neural ODE replaces an infinite number of transformations by change of variables with a single neural network, which represents the derivative of transformation at each depth. This enables more efficient and expressive probabilistic modeling and inference.

We define a continuous NF as a Neural ODE:

$$\frac{d\boldsymbol{z}(t)}{dt} = f(\boldsymbol{z}(t), t; \boldsymbol{\theta}), \tag{II.66}$$

where $\boldsymbol{z}(t)$ follows the density $p(\boldsymbol{z}(t))$ at an instantaneous step $t \in [t_0, t_1]$. The NF f is parameterized by $\boldsymbol{\theta}$. To apply the Neural ODE as continuous NFs, the function f should be uniformly Lipschitz continuous in the states $\boldsymbol{z}(t)$ and continuous in depth t.

The Lipschitz condition guarantees that there exists a unique solution curve for every initial condition $\boldsymbol{z}(0)$. This is known as the Picard–Lindelöf theorem (Coddington and Levinson, 1955). Assuming the dynamics function f is smooth, then it is a *diffeomorphism*, which means we also get the same $\boldsymbol{z}(0)$ by running the ODE backwards. Recalling the residual flows, where the Lipschitz constant must be smaller than 1 (see 4.3), invertibility is ensured naturally by a smooth and Lipschitz continuous ODE. In practice, neural networks with smooth continuous activation functions satisfy this requirement.

A central technical challenge in training continuous-depth networks lies in executing reverse-mode differentiation (backpropagation) through the ODE solver. Two primary strategies exist for this task. The first is the backward auto-differentiation (*discretize-*

then-optimize), which differentiates directly through the operations of the numerical solver. While this method is numerically stable and standard for stiff differential equations, it typically incurs a high memory cost as it requires storing intermediate states of the solver operations.

To mitigate this memory bottleneck, the continuous adjoint method (Bittner, 1963; Gavrilović et al., 1963; Chen et al., 2019) is often employed (*optimize-then-discretize*). This method computes gradients by solving an augmented ODE backwards in time. Specifically, instead of storing intermediate states, the solver reconstructs the state trajectory backwards from the final time point, effectively re-solving the dynamics in reverse alongside the adjoint state. This approach reduces memory cost to $\mathcal{O}(1)$ with respect to integration time. However, because the system dynamics must be evaluated a second time during this reconstruction, it results in increased computational runtime. This trade-off makes the adjoint method ideal for high-dimensional variables and long sequences, but computationally slower for models with expensive architecture dynamics.

4.4.2 Continuous Normalizing Flows

Besides for the ODE II.66, the log-density of states changes according to another ODE, the **instantaneous change-of-variables formula** (Chen et al., 2019):

$$\frac{\partial \log p(\boldsymbol{z}(t))}{\partial t} = -\mathrm{Tr}\left(\frac{\partial f(\boldsymbol{z}(t), t; \boldsymbol{\theta})}{\partial \boldsymbol{z}(t)}\right). \tag{II.67}$$

Directly calculating the trace is computationally expensive. For B backward passes to get the diagonal, $\mathcal{O}(B)$ computational cost and $\mathcal{O}(B^2)$ memory cost are required. Alternatively, the trace can be approximated through Skilling-Hutchinson trace estimator (Skilling, 1989; Hutchinson, 1990; Adams et al., 2018):

$$\mathrm{Tr}\left(\frac{\partial f}{\partial \boldsymbol{z}(t)}\right) = \mathbb{E}_{p(\boldsymbol{\epsilon})}\left[\boldsymbol{\epsilon}^T \frac{\partial f}{\partial \boldsymbol{z}(t)} \boldsymbol{\epsilon}\right], \tag{II.68}$$

where $\boldsymbol{\epsilon}$ is a noise vector sampled from a standard normal distribution and kept unchanged to maintain determinism of the ODE. The part of vector-Jacobian product $\boldsymbol{\epsilon}^T \partial f / \partial \boldsymbol{z}(t)$ is computed with automatic differentiation. In this way, the computational cost is reduced to $\mathcal{O}(1)$ with respect to the number of backward pass (one backward pass for each trace approximation). And there is no need to store the full Jacobian matrix. The trade-off is that the estimator introduces variance, as it yields a stochastic approximation of the trace rather than the exact value; in practice, one or a few samples per step are typically sufficient to keep training stable.

For a continuous NF between the data distribution $p(\boldsymbol{z}(t_1)) = p_X(\boldsymbol{x})$ and the latent

distribution $p(\boldsymbol{z}(t_0)) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, we define the augmented states as $[\boldsymbol{z}(t), \log p_X(\boldsymbol{x}) - \log p(\boldsymbol{z}(t))]$. In the original formulation (Grathwohl et al., 2018), the second term is $\log p(\boldsymbol{z}(t))$ itself; in that case, one must still calculate $\log p_X(\boldsymbol{x}) - \log p(\boldsymbol{z}(t))$. The difference formulation used here yields a consistent form for both initial values and solutions. The total dynamics becomes:

$$\frac{d}{dt}\begin{bmatrix} \boldsymbol{z}(t) \\ \log p_X(\boldsymbol{x}) - \log p(\boldsymbol{z}(t)) \end{bmatrix} = \begin{bmatrix} f(\boldsymbol{z}(t), t; \boldsymbol{\theta}) \\ \mathrm{Tr}\left(\frac{\partial f}{\partial \boldsymbol{z}(t)}\right) \end{bmatrix} \tag{II.69}$$

Then, building the continuous NF is equivalent to solving the initial value problem (Grathwohl et al., 2018):

$$\underbrace{\begin{bmatrix} \boldsymbol{z}(t_0) \\ \log p_X(\boldsymbol{x}) - \log p(\boldsymbol{z}(t_0)) \end{bmatrix}}_{\text{solutions}} = \underbrace{\begin{bmatrix} \boldsymbol{z}(t_1) = \boldsymbol{x} \\ \log p_X(\boldsymbol{x}) - \log p(\boldsymbol{z}(t_1)) = 0 \end{bmatrix}}_{\text{initial values}} + \int_{t_1}^{t_0} \underbrace{\begin{bmatrix} f(\boldsymbol{z}(t), t; \boldsymbol{\theta}) \\ \mathrm{Tr}\left(\frac{\partial f}{\partial \boldsymbol{z}(t)}\right) \end{bmatrix}}_{\text{dynamics}} dt. \tag{II.70}$$

It is worth noting that the dynamics is backwards integrated from t_1 to t_0, meaning from the data side to the latent side. The time span can also be optimized to further improve the computational efficiency (Du et al., 2022).

Continuous NFs release the architectural constraints of discrete flows, allowing more flexible and expressive density estimation through the integration of Neural ODEs. This improved expressiveness, however, comes at a cost: computing the Jacobian trace and the associated ODE integrals is significantly more expensive than in stacked coupling flows.

4.5 Flow Matching

Flow Matching (FM) (Lipman et al., 2023) is a novel approach for training continuous NFs. The key idea is to learn a vector field that directly matches the step-by-step transport (infinitesimal transport) between the data distribution and the latent distribution. This is achieved by minimizing the expected squared error between the learned vector field and the true velocity field defined by the optimal transport plan. The optimal transport plan is normally a linear interpolation between the latent samples $\boldsymbol{z}(t_0)$ and the data samples $\boldsymbol{x} = \boldsymbol{z}(t_1)$:

$$\boldsymbol{v}^{OT}(\boldsymbol{z}(t), t) = \frac{\boldsymbol{z}(t_1) - \boldsymbol{z}(t_0)}{t_1 - t_0}. \tag{II.71}$$

Along this path, the marginal distribution at time t is denoted $p_t(\boldsymbol{z})$. For the linear interpolation with $\boldsymbol{z}(t) = (1-t)\,\boldsymbol{z}_0 + t\,\boldsymbol{x}$, where $\boldsymbol{z}_0 \sim N(\mathbf{0}, \boldsymbol{I})$ (latent distribution) and

$\boldsymbol{x} \sim p_X(\boldsymbol{x})$ (data distribution), $p_t(\boldsymbol{z})$ is the joint distribution through the interpolation:

$$p_t(\boldsymbol{z}) = (1-t)\, p(\boldsymbol{z}_0) + t\, p_X(\boldsymbol{x}). \tag{II.72}$$

Thus, sampling $\boldsymbol{z} \sim p_t(\boldsymbol{z})$ amounts to drawing a random point on the transport path at time t. The training objective is formulated as:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\boldsymbol{z}\sim p_t(\boldsymbol{z})}\left[\left\| v_{\boldsymbol{\theta}}(\boldsymbol{z}(t),t) - \boldsymbol{v}^{OT}(\boldsymbol{z}(t),t)\right\|^2\right], \tag{II.73}$$

where $v_{\boldsymbol{\theta}}(\boldsymbol{z}(t),t)$ is the learned vector field parameterized by $\boldsymbol{\theta}$, and $\boldsymbol{v}^{OT}(\boldsymbol{z}(t),t)$ is the true velocity field derived from the optimal transport plan. Simply put, the FM loss regresses the vector field $\boldsymbol{v}^{OT}$ with a neural network $v_{\boldsymbol{\theta}}$. Upon reaching zero loss, the learned continuous NF model will generate data samples by integrating the learned vector field from t_0 to t_1.

The main difference from other NFs is that FM is not trained by maximizing likelihood via change of variables between distributions. Instead, it focuses on learning the vector field that defines the continuous transformation between distributions. This approach avoids the computational overhead of solving ODEs during training, making it more efficient and scalable to high-dimensional data. On the other hand, FM cannot be directly applied to exact density estimation because it does not transform the data distribution into a known latent distribution. The sample density it generates is only an approximation of the data density. For exact estimation, FM must invert the generative process via a backward ODE, which makes it computationally heavier than the single-pass inference of other NFs.

4.6 Conditional Normalizing Flows

Such capabilities of estimating complex densities are desirable in Bayesian inference of the posterior distribution of variables given observations $p(\boldsymbol{x}|\boldsymbol{y})$. Similar to VAE, amortized Bayesian inference (Radev, 2021) is applied in NF to achieve a general posterior estimation across all data points. Obtaining the posterior distribution is represented by a conditional density transformation via a Conditional Normalizing Flow (CNF). The change of variables for a CNF extends the standard one (II.46) by the condition $\boldsymbol{y}$:

$$p_{X|Y}(\boldsymbol{x}|\boldsymbol{y}) = p_{Z|Y}(f^{-1}(\boldsymbol{x},\boldsymbol{y})|\boldsymbol{y})\left|\det\frac{\partial f^{-1}(\boldsymbol{x},\boldsymbol{y})}{\partial \boldsymbol{x}}\right|, \tag{II.74}$$

where the mapping $f : Z \to X \times Y$ is bijective between X and Z, and $\boldsymbol{z} = f^{-1}(\boldsymbol{x},\boldsymbol{y})$.

Designing such a CNF requires addressing two key considerations in advance: (1) how to manage the conditioned latent variables, and (2) how to incorporate conditions into the

flow architecture while preserving invertibility. Winkler et al. (2023) proposed to build the conditional prior $p_{Z|Y}(\boldsymbol{z}|\boldsymbol{y})$ as a diagonal Gaussian:

$$p_{Z|Y}(\boldsymbol{z}|\boldsymbol{y}) = \mathcal{N}(\boldsymbol{z}|\mu(\boldsymbol{y}), \text{diag}[\sigma^2(\boldsymbol{y})]). \tag{II.75}$$

For simpler conditioning scenarios, incorporating the conditional prior could accelerate the transformation through leveraging the information from conditions. For complex conditions, summarizing the conditions through mean and variance may result in information loss, thereby degrading posterior estimation quality. This limitation likely explains why most existing work on CNFs adopts a standard Gaussian as the latent base distribution, bypassing the need for an explicitly conditioned prior and relying instead on the normalizing flows to capture conditional information.

In terms of architectures of CNFs, a direct adaptation based on the INN is the conditional Invertible Neural Network (cINN) (Ardizzone et al., 2019b). It takes the condition as augmented input to the neural networks. In cases of higher-dimensional conditions, such as images, Anantha Padmanabha and Zabaras (2021) apply a staged cINN coupled with a staged condition network, wherein the conditions are progressively processed, and each intermediate representation is fed into its corresponding cINN block.

In scenarios involving conditions in complex structures and target variables in low dimension, for example, parameter identification from observed time series signals, BayesFlow (Radev et al., 2020) offers a general framework by extracting informative representations through a summary network and incorporating them, alongside the inputs, as augmented inputs to the cINN. This framework is particularly well-suited for parameter identification from time series observations, where the summary network is typically chosen as a time series modeling architecture, such as RNN. The architecture is illustrated in Figure II.7. Both the summary network and cINN are optimized together by minimizing the KL-divergence between the true posterior density and the estimated one (similar to Equation (II.54)):

$$\begin{aligned} D_{\text{KL}}(p_{X|Y}(\boldsymbol{x}|\boldsymbol{y})||p_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}|\boldsymbol{y})) \approx const.- \\ \frac{1}{M}\sum_{m=1}^{M}[\log p_Z(f_{\boldsymbol{\theta}}^{-1}(\boldsymbol{x}^{(m)}, h_{\boldsymbol{\phi}}(\boldsymbol{x}^{(m)}))) + \log\left|\det \boldsymbol{J}_{f_{\boldsymbol{\theta}}^{-1}}(\boldsymbol{x}^{(m)}, h_{\boldsymbol{\phi}}(\boldsymbol{x}^{(m)}))\right|], \end{aligned} \tag{II.76}$$

where f is the cINN parameterized by $\boldsymbol{\theta}$ and h is the summary network parameterized by $\boldsymbol{\phi}$.

The joint optimization approach encourages the summary network to deliver maximally useful information to the cINN thus improve the inference capability. In this work, we adopt the BayesFlow framework as a flexible and scalable solution for amortized Bayesian inference, adapting its summary network to accommodate diverse time series observations.

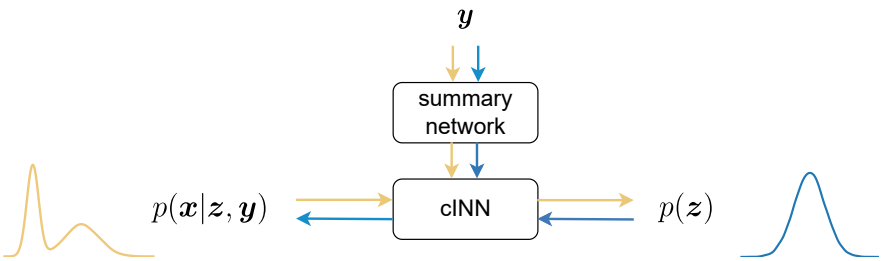


Figure II.7: The architecture of BayesFlow. The blue arrows indicate the forward direction for training BayesFlow, while the yellow arrows indicate the reverse direction for inference. The summary network extracts informative representations from the observations, which are then fed into the cINN alongside the target variables.

This framework forms the foundation of our proposed methods for parameter identification, system dynamics modeling, and state estimation.

5 From Bayesian Inference to Control

This section bridges the Bayesian inference methodologies discussed above to their practical applications in control engineering. We first review how Bayesian approaches are employed for system identification and state estimation. Next, we introduce Model Predictive Control (MPC), which is a widely used model-based control technique for optimal decision-making based on the system identification results. Finally, we discuss Reinforcement Learning (RL) as a data-driven control paradigm, highlighting its ability to learn control policies directly from interactions with the environment without requiring explicit system models. Together, these approaches illustrate the synergy between Bayesian inference and modern control strategies, enabling robust and adaptive solutions for complex dynamical systems.

5.1 Bayesian System Identification

An essential goal of system modeling and identification is to provide a rigorous framework for predicting and controlling the future evolution of system outputs. For any control strategy, the decision-maker needs to predict how the system will behave under any possible control input. Consider a discrete-time dynamical system with inputs $\boldsymbol{u}_n$ and outputs $\boldsymbol{y}_n$ at time t_n. The system evolution is governed by a (possibly stochastic) transition function:

$$\boldsymbol{y}_n = g(\boldsymbol{u}_n, \mathcal{D}_{0:(n-1)}; \boldsymbol{\theta}) + \boldsymbol{\epsilon}_n, \tag{II.77}$$

where g is the system function, $\boldsymbol{\theta}$ are unknown parameters, $\mathcal{D}_{0:(n-1)}$ denotes the history of input-output pairs up to time t_{n-1}, and $\boldsymbol{\epsilon}_n$ is process or measurement noise. Here, we apply the same-time convention ($\boldsymbol{u}_n \to \boldsymbol{y}_n$) instead of the one-step-delaying convention ($\boldsymbol{u}_n \to \boldsymbol{y}_{n+1}$). Given this setup, the joint probability distribution of the future input-output sequence from t_1 to t_N, conditioned on the initial data $\mathcal{D}_0$, can be factorized using the chain rule of probability (Peterka, 1981):

$$p(\mathcal{D}_{1:N}|\mathcal{D}_0) = \prod_{n=1}^{N} p(\boldsymbol{y}_n|\boldsymbol{u}_n, \mathcal{D}_{0:(n-1)}) p(\boldsymbol{u}_n|\mathcal{D}_{0:(n-1)}), \tag{II.78}$$

where $\mathcal{D}_n = \{\boldsymbol{u}_n, \boldsymbol{y}_n\}$ denotes the input-output pair at time step t_n. The conditional probability density $p(\boldsymbol{u}_n|\mathcal{D}_{0:(n-1)})$ describes the process of determining the input $\boldsymbol{u}_n$ on the basis of the known past history. The left part $p(\boldsymbol{y}_n|\boldsymbol{u}_n, \mathcal{D}_{0:(n-1)})$ represents the dependence of the output on the corresponding input and the known past history, which can be interpreted as the system function g. Identifying this system consists of two steps: model selection and parameter identification. The process of finding the most plausible model from a group of candidates is model selection. Then, system identification focuses on inferring parameters of a chosen system model.

5.1.1 Bayesian Model Selection

Among different model structures $\{\mathcal{M}_i\}$, $i = 1, \ldots, |\mathcal{M}|$, the probability density of the model $\mathcal{M}_i$ given the data $\mathcal{D}$ measures whether the model is suitable for the dataset, which can also be written using Bayes' theorem:

$$p(\mathcal{M}_i|\mathcal{D}) = \frac{p(\mathcal{D}|\mathcal{M}_i)p(\mathcal{M}_i)}{p(\mathcal{D})}. \tag{II.79}$$

Model selection is also as known as model comparison. The relative probability density of two different models given same data $\mathcal{D}$ is:

$$\frac{p(\mathcal{M}_i|\mathcal{D})}{p(\mathcal{M}_j|\mathcal{D})} = \frac{p(\mathcal{D}|\mathcal{M}_i)p(\mathcal{M}_i)}{p(\mathcal{D}|\mathcal{M}_j)p(\mathcal{M}_j)}. \tag{II.80}$$

Since a model has multiple undetermined parameters, the distribution $p(\mathcal{D}|\mathcal{M})$ is obtained by integral of all variants of parameters. The integral is usually untractable. Instead, Deviance Information Criterion (DIC) (Spiegelhalter et al., 2002, 2014; Li et al., 2025b) is applied as the model selection criterion. A deviance is defined as follows:

$$\mathrm{dev}(\boldsymbol{\theta}) = -2 \log p(\mathcal{D}|\boldsymbol{\theta}, \mathcal{M}), \tag{II.81}$$

where $\boldsymbol{\theta}$ denotes the parameters of the model $\mathcal{M}$. Then, the DIC is defined as:

$$\mathrm{DIC}(\mathcal{M}) = \mathbb{E}_{\mathcal{M}}[\mathrm{dev}(\boldsymbol{\theta})] + P_D(\mathcal{M}), \tag{II.82}$$

where the first term is interpreted as a Bayesian measure of model fit:

$$\mathbb{E}_{\mathcal{M}}[\mathrm{dev}(\boldsymbol{\theta})] = \int p(\boldsymbol{\theta}|\mathcal{D}, \mathcal{M})\mathrm{dev}(\boldsymbol{\theta})d\boldsymbol{\theta}, \tag{II.83}$$

and the second term is used to measure the model complexity:

$$P_D(\mathcal{M}) = \mathbb{E}_{\mathcal{M}}[\mathrm{dev}(\boldsymbol{\theta})] - \mathrm{dev}(\overline{\boldsymbol{\theta}}), \quad \overline{\boldsymbol{\theta}} = \int p(\boldsymbol{\theta}|\mathcal{D}, \mathcal{M})\boldsymbol{\theta}d\boldsymbol{\theta}. \tag{II.84}$$

The posterior $p(\boldsymbol{\theta}|\mathcal{D}, \mathcal{M})$ is obtained by MCMC. DIC balances the goodness of fit (how well the model explains the data) and the model complexity (the number of parameters in the model). A lower DIC value indicates better model fit for the given dataset.

5.1.2 Bayesian Parameter Identification

For parameter identification, we restrict to a simpler model class where the output depends only on the current input and parameters, without explicit history dependence (i.e., a memoryless or static mapping). We define the chosen system model with unknown parameters $\boldsymbol{\theta}$:

$$\boldsymbol{y}_n = f(\boldsymbol{u}_n, \boldsymbol{\theta}) + \boldsymbol{\epsilon}_n. \tag{II.85}$$

The optimal parameters are selected by maximum a posteriori (MAP) estimation:

$$\boldsymbol{\theta}^*_{MAP} = \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta}|\mathcal{D}). \tag{II.86}$$

Applying Bayes' rule (II.2), it becomes:

$$\boldsymbol{\theta}^*_{MAP} = \arg\max_{\boldsymbol{\theta}} \left[\prod_{n=1}^{N} p(\boldsymbol{y}_n|\boldsymbol{u}_n, \boldsymbol{\theta})\right] p(\boldsymbol{\theta}), \tag{II.87}$$

where $\mathcal{D} = \{(\boldsymbol{u}_n, \boldsymbol{y}_n)\}_{n=1}^N$ and $p(\mathcal{D})$ is dropped off since it is a constant with respect to $\boldsymbol{\theta}$. The noise $\boldsymbol{\epsilon}_n$ is commonly assumed to be Gaussian. Thus, the distribution $p(\boldsymbol{y}_n|\boldsymbol{u}_n, \boldsymbol{\theta})$ has a closed form, which makes this term equivalent to a weighted mean squared error. The term of the prior density $p(\boldsymbol{\theta})$ performs as $L2$ regularization. On top of the estimate of parameters, the confidence interval of parameters can also be obtained from the posterior distribution.

5.2 Bayesian State Estimation

Applying the estimated posterior of system parameters again as a prior, and updating the posterior with new data, this is the basic idea of Bayesian state estimation (Barker et al., 1995). State estimation involves inference of the internal system states, which are not directly measurable. Expressing the state estimation problem in a probabilistic framework, the goal is to infer the posterior distribution of the states given observations $p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_{1:n})$. Assuming a Markovian system, the posterior of the state chain is factorized as:

$$\begin{aligned} p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_{1:n}) &= p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_n, \boldsymbol{y}_{1:n-1}) \\ &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_{0:n}, \boldsymbol{y}_{1:n-1})\, p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_{1:n-1})}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})} \quad \text{[Bayes' rule]} \\ &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_n)\, p(\boldsymbol{x}_{0:n}|\boldsymbol{y}_{1:n-1})}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})} \quad \text{[Markov property]} \\ &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_n)\, p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1})}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})}\, p(\boldsymbol{x}_{0:n-1}|\boldsymbol{y}_{1:n-1}), \quad \text{[recursion]} \end{aligned} \tag{II.88}$$

where $p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1})$ is referred as *evolution model*, or *state transition model*, and $p(\boldsymbol{y}_n|\boldsymbol{x}_n)$ is called an *observation model*, or *measurement model*. The posterior $p(\boldsymbol{x}_{0:n-1}|\boldsymbol{y}_{1:n-1})$ is the result of the previous time step. The equation above describes a recursive process of state estimation. States are indexed $0:n$ (including the initial $\boldsymbol{x}_0$); observations $1:n$ (no observation at step zero). The prior $p(\boldsymbol{x}_0)$ is the base case of the recursion.

5.2.1 Bayesian Filters

Considering only one state $\boldsymbol{x}_n$ at a time, the problem is called marginal state estimation, or filtering. The equation becomes:

$$\begin{aligned} p(\boldsymbol{x}_n|\boldsymbol{y}_{1:n}) &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_n)\, p(\boldsymbol{x}_n|\boldsymbol{y}_{1:n-1})}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})} \quad \text{[update]} \\ &= \frac{p(\boldsymbol{y}_n|\boldsymbol{x}_n)}{p(\boldsymbol{y}_n|\boldsymbol{y}_{1:n-1})} \int p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1}) p(\boldsymbol{x}_{n-1}|\boldsymbol{y}_{1:n-1}) d\boldsymbol{x}_{n-1}. \quad \text{[prediction]} \end{aligned} \tag{II.89}$$

Usually, calculation of the above equation is divided into two steps: prediction and update. In the prediction step, the result of state transition from $\boldsymbol{x}_{n-1}$ to $\boldsymbol{x}_n$ is computed through the evolution model. In the update step, the last observation likelihood $p(\boldsymbol{y}_n|\boldsymbol{x}_n)$ weights the predicted belief. These two steps build the Bayesian filtering methods: Kalman filters and particle filters (Chen, 2003).

The Kalman Filter (KF) (Kalman, 1960) can be directly derived from Equation (II.89)

under the assumptions of linear systems and Gaussian noise. For a linear system:

$$\boldsymbol{x}_n = F_n \boldsymbol{x}_{n-1} + \boldsymbol{w}_n, \quad \boldsymbol{w}_n \sim \mathcal{N}(\mathbf{0}, \Sigma_{w_n}), \tag{II.90a}$$

$$\boldsymbol{y}_n = H_n \boldsymbol{x}_n + \boldsymbol{v}_n, \quad \boldsymbol{v}_n \sim \mathcal{N}(\mathbf{0}, \Sigma_{v_n}), \tag{II.90b}$$

the iterative algorithm of filtering is:

$$\textbf{Prediction:} \quad \boldsymbol{x}'_n = F_n \boldsymbol{x}_{n-1}, \qquad \Sigma'_n = F_n \Sigma_{n-1} F_n^T + \Sigma_{w_n}. \tag{II.91a}$$

$$\textbf{Update:} \quad K_n = \Sigma'_n H_n^T (H_n \Sigma'_n H_n^T + \Sigma_{v_n})^{-1}, \tag{II.91b}$$

$$\boldsymbol{x}_n = \boldsymbol{x}'_n + K_n(\boldsymbol{y}_n - H_n \boldsymbol{x}'_n), \qquad \Sigma_n = (I - K_n H_n)\Sigma'_n. \tag{II.91c}$$

For nonlinear systems, the Extended Kalman Filter (EKF) (Simon, 2006) obtains the matrices F_n and H_n by linearizing the nonlinear functions at the current estimate, i.e., $F_n = \nabla_{\boldsymbol{x}} f_n|_{\boldsymbol{x}_n}$ and $H_n = \nabla_{\boldsymbol{x}} h_n|_{\boldsymbol{x}_n}$. In such a way, the EKF can be considered as the first-order approximation to the optimal terms. This approximation, however, can introduce large errors, which may lead to suboptimal performance and sometimes divergence of the filter. The Unscented Kalman Filter (UKF) (Wan and Van Der Merwe, 2000) addressed the approximation issues by carefully choosing a minimal set of samples points, which are called *sigma points*. The sigma points completely capture the mean and covariance of states, which are viewed as Gaussian random variables. The sample points are then propagated through the nonlinear functions, and the mean and covariance of the estimate are reconstructed from the transformed points.

Relaxing the assumptions on model linearity and Gaussian noise, Particle Filter (PF) (Gordon et al., 1993; Elfring et al., 2021) is a more general Bayesian filtering method. The integral in Equation (II.89) is approximated through Monte Carlo estimation, yielding an average of all weighted particles. Each particle represents a possible state of the system, and its weight reflects the likelihood of that state given the observations. In the prediction step, each particle is propagated through the system dynamics. In the update step, weights are adjusted according to each particle based on the likelihood of the observation given the particle's state. Unlike Kalman filters, in each iteration of the PF, a resampling step is included to avoid weight degeneracy, meaning that most particles carry very small weights over time. In the resampling step, particles are replaced with equally weighted ones, replicating high-weight particles and discarding low-weight ones.

In summary, KF is efficient and provides optimal state estimates for linear systems with Gaussian noise. EKF extends KF to nonlinear systems through linearization, but may suffer from approximation errors. UKF, instead of linearization, applies sigma points to better capture the mean and covariance of the state distribution, leading to more accurate estimates for highly nonlinear systems. PF is the most flexible, capable of handling

arbitrary nonlinearities and non-Gaussian noise, but computationally expensive due to the need for numerous particles.

5.2.2 State-Space Models

For modeling system dynamics completely from data, State-Space Models (SSMs) (Durbin and Koopman, 2012) are widely used. An SSM consists of a latent state vector $\boldsymbol{x}_n$ and an observation vector $\boldsymbol{y}_n$. The state vector captures the internal dynamics of the system, while the observation vector represents the measurable outputs. $\boldsymbol{x}_n$ follows a Markov process, meaning that the previous state $\boldsymbol{x}_{n-1}$ carries all information to obtain the current state $\boldsymbol{x}_n$. $\boldsymbol{y}_n$ is solely dependent on the latent state $\boldsymbol{x}_n$. These assumptions lead to the following factorization of the joint distribution:

$$p(\boldsymbol{y}_{1:N}, \boldsymbol{x}_{0:N}) = p(\boldsymbol{x}_0) \prod_{n=1}^{N} p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1}) p(\boldsymbol{y}_n|\boldsymbol{x}_n). \tag{II.92}$$

In the simplest case, the state transition distribution $p(\boldsymbol{x}_n|\boldsymbol{x}_{n-1})$ and the observation distribution $p(\boldsymbol{y}_n|\boldsymbol{x}_n)$ are assumed to be linear Gaussian models:

$$\boldsymbol{x}_0 \sim \mathcal{N}(\mathbf{0}, \Sigma_0), \tag{II.93a}$$

$$\boldsymbol{x}_n = \boldsymbol{F}\boldsymbol{x}_{n-1} + \boldsymbol{w}_n, \quad \boldsymbol{w}_n \sim \mathcal{N}(\mathbf{0}, \Sigma_{w_n}) \tag{II.93b}$$

$$\boldsymbol{y}_n = \boldsymbol{H}\boldsymbol{x}_n + \boldsymbol{v}_n, \quad \boldsymbol{v}_n \sim \mathcal{N}(\mathbf{0}, \Sigma_{v_n}). \tag{II.93c}$$

In the previous section, we have shown estimation of states using Kalman filters with known system matrices ($\boldsymbol{F}$, $\boldsymbol{H}$) and noise covariance matrices ($\Sigma_{w_n}, \Sigma_{v_n}$). When these matrices are unknown, they can be estimated from data using the Expectation-Maximization (EM) algorithm (Dempster et al., 1977). The EM algorithm iteratively maximizes the expected log-likelihood of the complete data (both observed and latent variables) given the current parameter estimates. The algorithm consists of two main steps:

- E-step: Compute the expectations of the latent states using the current parameter estimates:

  $$E_n = E[\boldsymbol{x}_n], \quad E_{n,n} = E[\boldsymbol{x}_n \boldsymbol{x}_n^T], \quad E_{n,n-1} = E[\boldsymbol{x}_n \boldsymbol{x}_{n-1}^T].$$

  The states come from running the Kalman filter (Equation (II.91a), (II.91b)) on the current parameters.
- M-step: Update parameters using the expectations:

$$\boldsymbol{F} = \left(\sum_{n=1}^{N} E_{n,n-1}\right)\left(\sum_{n=1}^{N} E_{n-1,n-1}\right)^{-1}, \quad \boldsymbol{H} = \left(\sum_{n=0}^{N} \boldsymbol{y}_n E_n^T\right)\left(\sum_{n=1}^{N} E_{n,n}\right)^{-1},$$
$$\Sigma_{w_n} = \frac{1}{N}\left(\sum_{n=1}^{N} E_{n,n} - \boldsymbol{F}\sum_{n=1}^{N} E_{n,n-1}^T\right), \quad \Sigma_{v_n} = \frac{1}{N}\left(\sum_{n=0}^{N} \boldsymbol{y}_n \boldsymbol{y}_n^T - \boldsymbol{H}\sum_{n=0}^{N} \boldsymbol{y}_n E_n^T\right).$$

The E-step and M-step are iterated until convergence. The EM algorithm guarantees that the likelihood of the observed data does not decrease with each iteration, leading to convergence to a local maximum. Similar to Kalman filters, for nonlinear models, EKFs and UKFs are applied in the E-step; while for nonlinear and non-Gaussian SSMs, PFs are used to approximate the expectations in the E-step (Durbin and Koopman, 2000).

5.2.3 Stochastic Differential Equations

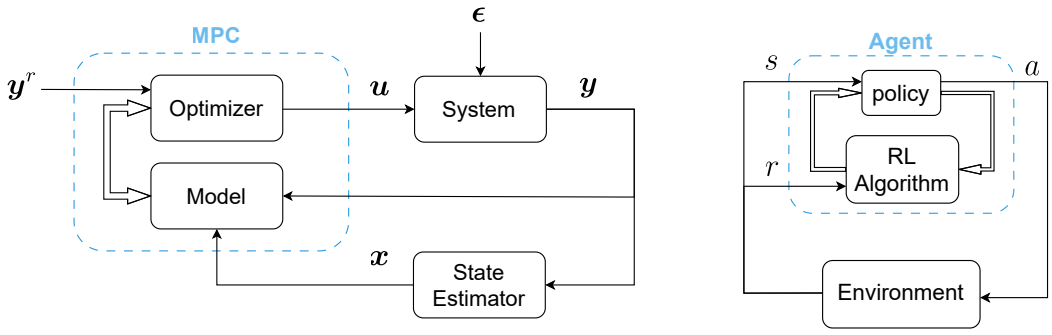
A key strength of SSMs is that they explicitly model latent states and noise as distinct components, when this separation is well-posed, e.g., additive noise, identifiable structures. When we upgrade SSMs from discrete time to continuous time, Stochastic Differential Equations (SDEs) (Øksendal, 1985) are a natural choice for modeling the evolution of latent states. For a stochastic process $X(t)$ for variables $\boldsymbol{x}$, the general form of an SDE is:

$$dX(t) = \mu(X(t), t)\, dt + \sigma(X(t), t)\, dW(t), \tag{II.95}$$

where $dX(t)$ is the infinitesimal change in the state, $\mu(X(t), t)$ is the drift function representing the deterministic part of the dynamics, $\sigma(X(t), t)$ is the diffusion function representing the stochastic part, and $W(t)$ is a Wiener process (introduced in Section 2.2.2) capturing the random fluctuations.

5.3 Model Predictive Control

After investigating Bayesian system identification and state estimation, we now turn our attention to control strategies, meaning optimizing system inputs given target and system observations. Control methods can be broadly categorized into model-based and model-free approaches. Model-based control relies on an explicit model of the system dynamics to predict future states and optimize control actions accordingly. In contrast, model-free control approaches do not require a state transition model of the system, instead learning control strategies directly from interactions with the system. A prominent example of model-based control is Model Predictive Control (MPC) (García et al., 1989; Rawlings et al., 2020).



(a) Schematic of MPC for a system with disturbance $\boldsymbol{\epsilon}$. This figure is adapted from Schwenzer et al. (2021).

(b) Block diagram of RL.

Figure II.8: MPC and RL.

5.3.1 Nominal Model Predictive Control

A to-be-controlled system is at first modeled in discrete time steps:

$$\boldsymbol{x}_{n+1} = f(\boldsymbol{x}_n, \boldsymbol{u}_n), \tag{II.96a}$$

$$\boldsymbol{y}_n = h(\boldsymbol{x}_n), \tag{II.96b}$$

where $\boldsymbol{x}_n$ is the system state, $\boldsymbol{u}_n$ and $\boldsymbol{y}_n$ are the control input and system output at time step n, respectively. The function f describes the state transition, and h is the measurement function.

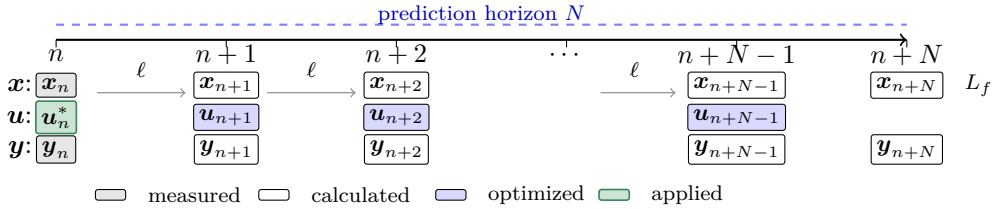


Figure II.9: Time dependency of variables in MPC. The state $\boldsymbol{x}_n$ is measured at the current step. The optimizer finds $\boldsymbol{u}_{n:n+N-1}$; only $\boldsymbol{u}_n^*$ is applied. Dynamics f and output h propagate along the horizon. Stage costs ℓ sum over $i = n, \dots, n+N-1$; L_f is the terminal cost.

Assuming that a full measurement or estimate of the state $\boldsymbol{x}_n$ is available, the objective of MPC is to find an optimal sequence of control inputs $\boldsymbol{u}_{n:n+N-1}$ over a finite prediction

horizon N. The objective function is:

$$\min_{\boldsymbol{u}_{n:n+N-1}} \sum_{i=n}^{n+N-1} \ell(\boldsymbol{y}_i, \boldsymbol{u}_i, \boldsymbol{y}_i^r) + L_f(\boldsymbol{y}_{n+N}^r, \boldsymbol{y}_{n+N}), \tag{II.97a}$$

$$\ell(\boldsymbol{y}_i, \boldsymbol{u}_i, \boldsymbol{y}_i^r) = (\boldsymbol{y}_i - \boldsymbol{y}_i^r)^T Q_y (\boldsymbol{y}_i - \boldsymbol{y}_i^r) + \boldsymbol{u}_i^T Q_u \boldsymbol{u}_i, \tag{II.97b}$$

$$\text{subject to} \quad \boldsymbol{x}_{i+1} = f(\boldsymbol{x}_i, \boldsymbol{u}_i), \quad \boldsymbol{y}_i = h(\boldsymbol{x}_i), \tag{II.97c}$$

$$g_j(\boldsymbol{x}_i, \boldsymbol{u}_i) \leq 0, \quad j = 1, \ldots, n_g, \quad i = n, \ldots, n+N-1, \tag{II.97d}$$

$$\boldsymbol{u}_{min} \leq \boldsymbol{u}_i \leq \boldsymbol{u}_{max}, \quad i = n, \ldots, n+N-1. \tag{II.97e}$$

Figure II.9 illustrates the time dependency of the variables in the MPC formulation. At the current time step n, the state $\boldsymbol{x}_n$ is measured or estimated. The optimizer computes the control sequence $\boldsymbol{u}_{n:n+N-1}$; only the first element $\boldsymbol{u}_n^*$ is applied to the system. The dynamics f and measurement h propagate states and outputs over the prediction horizon. The stage cost ℓ is evaluated at each step $i = n, \ldots, n+N-1$, and the final cost L_f at the terminal step $n+N$.

The cost function consists of a sum of stage costs ℓ and a final cost function L_f. In the stage cost, weighting matrices Q_y and Q_u are used to balance the trade-off between tracking performance and control effort. The reference trajectory $\boldsymbol{y}^r$ represents the desired output over the prediction horizon. The constraints $g_j(\boldsymbol{x}_i, \boldsymbol{u}_i) \leq 0$ can represent physical limitations of the system or other operational constraints. A simplified schematic of MPC is shown in Figure II.8a.

The optimization problem in MPC can be solved using various numerical methods, such as sequential quadratic programming, interior-point methods, or gradient-based approaches (Diehl, 2013). The choice of method depends on the specific characteristics of the system model and constraints. The first step of the solution (Equation (II.97)) $\boldsymbol{u}_n^*$ is applied to the system, and the whole process is repeated at the next time step with updated state information. That's why MPC is also known as receding horizon control (Christophersen, 2007).

5.3.2 Stochastic Model Predictive Control

Taking system uncertainties and disturbances into account, robust MPC (Campo and Morari, 1987) aims to satisfy system stability and constraints despite uncertainties and disturbances. There exist several sound robust MPC methods, including Min-MaxMPC (Raimondo et al., 2009), scenario MPC (Calafiore and Fagiano, 2013), and stochastic MPC (Heirung et al., 2018; Mesbah, 2016). The Min-Max MPC chooses control actions that minimize the worst possible cost under bounded disturbances. The scenario MPC samples a finite number of random uncertainty realizations as scenarios and optimizes control actions

that satisfy constraints for each scenario. Stochastic MPC incorporates the probabilistic descriptions of uncertainties into a stochastic optimal control problem through defining chance constraints. A mathematical formulation of stochastic MPC is as follows (Heirung et al., 2018) :

$$\min_{\boldsymbol{u}_{n:n+N-1}} \sum_{i=n}^{n+N-1} \mathbb{E}[\ell(\boldsymbol{y}_i, \boldsymbol{u}_i, \boldsymbol{y}_i^r)] + \mathbb{E}[L_f(\boldsymbol{y}_{n+N}^r, \boldsymbol{y}_{n+N})], \tag{II.98a}$$

$$\text{subject to} \quad \boldsymbol{x}_{i+1} = f(\boldsymbol{x}_i, \boldsymbol{u}_i, \boldsymbol{w}_i), \quad \boldsymbol{y}_i = h(\boldsymbol{x}_i, \boldsymbol{v}_i), \tag{II.98b}$$

$$P[g_j(\boldsymbol{x}_i, \boldsymbol{u}_i) \leq 0] \geq 1 - \beta_j, \quad \beta_j \in (0, 1], \quad j = 1, \ldots, n_g, \quad i = n, \ldots, n+N-1, \tag{II.98c}$$

$$\boldsymbol{u}_{min} \leq \boldsymbol{u}_i \leq \boldsymbol{u}_{max}, \quad i = n, \ldots, n+N-1. \tag{II.98d}$$

The chance constraints P regularize the inequality constraints g_i, allowing for a small probability β_j of constraint violation. The expectation in the cost function can be approximated using Monte Carlo simulation or analytical methods if the distributions are known.

5.4 Reinforcement Learning for Control

When an accurate model of the system is unavailable or difficult to obtain, model-free control methods can be employed. Classical model-free control methods are Proportional-Integral-Derivative (PID) controllers (Fliess and Join, 2013). In recent years, data-driven control methods attract increasing attention. Especially, Reinforcement Learning (RL) can be viewed as a model-free framework for solving optimal control problems, and has been successfully applied in various domains, including robotics, game playing, and autonomous systems.

We first introduce the terminology of RL:

- agent: the controller,
- environment: the to-be-controlled system,
- action: the control input, a_t,
- state: the complete state of the environment, s_t,
- policy: the control strategy, which can be deterministic ($a_t = \mu(s_t)$) or probabilistic ($a_t \sim \pi(\cdot|s_t)$),
- reward: the feedback on the agent's success, $r_t = R(s_t, a_t)$.

The foundation of RL is the Markov Decision Process (MDP) (Puterman, 1994). An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of

actions, $P(s_{t+1}|s_t, a_t)$ is the state transition probability, $R(s_t, a_t)$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. The interaction process between the agent and the environment at each time step t is as follows: the agent observes the current state s_t, selects an action a_t according to its policy $\pi(a_t|s_t)$, receives a reward $r_t = R(s_t, a_t)$ from the environment, and transitions to a new state s_{t+1} based on the state transition probability $P(s_{t+1}|s_t, a_t)$. The goal of the agent is to learn an optimal policy π^* that maximizes the expected return, i.e., the value function:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]. \tag{II.99}$$

In comparison to other optimal control methods, RL methods aim to optimize long-term sequential decision-making performance. Besides, generality and scalability are the two main advantages of RL methods. They can handle nonlinear, high-dimensional, and stochastic dynamics and also non-quadratic reward functions. The work by Buşoniu et al. (2018) provides a comprehensive review of RL for control.

It is worth noting that there is also an RL method applying a surrogate model for the sake of improving the policy, which is called model-based RL (Sutton, 1990; Moerland et al., 2022). The surrogate model is learned from data in advance. Then, the model can also be used for planning, simulation, or generating synthetic data to augment real-world data. In comparison to the model-free RL, the model-based RL can reduce the required amount of samples gathered directly from the environment. From the viewpoint of control, model-based RL is also referred to learning-based control, when the surrogate model is also adaptive online. From the perspective of leveraging a state transition model, model-based RL can be viewed as a hybrid approach that combines elements of both model-based and model-free control strategies.

RL provides a flexible framework for learning control policies directly from data, enabling adaptation to changing environments and complex system dynamics. However, it often requires a large amount of data and may suffer from explainability, stability, and reliability issues. In safety-critical applications, model-based control methods are often preferred.

Chapter III

Literature Review

This section reviews the relevant literature about applications of deep probabilistic models in three aspects: Bayesian parameter identification; Bayesian state estimation; deep generative models as control policies. The deep probabilistic models mainly include Deep Gaussian Processs (DGPs), Bayesian Neural Networks (BNNs), Conditional Variational Autoencoders (CVAEs), Conditional Normalizing Flows (CNFs), and also conditional diffusion models. [todo: flow matching]

1 Bayesian Parameter Identification

1.1 Deep Gaussian Processes

Deep Gaussian Processs (DGPs) are sequential compositions of Gaussian processes, where the outputs of one Gaussian process are the inputs to the next. They extend the single-layer Gaussian process regression (GPR) and can represent compositional or hierarchical structure of variables. A survey (Jakkala, 2021) summarizes the improved methods of DGPs.

Abraham and Deo (2023) proposed a DGP-based method for Bayesian inference of compositional parameters from Partial Differential Equations (PDEs). The parameters are assigned with a DGP prior which models compositional functions. Then, the posterior distribution of the parameters is optimized via variational inference.

Applications of DGPs for Bayesian inference span multiple domains. For instance, Park et al. (2018) utilized DGPs to identify contaminant release sources, while Desai et al. (2023) employed them for air quality estimation.

In biomedical contexts, Alaa and van der Schaar (2017) leveraged DGPs to assess patient survival under competing risks.

These studies show promising results in comparison to GPRs as well as other machine learning methods, and highlight the versatility and effectiveness of DGPs in addressing parameter inference challenges. However, the computational cost and storage cost remain the main drawbacks of DGPs, especially for high-dimensional data.

1.2 Bayesian Neural Networks

Applying Bayesian Neural Networks (BNNs) for Bayesian inference is usually combined with other methods for better extracting information from observation data. For example, Yang et al. (2021) integrated a BNN with Physics-Informed Neural Networks (PINNs) (B-PINN) to infer parameters in Partial Differential Equations (PDEs) with Gaussian noise. PINNs construct a neural network to approximate the forward solution to the underlying ordinary differential equation (ODE) or PDE problem by minimizing reconstruction loss of data, while regularized by the underlying governing ODE or PDE residual loss. Meanwhile, model parameters can also be estimated probabilistically via Markov Chain Monte Carlo (MCMC) or variational inference. The B-PINN achieved more accurate results with stronger noise in observations.

Merkatas and Särkkä (2023) used an autoregressive BNN for estimating the system parameters, together with a nonparametric noise model to capture complex noise characteristics in the data. The BNN network parameters are approximated through Hamiltonian Monte Carlo (HMC) sampling approach. BNNs are flexible to be integrated with other models and can provide uncertainty quantification, at the cost of higher computational complexity. On the other hand, applying BNNs for Bayesian inference is not straightforward and must be combined with other architectures. Besides, the uncertainty of network weights does not necessarily translate to the uncertainty of the target variables.

1.3 Conditional Variational Autoencoders

Conditional Variational Autoencoders (CVAEs) represent a conditional extension of the Variational Autoencoder (VAE) framework. CVAEs have been widely adopted for amortized Bayesian inference across various scientific domains. For example, Nautiyal et al. (2024) introduced a CVAE-based approach for efficient Bayesian inference in scientific simulators.

In astronomy, CVAEs have enabled Bayesian parameter estimation from gravitational wave data, as demonstrated by Yamamoto and Tanaka (2020), Gabbard et al. (2022), and Sun et al. (2025). Additionally, Ferreira and Bejger (2025) applied CVAEs to infer the equation of state of neutron stars.

In biomedical engineering, CVAEs have been used for neuroimaging analysis under varying correlated parameters (2025), and for parameter estimation in mouse cardiac muscle studies (2025).

These examples highlight the versatility and effectiveness of CVAEs for Bayesian inference of high-dimensional data. One shortcoming exhibited in the research (Sun et al., 2025) is that the CVAE tends to estimate broader posterior distributions compared to other methods, indicating a potential limitation in capturing precise uncertainty.

1.4 Conditional Normalizing Flows

BayesFlow (in Section 4.6) (Radev et al., 2020) provides a general framework for amortized Bayesian inference using CNFs. Besides our work, it has also been applied to various scientific domains. For instance, Kucharský and Bürkner (2025) applied BayesFlow in multiple cognitive models. Zeng et al. (2023) demonstrated its effectiveness in damage detection in structural health monitoring. Rahmandad et al. (2025) utilized BayesFlow to infer parameters from system dynamics, such as the random walk model.

Some other works share similar idea with BayesFlow. De Santi et al. (2024) referred the summary network as an embedding neural network. Zhdanov et al. (2022) extended the condition with richer synthetic data, which is generated by a CVAE. Dong et al. (2025) arranged a CNF through taking the conditioning variable as an additional input to each coupling layer and enlarging the latent space as well.

These works demonstrate the effectiveness of CNFs for amortized Bayesian inference. Especially, the design of the summary network needs to be tailored to the specific characteristics of the data, which is crucial for extracting informative representations from observations and inferring parameters precisely.

1.5 Conditional Diffusion Models

Diffusion models have recently emerged as powerful tools for Bayesian inference due to their ability to model complex, multimodal distributions. Zhu et al. (2025) introduced a diffusion-based framework for parameter identification in dynamic power systems, demonstrating improved accuracy over traditional methods.

Conditional diffusion models have also been applied to infer parameters, unobservable states, and system dynamics in PDEs (Haitsiukevich et al., 2024).

Furthermore, Baldassari et al. (2023) developed a framework for conditional score-based diffusion models in infinite-dimensional function spaces to perform Bayesian inference for linear inverse problems.

Chen et al. (2025) proposed a conditional diffusion model for amortized posterior estimation, showing superior performance compared to CNFs, particularly in reconstructing multimodal distributions and capturing variable boundaries. Nevertheless, the computational cost of inference with diffusion models is much higher than other Deep Generative Models (DGMs) due to its iterative sampling procedure.

2 Bayesian State Estimation

After introducing versatile applications of deep probabilistic models for Bayesian parameter identification, this section focuses on the applications of deep probabilistic approaches for

state estimation, including data-assisted Bayesian Filters and deep state space model.

2.1 Data-Assisted Bayesian Filters

From the section 5.2.1 we have known that estimating state from the noisy observations is strongly dependent on the completeness of system models and strong assumptions on noise. Under model mismatches or incompleteness, deep probabilistic alternatives have been proposed to integrate Bayesian filters with offline data. The data-assisted Bayesian filters take advantage of the rigor of the state prediction and update, achieving explainability and better results than solely data-driven methods.

For instance, Yan et al. (2024) introduced a Bayesian gated recurrent framework for non-Markov dynamical systems, which consists of a memory update gate for capturing the temporal regularities in state evolution, a state prediction gate with the evolution mismatch compensation, and a state update gate with the observation mismatch compensation; it achieved stronger state estimation performance than standard Recurrent Neural Networks (RNNs) and Transformer baselines.

Fraccaro et al. (2017) proposed the Kalman VAE, which combines a VAE with a linear Gaussian state-space model. The VAE encoder learns a low-dimensional latent representation of the observations, while the Kalman filter performs recursive Bayesian state estimation in the latent space. The decoder reconstructs the observations from filtered latent states, allowing uncertainty-aware tracking of temporal dynamics. Similar ideas have been explored in (Pasmans et al., 2025; Zhang et al., 2023).

Normalizing Flow (NF) have also been explored to be combined with linear Gaussian state-space model. Wang et al. (2025) utilized a NF to transform nonlinear model with non-Gaussian noise into latent space, where a Kalman filter is operated. The major advantage in NFs compared with VAEs lies in the invertibility, which allows efficient sampling of target filtering distribution in the original space. The authors shown that the flow-based Bayesian filter achieves better accuracy and computational efficiency compared to conventional filtering methods in challenging high-dimensional settings.

Another way of applying NF is proposed by Chen et al. (2021). A CNF is employed to construct flexible proposal distributions in differentiable particle filters. The NF is conditioned on the observed data to migrate particles to areas closer to the true posterior. The integration of CNFs improved the accuracy in a visual tracking task.

2.2 Deep State-Space Models

Constructing the state transition and observation models in State-Space Models (SSMs) using neural networks allows for the representation of a wide range of dynamic behaviors, thanks to their flexibility and expressive capability. The review (Lin and Michailidis,

2024) provides a selective review of recent advancements in deep neural network-based approaches for SSMs, and presents a unified perspective for discrete time deep state space models and continuous time ones such as latent neural ODEs and neural Stochastic Differential Equations (SDEs). Recent advancements in sequence modeling have led to the emergence of structured SSMs as an efficient alternative to RNNs and Transformers, addressing challenges in long-range dependency modeling and computational efficiency. A comprehensive survey about structured SSMs is conducted by Somvanshi et al. (2025).

Similar to the work of Wang et al. (2025), leveraging NFs to model noises in the SSMs also enables fast and accurate system dynamics learning, high-dimensional state estimation, and time-series forecasting. Lin et al. (2025) proposed an efficient transformed Gaussian process SSM that integrates a single shared Gaussian Process (GP) with input-dependent NFs, yielding an expressive implicit process prior that captures complex, non-stationary transition dynamics while significantly reducing model complexity.

3 Deep Probabilistic Models and Time Series Modeling

We also review the combinations of deep probabilistic models with time series modeling architectures, which provide us inspirations to extend normalizing flows in state estimation tasks. Besides, we summarize the applications of normalizing flows for modeling stochastic processes, which is closely related to our proposed method FlowSSM in Chapter IV.

3.1 Integration of Deep Probabilistic Models with Time Series Modeling Architectures

Identification of nonlinear system dynamics solely from input-output data is a challenging task with solely time series modeling methods. In contrast, deep probabilistic models, especially DGMs, have shown great potential in capturing complex nonlinear relationships while providing uncertainty quantification. The combinations of both methods have been explored in recent years.

Brusaferri et al. (2019) stacked Long Short-Term Memory (LSTM) layers with a probabilistic layer, obtaining a Bayesian inference model for nonlinear system identification, capturing nonlinear dynamics while providing uncertainty quantification.

Paniagua and López (2024) introduced a framework integrating a VAE with Nonlinear Autoregressive eXogenous (NARX) for identifying nonlinear systems using input/output data. The VAE encodes the input/output data into a latent space, and the NARX decodes the latent space information and reconstructs the system outputs. This approach effectively captures nonlinear relationships effectively and improves the robustness through the VAE in comparison to the NARX alone.

Roeder et al. (2019) proposed an amortized Bayesian inference method for learning hierarchical nonlinear dynamical systems. The method extends the VAE by incorporating the structural information of the system as hierarchical conditions. The encoder performs amortized inference over latent variables conditioned on observed trajectories and system structure. The decoder employs differentiable ODEs to describe the system dynamics, which can include both known mechanistic components and learned neural components. This method enables integrating knowledge of system structure with data-driven learning, improving interpretability and generalization.

3.2 Normalizing Flows and Stochastic Processes

Another viewpoint of probabilistic time series modeling is to consider it as learning the underlying stochastic process that generates the data. In this section, we focus on the research about leveraging NFs to model stochastic processes.

Stochastic Continuous Normalizing Flow (SCNF) (Hodgkinson et al., 2021) treats SDEs as random ODEs. Random ODEs are a type of differential equation where the vector field includes a stochastic process, which is approximated by deterministic functions. Random ODEs are trained in the manner of continuous normalizing flows. However, SCNF suffers from the overfitting issue of the vanishing diffusion term.

Continuous Time Flow Process (CTFP) (Deng et al., 2021b) models stochastic processes by transforming latent Wiener processes via continuous normalizing flows. The continuous normalizing flows are trained in the forward direction through mapping the data samples to the latent variables. Time-Changed Normalizing Flow (Bekri et al., 2024) extends the base Wiener process in CTFP with a time function. Continuous Latent Process Flow (Deng et al., 2021a), as an improvement of CTFP, applies a LatentSDE (Li et al., 2020a) as the encoder to generate latent variables, and a continuous normalizing flow as the decoder. Although these methods shown good performance in generating probabilistic time series, they are computationally expensive and not able to solve initial value problems directly or model dynamical systems with control inputs, which makes them unsuitable for many engineering applications.

Although several works have demonstrated the potential of normalizing flows for modeling stochastic processes, their application to system dynamics modeling and state estimation remains limited. Key challenges include high computational cost, difficulty handling control inputs and initial-value problems, and risks of overfitting or vanishing diffusion. Addressing these issues represents a promising direction for our research.

4 Deep Generative Models as Control Policy

Moreover, deep generative approaches for capturing input-output relationships are well-suited for integration into model-free Reinforcement Learning (RL) frameworks. The modeled uncertainty plays a crucial role in enabling robust decision-making. A survey (Zhu et al., 2024) provides a comprehensive overview of recent research efforts focused on the application of diffusion models within the domain of reinforcement learning. Diffusion models demonstrate significant flexibility and capacity as general-purpose tools for planning, policy generation, and data synthesis. The recent convergence with powerful Large Language Models (LLMs) has significantly enhanced this capability. This integration unlocks the multi-modal strength of diffusion models, enabling them to tackle diverse and complex tasks in robotics (Wen et al., 2025).

However, in specific single-task scenarios, NFs may prove more suitable. NFs often offer a superior balance between computational cost and the strength of accurate density estimation. This is particularly required in domains like physics-based modeling, where the underlying noise distribution is often less complex than in general image/language generation tasks. Especially, Ghugare and Eysenbach (2025) brought up that the probabilistic modeling and inference capability of NFs suits better for RL framework in comparison to diffusion models, including acting as a policy, as a Q-function or value model, and for modeling state densities.

In the following, we summarize different RL-methods equipped with NF or Flow Matching (FM).

Table III.1: Normalizing Flows in Reinforcement Learning

Flow Model	**Role in RL**	**Main Contribution**
Discrete NF (Ward et al., 2019; Mazoure et al., 2020)	Policy of SAC	Improved expressiveness of the policy distribution and efficient exploration of sparse reward.
Neural Spline Flow (Lind et al., 2025)	Visuomotor Policy	Comparable performance to Diffusion Policy and improved sample efficiency in training while up 30x faster in inference.
Neural Spline Flow (Chang et al., 2022)	Expert policy of inverse RL	Achieving good recovery of the true reward signal and imitation from observation.

Continued on next page...

Table III.1: Normalizing Flows in RL (Continuation)

Flow Model	Role in RL	Main Contribution
Causal NF (Cho et al., 2024)	Learning transition dynamics and reward function in offline policy training	Improved generalization in sequential decision-making tasks, revealing a high potential for OoD adaptation in offline RL.
Coupling Flow (Khader et al., 2021; Yin et al., 2022)	Diffeomorphic transfomration from state to action (policy)	Application in complex continuous control tasks while ensuring stability.
Coupling Flow (Rietz et al., 2024)	Constrained policy of SAC	Ensuring constraints satisfactory over reward engineering while offering enhanced interpretability.
Coupling Flow (Tarasov et al., 2025)	Policy in VLA models	Comparable performance to Diffusion Policy and improved parameter efficiency while faster in inference.
RealNVP (Brahmanage et al., 2023)	Policy of DDPG	Significant reduction of constraint violations.
RealNVP (Power and Berenson, 2022)	Learning control policy in MPC	FlowMPPI with projection outperforms state-of-the-art MPC baselines on both in-distribution and OoD environments.
RealNVP (Boborzi et al., 2021)	Policy of DAC	Enabling imitation of complex continuous control policies.
Autoregressive Flow (Agarwal et al., 2022)	Motion planner in IL	Achieving good performance in real-world autonomous driving tasks.
RealNVP (Ma et al., 2020)	Policy representation	Capturing multi-modal action distributions.
RealNVP (Lee and Diwan, 2023)	Policy gradient in model-based RL	Capturing multi-modal action distributions.
MAF and Glow (Urain et al., 2020)	Learning dynamics model	Improved representation of stochastic dynamics while ensuring stability.
MAF (Freund et al., 2023)	Distribution matching in IL	Better capturing state-action distribution.
FM (Pivezhandi and Saifullah, 2025)	Augmenting data	Achieving stable convergence in model-based RL.

Continued on next page...

Table III.1: Normalizing Flows in RL (Continuation)

Flow Model	Role in RL	Main Contribution
FM (Xu and Liu, 2024)	Generating trajectories	Outperforming other baselines in continuous and discrete environments with stochastic dynamics.
FM (Lv et al., 2025)	Optimal-behavior constrained policy in online RL	Regularization of learned policy through implicit guidance mechanism; achieving stable training and improved sample efficiency.
FM (Zhang et al., 2025d)	Policy of SAC	Improved performance and sample efficiency in continuous control tasks.
FM (Kong et al., 2025)	Estimating dynamics gap	Improved generalization for learning online dynamics; Outperforming SotA-RL baselines with shifted dynamics.
FM (Zhang et al., 2025a)	Policy in IL	Fast policy generation based on 3D vision; Achieving improved success rates and faster inference time for robotic manipulation.
FM (Rouxel et al., 2025)	Goal conditioned policy	Estimating distributional bounds.
FM (McAllister et al., 2025)	Policy in PGRL	Capturing multimodal action distributions in continuous control tasks.
FM (Wan et al., 2025)	Student & teacher policy in IL	Improved generalization of learned policy and the robustness against sub-optimal expert data.
FM (Zhong et al., 2025)	Modeling value distribution (Critic)	Reduction of gradient variance.
Energy-based NF (Chao et al., 2024)	Policy of Maximum Entropy RL	Integration of the policy evaluation steps with the policy improvement steps, resulting in a single objective training process.
Energy-guided FM (Alles et al., 2025)	Policy for offline RL	Competitive performance and constant training time in flow sampling steps.
Energy-guided FM (Zhang et al., 2025b)	Policy in offline RL	Faster inference time.
Rectified Flow (FM) (Zhang et al., 2025c)	Policy in online RL	Surpassing diffusion policy in continuous robotic control.

Chapter IV

Methodology and Numerical Validation

1 Parameter Identification

As we have introduced in Section 3, different deep probabilistic methods are proposed for Bayesian parameter identification. In real applications, it is usually desired to identify the system parameters from multimodal observations. We start to introduce a universal framework with deep generative models for amortized Bayesian inference. This framework can be seen as a generalization of the BayesFlow (Radev et al., 2020).

1.1 Universal Framework with Deep Generative Models for Amortized Bayesian Inference

Radev et al. (2020) proposed BayesFlow as an amortized Bayesian inference approach with normalizing flows. BayesFlow consists of a summary network for extracting information from the observations, and a conditional normalizing flow for learning the posterior distribution of parameters conditioned on the extracted information. This framework can be generalized to other deep generative models, such as Variational Autoencoder (VAE), Flow Matching (FM), and diffusion models. Figure IV.1 illustrates the framework. The accompanying table (Table IV.1) summarizes how different Deep Generative Models (DGMs) differ in their training objectives and inference mechanisms. Algorithm 1 details the two-stage procedure: amortized learning from data followed by posterior sampling given new observations. We call the summary network here the condition network to emphasize that the embeddings of observations function as condition in the DGMs. The major advantages of applying deep generative models as the Bayesian inference backbone are threefold:

- DGMs consider the parameter identification as posterior inference, which naturally provides trustworthiness of the estimates;
- DGMs seek to model the posterior distribution on a large dataset, which is suitable for amortized Bayesian inference;
- compared with other architectures, like Bayesian Neural Networks (BNNs), DGMs achieve more precise posterior inference because of the separate inference and generation procedures.

In many generative tasks, such as text, image, and video generation, the condition is assumed to be simple. In contrast, for solving inverse problems, such as parameter identification, the generative model is supposed to generate variables under some more complex conditions. Thus, a well-designed condition network is essential to extract most informative features from the observations. This universal framework automatically encourages the condition network to deliver desirable embeddings to the deep generative model. Besides, each observation is only required to be processed once while sampling the trained generative model for multiple times, which is computationally efficient.

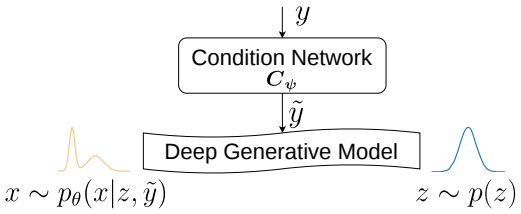


Figure IV.1: Universal Framework of Using Deep Generative Models for Amortized Bayesian Inference. The corresponding algorithm is shown on the right (Algorithm 1).

Model	Training Objective	Inference
VAE	Maximize ELBO	Amortized encoder
NF	Maximize log-likelihood	Direct deterministic mapping
FM	Minimize matching cost (vector field)	Integrate ODE
SBM	Minimize matching cost (score function)	Integrate SDE
DDPM	Minimize matching cost (added noise)	Integrate SDE

Table IV.1: Training paradigms and inference across DGMs.

Algorithm 1 DGM-ABI Training & Inference

Require: Dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{M}$
Require: Prior $p(z)$, Condition network C_ψ
Require: Generator $p_\theta(x \mid z, \tilde{y})$, (Optional: Inference $q_\phi(z \mid x, \tilde{y})$)

Stage 1: Amortized Learning
while not converged **do**
  Sample batch $\{(x, y)\} \sim \mathcal{D}$
  Compute embeddings: $\tilde{y} \leftarrow C_\psi(y)$
  Sample latent: $z \sim p(z)$
  Compute objective gradient:
  $g \leftarrow \nabla_{\theta,\psi,\phi} \mathcal{L}$
  Update parameters: $(\theta, \psi, \phi) \leftarrow (\theta, \psi, \phi) - \eta \cdot g$
end while
Stage 2: Bayesian Inference
Given observed condition y_{obs}:
$\tilde{y}_{\text{obs}} \leftarrow C_\psi(y_{\text{obs}})$
for $k = 1$ to K **do**
  Sample $z^{(k)} \sim p(z)$
  Generate $x^{(k)} \sim p_\theta(x \mid z^{(k)}, \tilde{y}_{\text{obs}})$
end for
return Posterior samples $\{x^{(k)}\}_{k=1}^{K}$

1.2 Normalizing Flows for Parameter Identification

Why are Normalizing Flows (NFs) more suitable for parameter identification tasks given time series observations? In this section, we will answer this question in terms of (1) inference accuracy and precision, (2) reliability, i.e., whether the ground truth lies in the

posterior, (3) speed, i.e., inference computational time, (4) sample efficiency, i.e., posterior contraction while increasing data amount, (5) correlation, i.e., identifying linearly correlated parameters, (6) complexity, i.e., handling multimodal distributions.

1.2.1 Overall Settings

In this section, we take a classic mass-spring-damper system as an example, which has three parameters including mass m, spring constant k, and damping coefficient c. As observations, we take the position and velocity trajectories. For evaluation of (1) - (4), an external force is introduced. The ordinary differential equation (ODE) is:

$$\dot{x} = v,\ \dot{v} = \frac{1}{m}(F_t - kx - cv),\ F_t = 10 \cdot \sin(3t). \tag{IV.1}$$

For the (5) linear correlation case, the external force is removed, and the parameters m and k are correlated, i.e., $k = 10 \cdot m$. To investigate the (6) multimodal case, the system is extended into the topology Wall – k_1 – m – k_2 – m – k_3 – Wall. When the parameters m, k_2, and c are fixed as constant, and we take the middle points of the positions and velocities of the two masses as observations, the value of k_1 and k_3 cannot be uniquely distinguished. The ODE becomes

$$\dot{x}_1 = v_1,\ \dot{v}_1 = \frac{1}{m}(-k_1 x_1 + k_2(x_2 - x_1) - cv_1); \tag{IV.2}$$

$$\dot{x}_2 = v_2,\ \dot{v}_2 = \frac{1}{m}(-k_3 x_2 - k_2(x_2 - x_1) - cv_2). \tag{IV.3}$$

We compare two normalizing flow architectures Real-Valued Non-Volume Preserving (RealNVP) and Neural Spline Flow (NSF) with FM, Denoising Diffusion Probabilistic Model (DDPM), and Score-based Diffusion Model (SBM), as well as VAE and BNN. As the condition network, we use an Long Short-Term Memory (LSTM) layer followed by two linear layers with Exponential Linear Unit (ELU) activations. For a group of estimated $\{\widehat{x}^{(m)}\}_{m=1}^{M}$ and true parameters $\{x^{(m)}\}_{m=1}^{M}$ with the mean of the true parameters $\overline{x}$, the following evaluation metrics are applied:

- Accuracy: Root Mean Squared Error (RMSE) and R^2 score:

$$RMSE = \sqrt{\frac{1}{M}\sum_{m=1}^{M}(x^{(m)} - \widehat{x}^{(m)})^2}\,, \tag{IV.4}$$

$$R^2 = 1 - \sum_{m=1}^{M}\frac{(x^{(m)} - \widehat{x}^{(m)})^2}{(x^{(m)} - \overline{x})^2}\,. \tag{IV.5}$$

- Precision: Mean Confidence Interval Width (MCIW). For the m-th parameter by sampling it L times, the standard deviation is $\sigma_{\widehat{x}^{(m)}}$, and we have:

$$CI^{(m)}(95\,\%) = [L_m, U_m] = \widehat{x}^{(m)} \pm 1.96\frac{\sigma_{\widehat{x}^{(m)}}}{\sqrt{L}}. \tag{IV.6}$$

  Then the MCIW can be obtained by:

$$MCIW = \frac{1}{M}\sum_{m=1}^{M}(U_m - L_m). \tag{IV.7}$$

- Reliability: Confidence Interval Coverage Probabiltiy (CICP). The larger CICP indicates higher reliability of estimation.

$$CICP = \frac{1}{M}\sum_{m=1}^{M} c_m,\ c_m = \begin{cases} 0 & \text{if } x^{(m)} \notin [L_m, U_m], \\ 1 & \text{otherwise.} \end{cases} \tag{IV.8}$$

1.2.2 Performance Comparison (Winner: RealNVP)

Table IV.2: Performance Benchmarking of Deep Bayesian Inference Methods on the Mass-Spring-Damper System. All metrics represent averages across the three system parameters. Predictive accuracy is assessed via RMSE (lower is better) and R^2 (higher is better). Uncertainty quantification is evaluated through the CICP and MCIW together. Ideal performance is characterized by a CICP near the 95 % calibration target; significantly lower values indicate overconfidence, while excessively high MCIW values indicate model imprecision. Inference time denotes the total computational time for a single test scenario based on 100 posterior samples.

Model	**RMSE** ↓	R^2 ↑	**CICP** ↑	**MCIW** ↓	**Inference Time** (s) ↓
BNN	0.0393	0.9965	0.9900	0.1874	0.0209
VAE	0.0287	**0.9987**	0.7250	**0.0497**	**0.0018**
DDPM	0.0915	0.9852	0.9883	0.5338	0.2310
SBM	0.0699	0.9909	0.9700	0.3289	0.0534
FM	0.0547	0.9938	**0.9933**	0.3730	0.0050
RealNVP	**0.0278**	0.9986	0.9917	0.1223	0.0043
NSF	0.0401	0.9966	0.9917	0.1845	0.0069

In Table IV.2, the evaluation of different models is shown. The observed time series are from 0 s to 1 s with the time step of 0.05 s, and no noises are added onto them. In terms of accuracy, RealNVP (RMSE: 0.0278) and VAE (RMSE: 0.0287) are the most accurate models. Their highest R^2 scores indicate that they explain almost all the variance

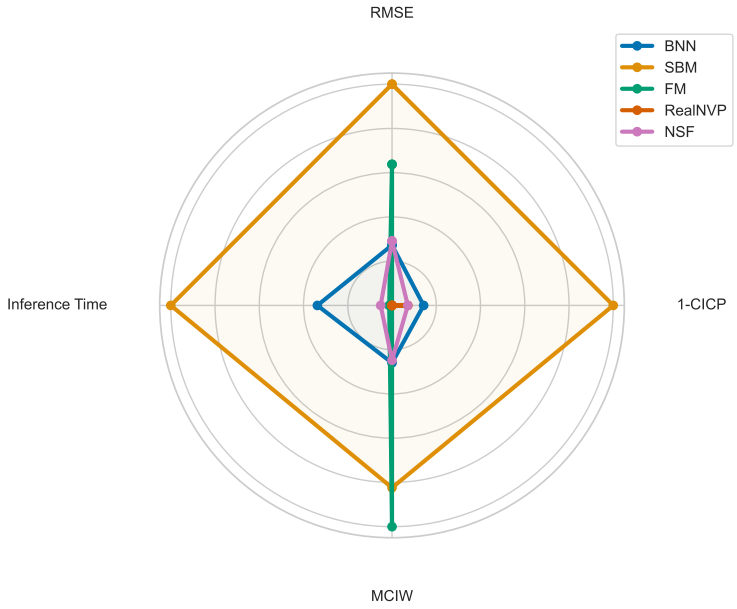


Figure IV.2: Detailed Comparison of Deep Bayesian Inference Methods on the Mass-Spring-Damper System. The values are the same as those in Table IV.2. 1-CICP is shown for the sake of consistency of comparison, meaning that for all metrics in the spider chart, lower is better. The center of the radar plot represents optimal performance.

in the mass-spring-damper system. DDPM (RMSE: 0.0915) has the highest error in this group. While an R^2 of 0.985 is still objectively good, it is significantly less precise than the NFs (RealNVP, NSF) for this specific physical task. Overall, the flow-based methods (RealNVP, NSF, and FM) achieve higher accuracy than diffusion-based methods (DDPM and SBM). The BNN is comparable with the NFs.

In terms of uncertainty quantification, almost all models (except VAE) show CICP values near 1.0. Theoretically, the ideal CICP with 95 % confidence intervals should also be 0.95. Higher CICPs mean that they are conservative and under-confident, whereas the lower CICPs indicate overfitting. Besides, in an ideal noise-free scenario, the posterior distribution for the mass-spring-damper parameters should approach a Dirac delta function. However, in the practice of solving inverse problems, the interval should never be zero, because different parameters might produce very similar trajectories. Moreover, the models themselves have uncertainty due to limited data. Therefore, the CICPs higher than 0.95 mean that the inference models prioritize the reliability over precision. Consequently, the MCIW should be considered together with CICP. VAE (CICP: 0.7250 and MCIW: 0.0497) is overconfident. Its intervals are so narrow that the truth falls outside of them 27.5 % of the cases. DDPM (CICP: 0.9883 and MCIW: 0.5338) is imprecise. It achieves high coverage simply by producing very wide intervals.

In terms of computational efficiency, VAE exhibits the highest inference speed. Flow-based methods demonstrate comparable performance to one another, though they are

approximately 5× slower than the VAE baseline. BNN follows, showing higher latency than the flow-based models but maintaining a clear advantage over SBM. Finally, DDPM is identified as the most computationally intensive approach, recording the slowest inference times.

With VAE and DDPM excluded due to miscalibration and computational overhead respectively, Figure IV.2 visualizes the performance trade-offs of the remaining five architectures. RealNVP exhibits the most optimal profile, represented by the smallest polygon area; it successfully minimizes RMSE and MCIW while maintaining near-perfect coverage. NSF and FM show comparable speed and accuracy but yield less precise (wider) credible intervals. The BNN maintains high reliability (low 1-CICP) but suffers from increased inference time. Finally, SBM shows the least favorable performance, characterized by the largest predictive error, the widest uncertainty bounds, and the longest computational time, indicating a difficulty in parameter identification compared to flow-based alternatives.

1.2.3 Sample Efficiency (Winner: RealNVP)

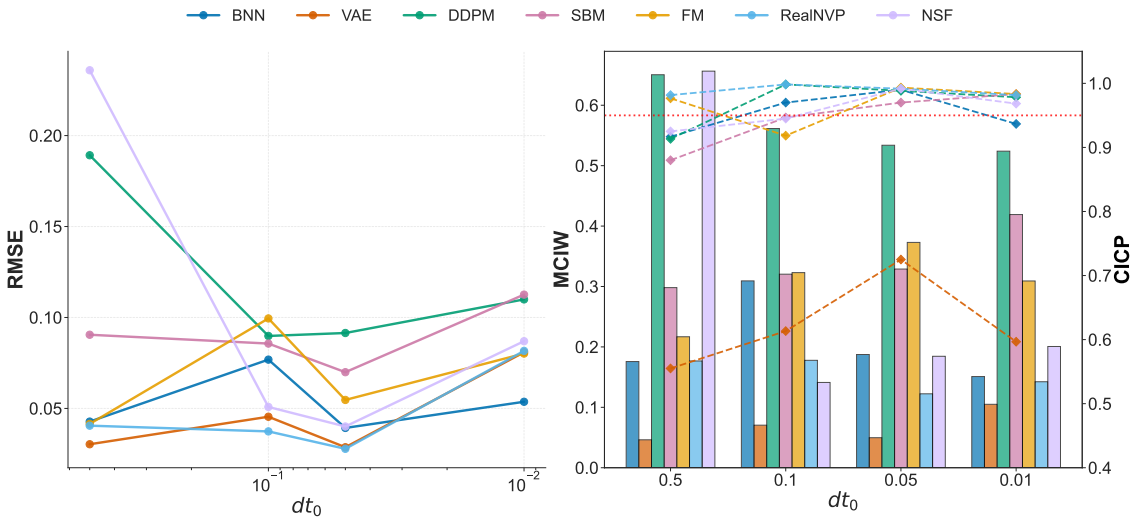


Figure IV.3: Sample Efficiency Comparison of Deep Bayesian Inference Methods under Varying Observation Densities. The left panel illustrates the predictive accuracy (RMSE) across time step $dt_0 = [0.5\,\text{s},\ 0.1\,\text{s},\ 0.05\,\text{s}$ and $0.01\,\text{s}]$ plotted on a logarithmic scale where smaller dt_0 represents higher sampling density. The right panel provides a hybrid uncertainty analysis: the grouped bars indicate the MCIW (uncertainty precision), while the superimposed dashed lines represent the CICP (calibration reliability). The horizontal dotted line denotes the 95 % coverage target.

Figure IV.3 shows how observation density affects posterior estimation. As dt_0 de-

creases from 0.5 s to 0.05 s, all models show consistent improvement in predictive accuracy, attributed to improved identifiability of the system dynamics. Accuracy degrades at $dt_0 = 0.01$ s—likely because the encoder is not designed for long time series. Adding more points adds redundancy without new constraints; this can induce overfitting in the latent space, where the model interprets numerical rounding errors or minor architectural biases as significant physical features. For this system, 0.05 s is the optimal resolution. BNN, RealNVP, and VAE are less sensitive to sample efficiency; diffusion-based methods and NSF require more data. While almost all models reach their accuracy peak at $dt_0 = 0.05$ s, FM favors lower resolution, possibly because vector-field learning benefits from larger differences between data samples.

Regarding uncertainty quantification, VAE is overfitted with narrow intervals and inadequate coverage. As data density increases, DDPM and RealNVP show consistent posterior shrinkage—as the likelihood becomes more peaked, the posterior contracts around the true parameters. Other architectures exhibit more volatile trends: the MCIW of BNN initially increases before dropping; NSF shows a decrease-then-rise pattern; FM and SBM show paradoxical MCIW increases, as the longer sequences introduce numerical complexity or perceived contradictions in the latent space that temporarily outweigh the additional information. The reliability requirement (CICP ≥ 0.95) is hardest under sparse sampling. At $dt_0 = 0.5$ s, only RealNVP and FM are calibrated, highlighting their robustness for sparse trajectories. As density improves to $dt_0 = 0.1$ s, FM temporarily drops out of the reliable regime (its MCIW rise not aligned with predictive accuracy), while SBM and NSF reach the target via distinct mechanisms—SBM through corrective expansion of posterior variance, NSF through efficient shrinkage. At $dt_0 = 0.05$ s, all models satisfy the requirement; 0.05 s provides sufficient information without numerical saturation.

At the highest density ($dt_0 = 0.01$ s), models break down. DDPM and BNN continue to shrink MCIW; for BNN, intervals shrink faster than error, causing it to miss the ground truth and become overconfident. Conversely, SBM, RealNVP, and NSF expand MCIW defensively, as the numerical difficulty of processing high-frequency noise-free data broadens uncertainty bounds to maintain coverage. FM reverses its trend and eventually finds a more stable numerical representation than the other flow variants. In summary, more observations do not always improve performance. RealNVP is most precise under sparse data (excluding VAE); BNN shrinks bounds consistently but its variance may reflect weight convergence rather than physical correctness. Notably, at $dt_0 = 0.1$ s, SBM and NSF define the boundary of the reliable regime, whereas BNN and DDPM remain under-calibrated despite increased data density.

1.2.4 Linear Correlation (Winners: SBM, RealNVP, NSF)

When parameters are linearly correlated, the observations only constrain their ratio, rendering the individual parameters non-identifiable. While DGMs are capable of capturing these dependencies, the task is computationally harder than identifiable ones. The ill-posedness manifests as a manifold of equivalent solutions, making it difficult for optimization algorithms to distinguish between physically distinct systems that produce identical observations. When the correlation between m and k are fixed to be $k = 10 \cdot m$, the damping coefficient should also be proportional to the mass, i.e., $c \propto m$. The damping coefficient c is related to the damping ratio ζ (which determines if the system is under-damped, critically damped, or over-damped) by the formula:

$$c = 2\zeta\sqrt{mk}. \tag{IV.9}$$

Therefore, the following relation holds:

$$c/m = 2\sqrt{10}\,\zeta. \tag{IV.10}$$

Figure IV.4 shows two identification examples of linear correlated parameters.

As illustrated in Figure IV.4, the BNN and DDPM fail to capture the underlying dependencies, resulting in wide, uninformative posterior clouds that lack alignment with the physical constraints. While the VAE recovers the k/m correlation, it fails significantly on c/m, where the posterior mass is biased and poorly aligned with the diagonal line. In contrast, the remaining four methods demonstrate higher fidelity.

Among the successful models, SBM and the NF architectures (RealNVP and NSF) show their superior capability in identifying and enforcing linear parameter correlations. Their k/m estimates are tightly aligned with the theoretical diagonal, and their c/m estimates manifest as narrow ellipses centered on the ground truth. On the other hand, FM successfully captures the linear trend, but it exhibits noticeably higher variance compared to SBM, RealNVP, and NSF, particularly in the c/m correlations. This suggests that while FM identifies the correlation, it is less precise in enforcing the constraint than other architectures.

Linear correlations represent a low-dimensional manifold within the parameter space. Figure IV.4 demonstrates that NFs (RealNVP/NSF) and SBM are superior at manifold learning by effectively collapsing the posterior mass onto a line, whereas BNN and DDPM struggle to resolve such sharp constraints. Besides, the VAE's failure in the c/m correlation is a classic symptom of posterior collapse or architectural under-fitting for specific parameter dimensions.

The performance gap between the two diffusion models is particularly striking: SBM

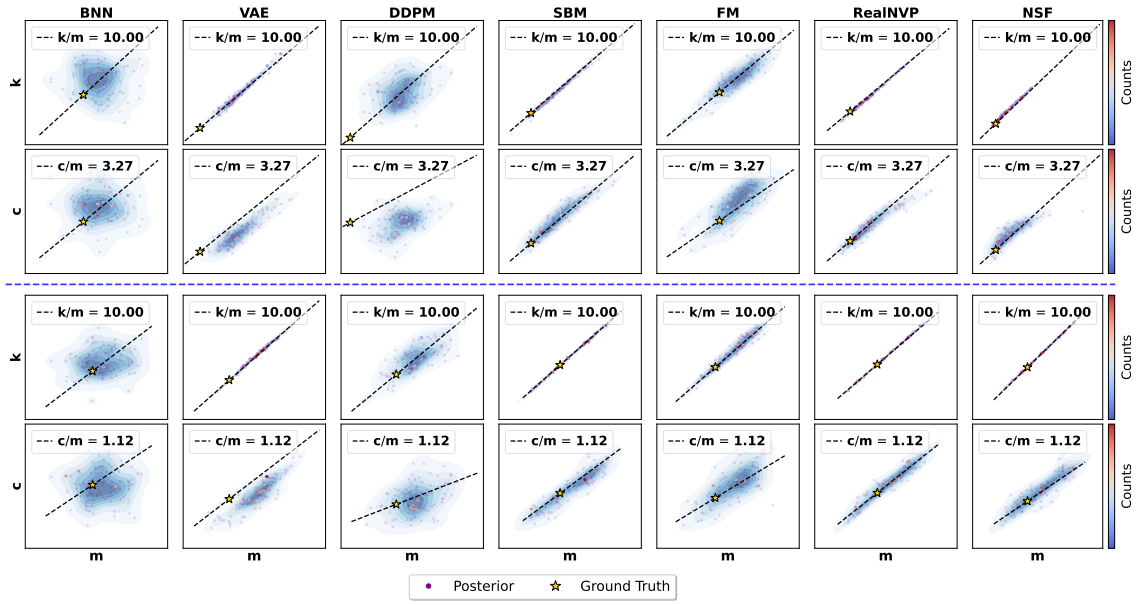


Figure IV.4: Comparative analysis of deep Bayesian inference architectures in identifying linearly correlated parameters. The 4×7 grid displays the joint posterior distributions for seven different models (columns) across two test scenarios (separated by a horizontal blue dashed line in the middle). Rows 1 and 3 visualize the $m - k$ correlation, while Rows 2 and 4 visualize the $m - c$ correlation. Within each sub-figure, the gold star represents the ground truth, and the black dashed line illustrates the theoretical linear constraint (k/m or c/m). Individual posterior samples are plotted as purple points, overlaid on a logarithmic density heatmap where warm hues indicate regions of high probability mass. Note: To ensure high visibility of the posterior topology across models with varying precision, axis limits and density color-scales are individually optimized for each sub-figure. Each color map suits the whole row.

exhibits a far greater capability for recovering precise physical constraints than DDPM. Despite DDPM's established success in generating high-dimensional complex data, it fails to resolve the specific linear parameter correlations in this task. Similarly, despite their theoretical proximity, FM appears more dispersed than SBM. This is primarily because FM is a deterministic transport method that is more sensitive to cumulative approximation errors along the integration path, whereas the stochastic nature of SBM provides a corrective mechanism through noise-driven sampling.

Within the NF architectures, a notable distinction exists between RealNVP and NSF when we zoom in on the c/m figures. While both successfully identify the correlation, RealNVP exhibits a higher degree of centralization along the diagonal correlation axes. While this leads to a narrower, line-like distribution with high precision, it also places the

ground-truth (yellow star) at the extreme periphery of the high-density core. In contrast, NSF estimates the density with higher variance, which is less desirable in this case.

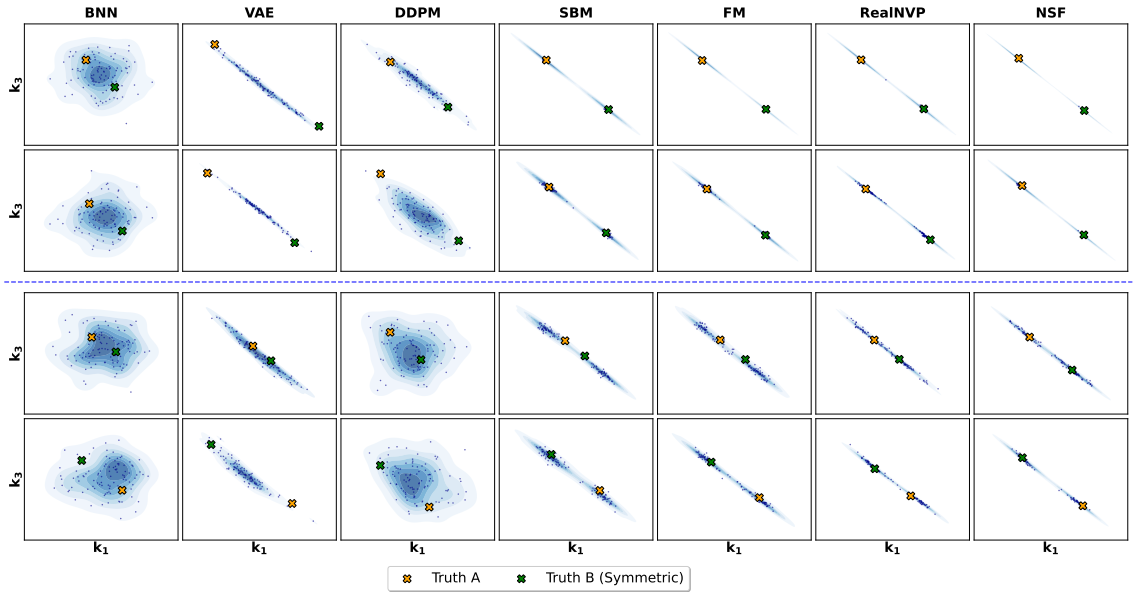


Figure IV.5: Comparative evaluation of deep Bayesian architectures in recovering multimodal posterior distributions. The 4×7 grid displays the joint posterior of k_1 and k_3 for seven different models (columns) across four distinct test scenarios (rows). The top two rows represent scenarios with well-separated modes, while the bottom two rows represent scenarios with proximal (overlapping) modes (partitioned by a blue dashed line). Within each subplot, the orange cross denotes the target ground truth, while the green cross represents the symmetric ground truth (swapped k_1 and k_3 values), highlighting the models' ability to capture the inherent exchangeable symmetry of the parameter space. Blue contours represent the estimated posterior density. Note: To ensure high visibility of the posterior topology across models with varying precision, axis limits and density color-scales are individually optimized for each sub-figure.

1.2.5 Multimodal Distribution (Winner: NSF)

The ability to capture multimodal distributions is a key advantage of applying DGMs for Bayesian inference. Different DGMs exhibit different characteristics. Figure IV.5 illustrates these differences. As test scenarios, we consider two cases: one where the values of k_1 and k_3 are far apart (the first two samples), and another where they are close to each other (the last two samples).

As demonstrated in Figure IV.5, the performance is consistent with the linear correlation results. SBM, FM, NSF, and RealNVP significantly outperform BNN, VAE, and DDPM

in resolving the symmetric multimodal structure. The BNN produces broad, uninformative posterior clouds that fail to recognize any specific multimodal topology. The VAE exhibits over-concentration onto narrow, biased ridges; in all these four scenarios, the ground truth and its symmetric counterpart lie entirely outside the high-density regions, indicating the posterior collapse again. DDPM successfully resolves the two distinct modes only in the first scenario (Row 1), but degrades into a single diffuse, biased cluster in the remaining test cases.

In contrast, SBM and the three NF architectures (RealNVP, NSF, and FM) demonstrate a superior capacity for multimodal recovery. In well-separated scenarios (top two rows), all architectures successfully resolve two distinct high-probability modes. Notably, however, RealNVP exhibits a slight bias, failing to precisely align its right posterior mode with the ground truth. As the modes become proximal (bottom two rows), the identification task becomes increasingly ill-posed, with models struggling to decouple the two ground-truth points and producing a higher density of samples in the interstitial region. While NSF maintains high fidelity by successfully resolving both modes and accurately centering the posterior on the ground-truth coordinates, the remaining three models exhibit varying degrees of dislocation. This suggests that while most architectures can handle discrete multimodality, only NSF remains robust in proximal scenarios where structural symmetry leads to significant parametric overlap.

1.3 Relationship Between Bayesian Inference and Sensitivity Analysis

Previous experiments applying mass-spring-damper models demonstrate that Bayesian inference is intrinsically linked to parameter identifiability. Our findings indicate that spring constants converge rapidly during training, which will be further elucidated through sensitivity analysis. Conversely, linearly correlated parameters lack unique identification, and in symmetric coupled models, individual spring constants become indistinguishable. These varying levels of identifiability can be rigorously analyzed through Sensitivity Analysis (SA), which studies how uncertainty in the output of a mathematical model can be partitioned and allocated to different sources of uncertainty in its inputs. This framework aligns naturally with the objectives of amortized Bayesian inference. For purely data-driven methods, SA provides essential insights into the physical model and practical training suggestions: weakly identifiable or correlated parameters that require additional observational data for resolution.

Unlike local SA, which examines instantaneous changes in output relative to specific parameter perturbations, our research focuses on total trajectories, necessitating a global SA.

1.3.1 Sobol' index

A common approach for global SA is the Sobol' index, which is a variance-based method that decomposes the total output variance into contributions from individual parameters and their interactions. For a model f with inputs $X_1, \ldots, X_d$ and the output Y, the total variance of the model output is:

$$Var(Y) = \sum_i V_i + \sum_{i<j} V_{ij} + \cdots + V_{1,2,\ldots,d}. \tag{IV.11}$$

The first-order Sobol' index S_i^1 measures the contribution of a single parameter X_i to the output variance, which can be regarded as the main effect. It is given by:

$$S_i^1 = \frac{V_i}{Var(Y)}, \quad V_i = Var_{X_i}[E_{\boldsymbol{X}_{\setminus i}}(Y|X_i)]. \tag{IV.12}$$

The total Sobol' index accounts for the total contribution of a parameter to the output variance, including both its main effect and all higher-order interactions with other parameters. The total Sobol' index of X_i is

$$S_i^T = \frac{Var_{\boldsymbol{X}_{\setminus i}}[E_{X_i}(Y|\boldsymbol{X}_{\setminus i})] + \sigma_n^2}{Var(Y) + \sigma_n^2} > \frac{Var_{\boldsymbol{X}_{\setminus i}}[E_{X_i}(Y|\boldsymbol{X}_{\setminus i})]}{Var(Y)} > \frac{Var_{\boldsymbol{X}_{\setminus i}}[E_{X_i}(Y|\boldsymbol{X}_{\setminus i})]}{Var(Y)}. \tag{IV.13}$$

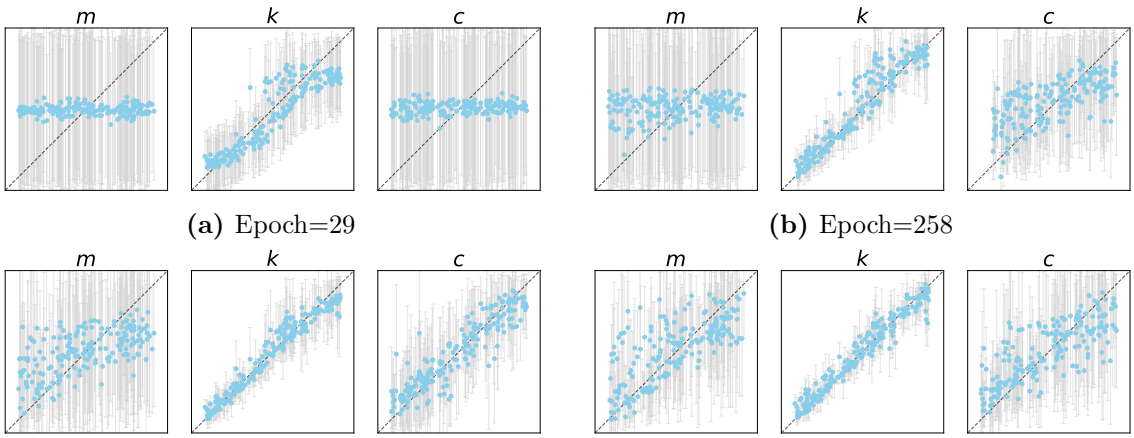
Another key issue of using variance-based SA is to sample the parameter space efficiently. The standard random sampling often creates clusters or gaps in the parameter space. Instead, Sobol' sequences are applied, which are low-discrepancy sequences. For the sake of stability, a larger number of samples should be set.

By comparing these two indices, we can diagnose the structural complexity of the mass-spring-damper system. For instance, a parameter with a high total Sobol' index but a low first-order Sobol' index indicates that while the parameter is influential, its effect is primarily driven by interactions with other components of the model. The linear correlation and multimodality cases are supposed to have zero first-order Sobol' indices and total Sobol' indices of one. If a parameter exhibits a total Sobol' index near zero, it suggests that the model output is insensitive to that parameter across its entire prior range. Such parameters are unidentifiable under the provided observed data.

Ultimately, integrating global SA into the amortized Bayesian workflow allows us to preventively identify blind spots in the model. By detecting parameters with low sensitivity or high correlation, we can strategically focus on data collection efforts or reparameterize the model to ensure that the posterior distributions are well-defined and physically meaningful. When parameters exhibit significantly different levels of identifiability, a sensitivity-guided iterative identification strategy is recommended. In such cases, strongly identifiable

parameters (those with high S_i^1 values, like the primary spring constants) should be identified first to stabilize the model. Once these dominant parameters are constrained, the remaining weakly identifiable or highly correlated parameters can be addressed in subsequent stages by adaptation of observational data or by refining the prior distributions. This hierarchical approach prevents the variance of dominant parameters from obscuring the signals of secondary parameters, ensuring a more robust and physically meaningful calibration of the complex system.

The sensitivity of parameters also reflects the convergence speed of training deep Bayesian inference models as well. Taking a closer look at the mass-spring-damper model with external force, we get very close first-order and total-order indices, i.e., $S_m^1 = 0.07, S_k^1 = 0.88, S_c^1 = 0.03$ and $S_m^T = 0.07, S_k^T = 0.89, S_c^T = 0.05$. It suggests that for the single-degree-of-freedom system under external excitation, the stiffness parameter k dominates the system response and there is hardly any parametric interaction. This means the model is effectively additive in the chosen parameter ranges. From an identifiability perspective, the k should be much easier to estimate than the other two parameters.



(a) Epoch=29 **(b)** Epoch=258

(c) Epoch=393 **(d)** Epoch=951

Figure IV.6: Parameter identification results of RealNVP at the early training stages. The figures show the estimation results of parameters m, k, c. The horizontal axis represents the ground truth, while the vertical shows the estimated values. The perfect convergence is demonstrated by the diagonal dashed line.

Figure IV.6 illustrates the convergence behavior of the three parameters during the early stages of training. By the 29th epoch, the stiffness k begins to converge toward the diagonal parity line, whereas the other two parameters remain unresolved, centered at the mean of their respective ranges. At the 258th epoch, c begins to show a slight tendency toward convergence, while k is further refined; however, m shows no significant

improvement. By the 393rd epoch, m also begins to converge. Finally, at the 951st epoch, m and c reach a similar level of accuracy, while the estimate for k continues to improve. This development reflects the sensitivity of parameters. It is worth noting that faster convergence of c than m results from the larger data range of c. The convergence speed of both parameters eventually synchronizes.

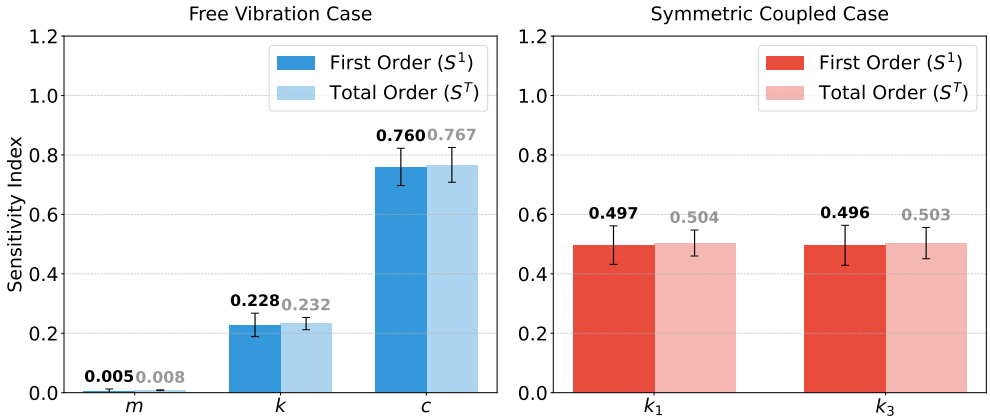


Figure IV.7: Global SA comparison in free-vibration and symmetric coupled mass-spring-damper systems. The bar charts display the first-order (S^1) and total-order (S^T) Sobol' indices, with error bars representing the 95 % confidence interval. (Left) In the free-vibration case, the parameters m, k, c have highly different sensitivities. (Right) In the symmetric coupled two-mass system, k_1 and k_3 exhibit nearly identical sensitivity.

Figure IV.7 compares the sensitivity analysis of the free and coupled mass-spring-models. In the free-vibration case, the external force is removed. The behavior of the mass-spring-model is heavily influenced by how fast the energy loses, which is caused by the damping c. The global SA of the free-vibration system reveals a clear hierarchy of parameter identifiability. The damping coefficient c dominates the output variance ($S_c^T = 0.767$), followed by the stiffness k ($S_k^T = 0.232$), and the mass m exhibits negligible sensitivity ($S_m^T = 0.008$). Furthermore, the near-equality of S^1 and S^T across all parameters indicates a lack of higher-order interactions. This suggests m is weakly identifiable, whereas the other two parameters can be identified independently.

The global SA of the symmetric coupled two-mass system reveals a perfect parity between the boundary stiffness parameters. Both k_1 and k_3 exhibit nearly identical first-order ($S^1 \approx 0.496$) and total-order ($S^T \approx 0.503$) indices. The near-unity sum of their first-order indices ($S_{k_1} + S_{k_3} \approx 1$) suggests that these two parameters account for almost the entire variance of the system's dynamic response, with negligible higher-order interactions ($S^T - S^1 < 0.01$). From an identifiability perspective, this symmetry presents a unique

challenge for Bayesian inference. While the high sensitivity ensures that the parameters will converge quickly away from the prior mean, the identical nature of their influence results in a bimodal posterior distribution. This confirms the indistinguishability and multimodality: the amortized inference model can identify the required stiffness levels but cannot uniquely assign them to k_1 or k_3 without additional asymmetric constraints or localized observations.

1.3.2 Mutual Information

Global SA does not always provide a reliable measure of parameter identifiability, particularly when observations are corrupted by additive noise. Consider a model $f(\boldsymbol{X})$ whose observations contain additive Gaussian noise $\epsilon \sim N(0, \sigma_n^2)$, such that $Y = f(\boldsymbol{X}) + \epsilon$. Assuming the noise is independent of the parameters, the total observed variance becomes $Var(Y) = Var(f(\boldsymbol{X})) + \sigma_n^2$. Under this condition, the first-order and total-order Sobol' indices behave differently:

$$S_i^1 = \frac{V_i}{Var(Y) + \sigma_n^2} < \frac{V_i}{Var(Y)}; \tag{IV.14}$$

$$S_i^T = \frac{Var_{\boldsymbol{X}_{\setminus i}}[E_{X_i}(Y|\boldsymbol{X}_{\setminus i})] + \sigma_n^2}{Var(Y) + \sigma_n^2} > \frac{Var_{\boldsymbol{X}_{\setminus i}}[E_{X_i}(Y|\boldsymbol{X}_{\setminus i})]}{Var(Y)}. \tag{IV.15}$$

As the noise ratio increases, the first-order Sobol' index decreases because the denominator grows while the numerator remains constant. In contrast, the total-order Sobol' index increases by adding the variance of noise in both numerator and denominator. This behavior is highly counter-intuitive: a higher total Sobol' index usually suggests that a parameter is more influential, yet in this case, the increase is driven entirely by noise, which actually makes the parameter more difficult to identify. Consequently, variance-based metrics can fail to distinguish between true parameter sensitivity and the presence of measurement uncertainty. To address these limitations, Mutual Information (MI) (Bhola and Duraisamy, 2023) offers a robust alternative for evaluating the practical identifiability of parameters. Unlike global SA, which only considers the second moment (variance), MI is an information-theoretic measure that quantifies the total information gain from the prior to the posterior distribution given the data:

$$MI(X_i, y) = H(p(X_i)) - H(p(X_i|y)), \tag{IV.16}$$

A comparison between the total-order Sobol' index and MI across varying noise levels is shown in Figure IV.8. While S^T suggests a rising sensitivity as σ increases, which results from the noise contributing to the total variance, the MI captures the actual degradation of the signal-to-noise ratio. The divergence between these metrics demonstrates that

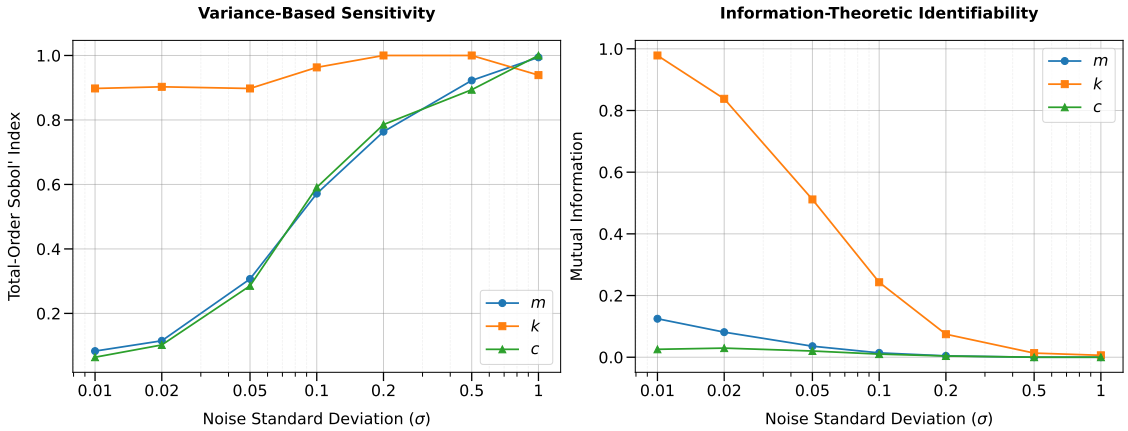


Figure IV.8: Comparison of parameter sensitivity and identifiability under varying noise levels. The left panel displays the total-order Sobol' index, while the right panel shows the Mutual Information (MI) for the parameters m, k, c of a forced mass-spring-damper model. Both metrics were estimated using 1024 samples generated from a low-discrepancy sequence. The horizontal axis represents the noise ratio (scaling a standard normal distribution) on a logarithmic scale.

sensitivity can be a misleading proxy for identifiability in noisy environments. Beyond a threshold of $\sigma = 0.02$, the sharp drop in MI indicates that the parameters cannot be identified as perfectly as the clean observations. Beyond a threshold of $\sigma = 0.2$, all parameters are poorly identifiable, meaning the observations no longer provide sufficient information to constrain the prior.

1.4 When Does the Framework Fail?

From the previous sections, we have known the framework frequently struggles with posterior identifiability; specifically, when the system exhibits multimodality, generative models may fail to resolve disjoint parameter regimes. Apart from identifiability, the efficacy of amortized Bayesian inference via deep generative models is intrinsically tied to the fidelity of the latent representations extracted by the condition network. This framework encounters significant bottlenecks under three primary conditions. First, in the presence of high-dimensional, noisy, or highly stochastic time-series data, the condition network often fails to distill sufficient summary statistics, leading to information loss. Second, the framework is susceptible to model misspecification. When a discrepancy exists between the training simulator and real-world dynamics (the sim-to-real gap), the amortized posterior yields biased and poorly calibrated estimates. Third, the framework fails to generalize to parameter ranges outside its training boundaries, even if the underlying physics remain

the same.

We evaluate the robustness of NFs (including FM, RealNVP, and NSF) by subjecting the framework to the following stress-test scenarios:

- Information Extraction Bottlenecks: Increasing observational noise to assess the limits of the condition network's feature extraction.
- Structural Misspecification: Evaluating models trained on linear dynamics against data generated from non-linear physical systems.
- Out-of-Distribution (OOD) Generalization: Testing the framework on parameter ranges that lie outside the support of the training prior.

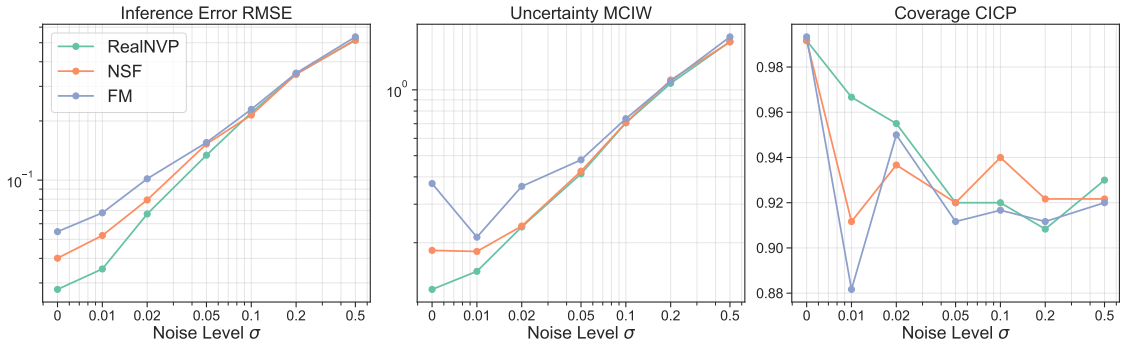
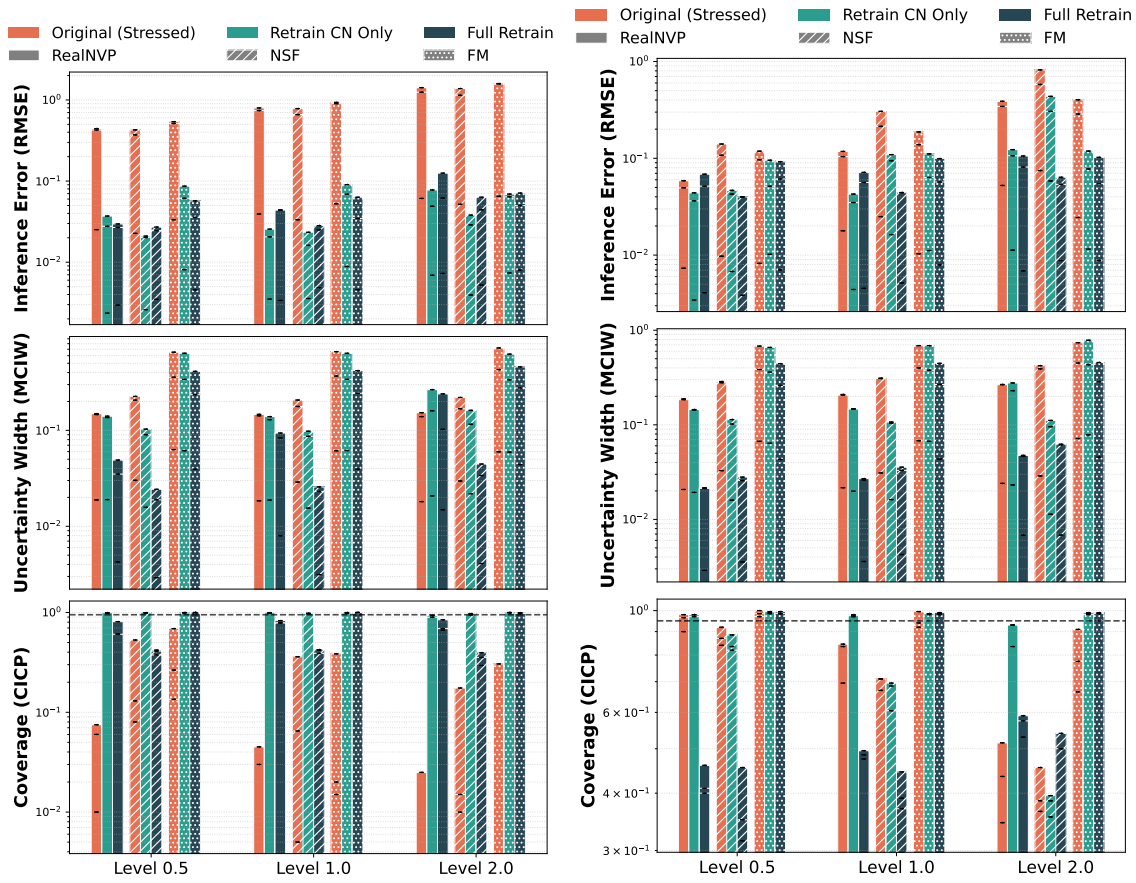


Figure IV.9: Stress test of Normalizing Flow architectures under increasing additive Gaussian noise. The panels illustrate the evolution of predictive accuracy (RMSE), uncertainty quantification (MCIW), and posterior calibration (CICP) for RealNVP, NSF, and FM as the noise intensity σ changes. Logarithmic scaling is applied to both the noise level and the corresponding RMSE and MCIW metrics to highlight performance degradation across several orders of magnitude.

Figure IV.9 illustrates the performance of the three NF architectures under varying levels of additive Gaussian noise σ. At the low-noise level ($\sigma = 0.01$), all models exhibit a sharp decline in coverage probability, particularly the FM and NSF variants. This indicates a calibration failure where the condition network fails to fully distinguish observational stochasticity from parameter variance, leading to overly confident (narrow) posteriors. As noise intensity increases beyond $\sigma = 0.05$, the models show a monotonic degradation in accuracy (RMSE) accompanied by a proportional expansion of uncertainty (MCIW). In the high-noise range ($\sigma \geq 0.05$), the models converge in behavior, successfully broadening their posterior estimates to maintain a stable, albeit lower, coverage floor of approximately 0.92.



(a) Misspecification **(b)** OOD

Figure IV.10: Evaluation of model performance under structural misspecification and OOD stress tests. Rows from top to bottom display inference accuracy (RMSE), uncertainty estimation (MCIW), and coverage probability (CICP). The dashed line in CICP subfigure shows 0.95 coverage probability. Metric values are averaged across 200 test scenarios and three mechanical parameters (m, k, c). The grouped bar plots illustrate performance at varying stress levels $0.5, 1, 2$. For structural misspecification, the level corresponds to the magnitude of the nonlinear factor β introduced into the system dynamics: $\dot{v} = (F_t - kx + \beta x^3 - cv + \beta v|v|)/m$. For OOD testing, the level denotes the shift Δ of the test data range relative to the original training set, where $\Delta = (+0.1\alpha, +\alpha, +0.5\alpha)$. At each level, colors indicate the training state: the original baseline under stress, and the improved results after retraining either the condition network or the entire network. Bar patterns represent the normalizing flow architecture (RealNVP, NSF, and FM).

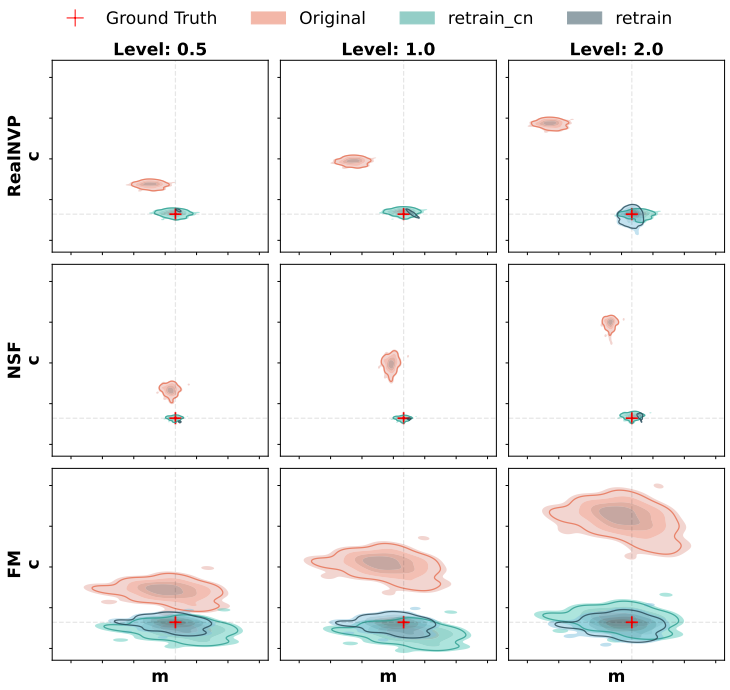


Figure IV.11: Micro View of posterior distributions under misspecification for a single test scenario. The 3×3 grid illustrates the joint posterior distributions of parameters m and c across three architectures (RealNVP, NSF, and FM) and three misspecification levels ($\beta = 0.5, 1, 2$). Colors represent the model state: the baseline under stress (Original), the improved result after retraining the condition network (retrain-cn), and the result after full-network retraining (retrain). The ground truth is marked by a red cross (+).

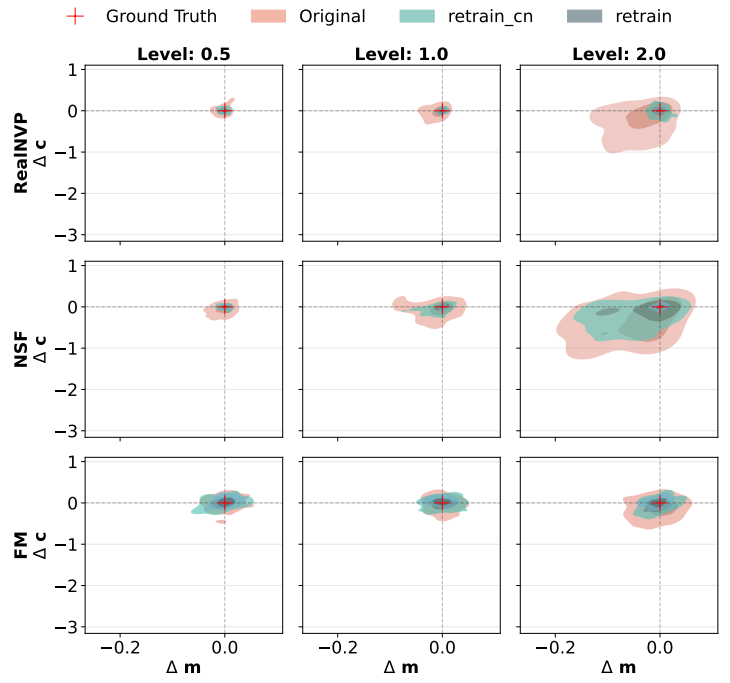


Figure IV.12: Global residual analysis under OOD for 200 test scenarios. The 3×3 grid illustrates a global residual analysis of the joint distributions for mass $\Delta m = \hat{m} - m$ and damping $\Delta c = \hat{c} - c$ across three architectures (RealNVP, NSF, and FM) and three OOD levels ($\alpha = 0.5, 1, 2$). Colors represent the model state: the baseline under stress (Original), the improved result after retraining the condition network (retrain-cn), and the result after full-network retraining (retrain). The perfect match (zero error) is marked by a red cross (+).

In such cases, the condition network encounters a representational bottleneck; it fails to extract sufficient summary statistics, preventing the observational variance from being accurately mapped to the parameter sensitivity. To mitigate this, an Markov Chain Monte Carlo (MCMC) algorithm can be employed to correct the estimates, using the amortized posterior as an informative proposal distribution. Alternatively, a more computationally efficient approach involves Stochastic Normalizing Flow (SNF), which integrates the simulator directly into the latent transformation as a log-posterior diagnostic, as detailed in Section 1.5.

The structural misspecification and OOD generalization problems can be solved by active learning, i.e., retraining the architectures with new data. Figure IV.10 shows the improvement through model adaptation of misspecification and OOD generalization, respectively. In both cases, comparing different retraining methods (different colors), updating only the condition network effectively centers the distribution, significantly reducing RMSE and bringing CICP back toward the 0.95 target. While retraining the whole networks produces the most improvement in terms of minimizing error and tightening confidence intervals, it carries the highest risk of overfitting the posterior. Comparing different flow architectures (different patterns), RealNVP and NSF achieve the lowest RMSE and narrowest MCIW after retraining. However, they are highly sensitive to the misspecification or OOD stress. In contrast, FM shows higher inference error and wider uncertainty than others but maintains higher coverage probability. It is less prone to overfitting and resilient against the stress.

Figure IV.11 is a micro view of the joint distributions between m and c in a test scenario. As the stress level increases, a clear divergence from the ground truth is observed in the baseline models. While FM exhibits a larger posterior variance compared to the other architectures, it demonstrates superior robustness. Retraining the condition network successfully shifts the posteriors back toward the ground truth. However, full-network retraining for RealNVP and NSF leads to overconfidence; while point accuracy increases, the distributions shrink excessively, causing a drop in coverage. In contrast, FM retraining further constrains the distribution without inducing this overfitting effect, maintaining better calibration than the condition-network adaptation.

Figure IV.12 represents a global residual analysis under OOD. By centering the ground truth of each scenario at $(0, 0)$, the subplots reveal the systematic bias and uncertainty induced by increasing OOD shifts. As the shift increases from 0.5 to 2.0, the Original baseline distributions exhibit both a significant expansion in variance and a pronounced negative systematic bias, particularly in the NSF and RealNVP architectures. NSF displays the highest sensitivity to OOD, showing the largest displacement from the origin and the most irregular posterior shapes. In contrast, FM demonstrates superior baseline robustness; even at Level 2.0, its original distribution remains more symmetric and better centered than

the other architectures. Consequently, while retraining the conditional network and the full network significantly reduces error for RealNVP and NSF by recentering the distributions on the ground truth, FM shows a smaller marginal improvement.

1.5 Stochastic Normalizing Flows

Normalizing flows, including flow matching, are deterministic by design, which enables precise and efficient density estimation. However, in practice this deterministic structure can limit their ability to fully capture complex, highly multimodal target distributions, especially when model capacity or data are limited. One way to alleviate this limitation is to combine normalizing flows with MCMC methods. In such hybrid approaches, the normalizing flow serves as an expressive proposal or initialization mechanism, while MCMC refines the samples by enforcing consistency with the true posterior through its accept–reject mechanism or transition kernel. The stochastic exploration of MCMC improves mode coverage and helps the overall system discover and accurately represent multiple modes that might be underrepresented by the deterministic flow alone. Moreover, by repeatedly evaluating the forward dynamics within the MCMC procedure, the method can better discriminate between parameter configurations that produce similar noisy observations, thereby improving parameter identifiability under measurement noise.

1.5.1 Theory of Stochastic Normalizing Flows

Wu et al. (2020) proposed to construct SNFs by alternating a sequence of deterministic Invertible Neural Networks (INNs) and stochastic MCMC sampling layers (INN - MCMC - $\cdots$ - INN - MCMC). Hagemann et al. (2022) provided the theoretical foundations for stochastic normalizing flows and proved their consistency with Conditional Normalizing Flows (CNFs) for solving inverse problems. An SNF is a pair of Markov chains of random variables $((U_0, \ldots, U_T), (V_T, \ldots, V_0))$. The transformation between two adjacent variables is either a deterministic INN block or a stochastic MCMC sampling layer. The process $(U_t)_{t=0}^T$ is denoted as the process from data to latent space, whereas the process $(V_t)_{t=0}^T$ is the converse process from latent to data space. V_T represents the latent variable, which follows a standard normal distribution and U_0 represents the data distribution. Since both processes are Markov chains, the transformations can be written as conditional distributions, where each state depends only on its immediate predecessor. Figure IV.13 illustrates this architecture.

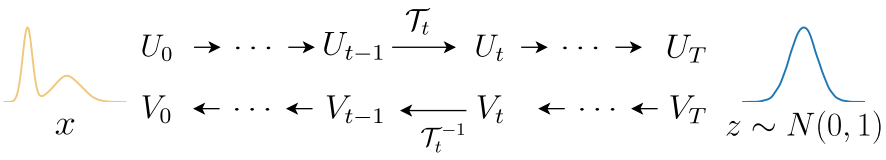


Figure IV.13: The architecture of an SNF with Markov chains $((U_0, \ldots, U_T), (V_T, \ldots, V_0))$. T is the number of stochastic blocks. We denote the transformation in the forward process (from data to latent) by $\mathcal{T}_t = P_{U_t|U_{t-1}}$ and that in the inverse process (from latent to data space) by $\mathcal{T}_t^{-1} = P_{V_{t-1}|V_t}$. To be consistent with NF as shown in Figure II.7, the left side of the figure is the data distribution, while the right side is the latent distribution. Usually, the latent variable follows a standard normal distribution. The upper row illustrates the forward direction from data to latent space, whereas the bottom row shows the inverse direction from latent to data space.

Unlike CNFs, for which the objective function is minimizing the KL-divergence between the learned and target density, the objective function for training an SNF is to minimize the difference of the joint distributions between the forward and inverse processes. We define the joint distributions as:

$$P_{U_{0:T}} = P_{U_0 \sim p_X} \prod_{t=1}^{T} P_{U_t|U_{t-1}} \qquad \text{(forward: data} \to \text{latent)}, \tag{IV.17a}$$

$$P_{V_{0:T}} = P_{V_T \sim p_Z} \prod_{t=1}^{T} P_{V_{t-1}|V_t} \qquad \text{(inverse: latent} \to \text{data)}. \tag{IV.17b}$$

Then, the objective taking the forward process as training variables is:

$$\mathcal{L}_{SNF}(\theta) = D_{\mathrm{KL}}(P_{V_{0:T}} || P_{U_{0:T}}) = \mathbb{E}_{u_{0:T} \sim P_{V_{0:T}}} \log \frac{dP_{V_{0:T}}}{dP_{U_{0:T}}}. \tag{IV.18}$$

Similar to CNFs, the objective function of an SNF is trained from data to the latent space. The derivative term can be expanded as:

$$\frac{dP_{V_{0:T}}}{dP_{U_{0:T}}} = \frac{p_{V_0}(u_0)}{p_{U_0}(u_0)} \prod_{t=1}^{T} \frac{p_{V_t|V_{t-1}}(u_t|u_{t-1})}{p_{U_t|U_{t-1}}(u_t|u_{t-1})} = \frac{p_{V_T}(u_T)}{p_{U_T}(u_T)} \prod_{t=1}^{T} \frac{p_{V_{t-1}|V_t}(u_{t-1}|u_t)}{p_{U_{t-1}|U_t}(u_{t-1}|u_t)}. \tag{IV.19}$$

Because $V_T \equiv Z$ and $U_0 \equiv X$ and the SNF is trained from U_0 to U_T, the term can be rewritten as:

$$\frac{dP_{V_{0:T}}}{dP_{U_{0:T}}} = \frac{p_{V_T}(u_T)}{p_{U_0}(u_0)} \prod_{t=1}^{T} f_t(u_{t-1}, u_t) \frac{p_{V_{t-1}}(u_{t-1})}{p_{V_t}(u_t)}, \quad f_t(u_{t-1}, u_t) = \frac{p_{V_t|V_{t-1}}(u_t|u_{t-1})}{p_{U_t|U_{t-1}}(u_t|u_{t-1})}. \tag{IV.20}$$

Then, the objective is:

$$\mathcal{L}_{SNF}(\theta) = \mathbb{E}_{u_{0:T}\sim P_{V_{0:T}}}\left[\log p_{V_T}(u_T) - \log p_{U_0}(u_0) + \sum_{t=1}^{T} \log f_t(u_{t-1}, u_t)\frac{p_{V_{t-1}}(u_{t-1})}{p_{V_t}(u_t)}\right] \tag{IV.21}$$

The term $f_t(u_{t-1}, u_t)p_{V_{t-1}}(u_{t-1})/p_{V_t}(u_t) \overset{\text{def}}{=} r$ differs for deterministic and stochastic layers. For a **deterministic** transformation, $f_t(u_{t-1}, u_t) = 1$ and $p_{V_{t-1}}(u_{t-1})/p_{V_t}(u_t)$ equals the change of variable:

$$r = 1 \cdot \frac{p_{V_{t-1}}(u_{t-1})}{p_{V_t}(u_t)} = \det\left|J_{\mathcal{T}_t}(u_{t-1})\right|. \tag{IV.22}$$

For a **stochastic** transformation, r measures how well the backward kernel inverts the forward one; it can be reformulated as:

$$r = \frac{p_{V_t|V_{t-1}}(u_t|u_{t-1})}{p_{U_t|U_{t-1}}(u_t|u_{t-1})}\frac{p_{V_{t-1}}(u_{t-1})}{p_{V_t}(u_t)} = \frac{p_{V_{t-1}|V_t}(u_{t-1}|u_t)}{p_{U_t|U_{t-1}}(u_t|u_{t-1})}. \tag{IV.23}$$

The stochastic layer must be invertible and fulfill the detailed balance condition for some density π_t:

$$\pi_t(u_t)p_{V_{t-1}|V_t}(u_{t-1}|u_t) = \pi_t(u_{t-1})p_{U_t|U_{t-1}}(u_t|u_{t-1}). \tag{IV.24}$$

where π_t is the target density at step t, interpolating between p_Z and p_X: $\pi_t = c \cdot p_Z^{t/T} \cdot p_X^{(T-t)/T}$ with c a normalizing constant and $E_t(u) = -\log(\pi_t(u))$. Rearranging Equation IV.24:

$$r = \frac{p_{V_{t-1}|V_t}(u_{t-1}|u_t)}{p_{U_t|U_{t-1}}(u_t|u_{t-1})} = \frac{\pi_t(u_{t-1})}{\pi_t(u_t)}. \tag{IV.25}$$

For MCMC this is computed by evaluating π_t; for efficiency an Metropolis-adjusted Langevin algorithm (MALA) kernel (Section 2.2.2) yields:

$$r = \frac{\pi_t(u_{t-1})}{\pi_t(u_t)} = \exp\left(\frac{1}{2}\left(\|\tilde{\eta}_t\|_2^2 - \|\eta_t\|_2^2\right)\right), \tag{IV.26}$$

where the Gaussian noise vectors are:

$$\eta_t := \frac{1}{\sqrt{\tau}}\left(u_{t-1} - u_t - \frac{\tau}{2}\nabla E_t(u_{t-1})\right) \qquad \text{(forward: } u_{t-1} \to u_t\text{)}, \tag{IV.27}$$

$$\tilde{\eta}_t := \frac{1}{\sqrt{\tau}}\left(u_{t-1} - u_t + \frac{\tau}{2}\nabla E_t(u_t)\right) \qquad \text{(backward: } u_t \to u_{t-1}\text{)}. \tag{IV.28}$$

1.5.2 Experiments on Stochastic Normalizing Flows

We evaluate SNFs on two fronts: multimodal posterior modeling and parameter identification under noisy observations. The following experiments demonstrate the practical benefits of combining normalizing flows with stochastic MCMC layers.

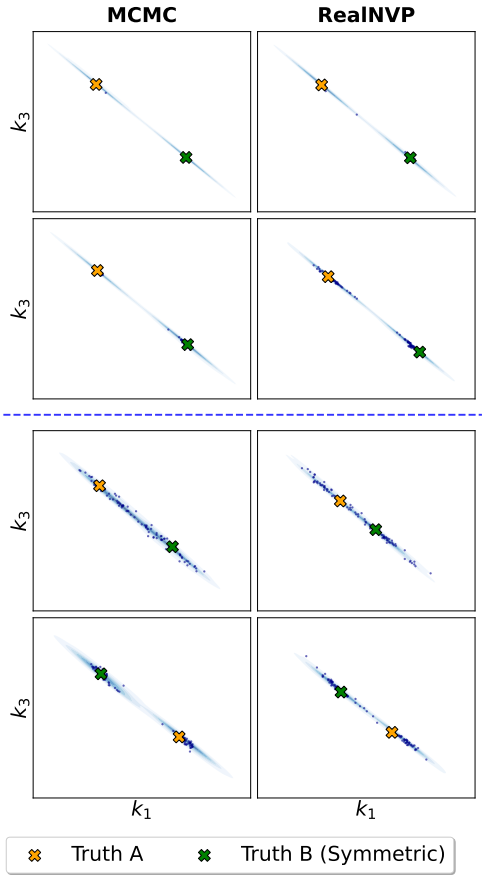


Figure IV.14: Comparison of MCMC-RealNVP (left) and RealNVP (right) for multimodal modeling. Experimental settings match Fig. IV.5.

Table IV.3: Comparison of Posterior Estimation Metrics. For RMSE/MCIW, lower is better (↓); for CICP, ≈ 0.95 is ideal. Metrics are mode-aligned with centers identified via K-means clustering. The results except the MCMC are corresponding to Figure IV.5. The MCMC stands for an SNF with MCMC and RealNVP.

Model	RMSE (↓)	MCIW (↓)	CICP
BNN	1.611	4.581	0.843
VAE	1.375	1.527	0.460
DDPM	1.415	2.211	0.560
SBM	0.540	0.838	0.553
FM	0.300	0.830	0.873
RealNVP	0.258	1.067	0.925
NSF	0.108	**0.412**	0.928
MCMC	**0.105**	0.508	**0.970**

To enhance multimodal modeling, we implement a hybrid SNF architecture combining RealNVP with four intermediate MCMC refinement steps (step size is 0.001). In the forward (training) pass, the MCMC updates are applied to the latent encoding to improve likelihood estimation; in the inverse (generative) pass, they are applied to the output samples to ensure better alignment with the target data distribution. This stochastic refinement enables the flow to capture disconnected modes, which are challenging for RealNVP as a simpler deterministic NF. Table IV.3 demonstrates that this approach yields the best-in-class RMSE and a near-ideal coverage probability, outperforming the NSF (top performer in our study) in terms of both precision and uncertainty quantification.

In order to improve the parameter identifiability under noisy observations, we apply a

MALA-refinement after sampling through NF architecture. We define the log-likelihood by measuring the residual between the actual observations and the signals generated by the model's governing dynamics using the inferred parameters. For the sake of handling different parameter sensitivity, the Riemannian Manifold MALA (RMALA) refinement leverages the Fisher Information Matrix (or Hessian) to adapt to the local curvature of the log-posterior. While standard MALA uses a constant step size in all directions, RMALA uses a position-dependent metric tensor. This preconditioning allows the flow to take larger steps in directions of low sensitivity and smaller, more precise steps in directions of high sensitivity.

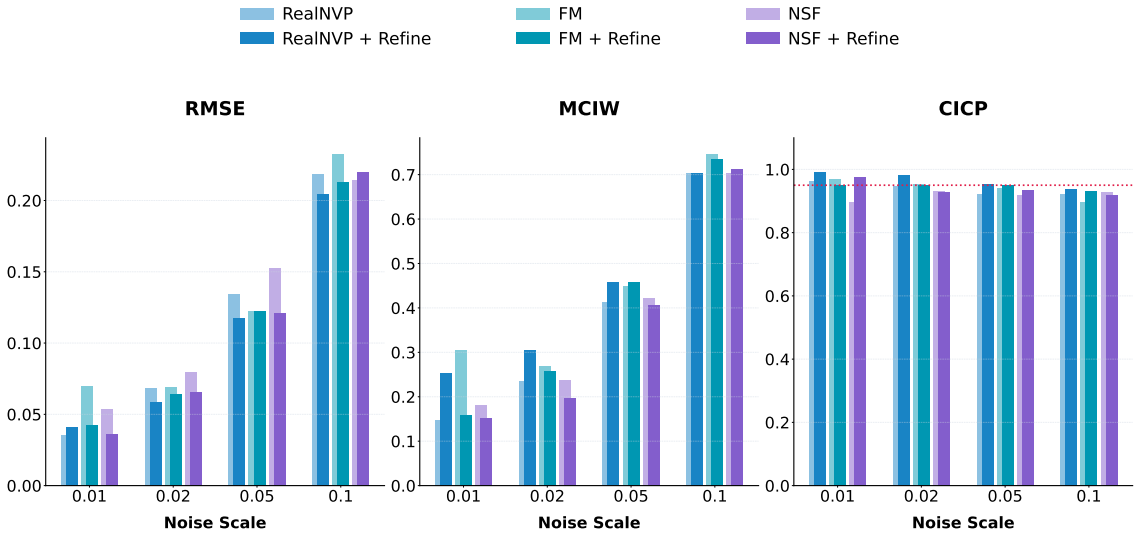


Figure IV.15: Improved Parameter Identification under Noisy Observations using SNFs. Three NF architectures—RealNVP, FM, and NSF—are evaluated across varying noise scales (0.01, 0.02, 0.05, and 0.1). Performance is measured in terms of accuracy (RMSE) and uncertainty quantification (MCIW and CICP). Grouped bars contrast the original baseline results with those obtained via RMALA-based refinement. In the CICP subfigure, the dashed red line indicates the ideal coverage probability of 0.95.

Figure IV.15 shows the comparison of parameter identification under noisy observations between NF and SNF. The RMSE panel reveals a clear trend where error increases proportionally with observational noise. However, the stochastic refinement consistently outperforms the deterministic base models across all noise scales. At lower noise levels (0.01 and 0.02), the refined models often exhibit a higher MCIW than the base models. It indicates that the base models were producing intervals that were too narrow (overconfident). The MALA layers broaden the distribution to accurately reflect the true posterior width. At a noise scale of 0.1, the MCIW values converge. This suggests that in high-noise regimes,

the scale of uncertainty is largely driven by the data likelihood rather than the specific choice of flow architecture. The uncertainty calibration can be proven in the CICP panel as well. Note that at noise scale 0.01, the base NSF model (light purple) significantly under-0.95-covers. The refinement step (dark purple) lifts this probability back to the target. While deterministic models start to lose coverage as noise increases (falling below 0.90), the refined models maintain a much more stable coverage probability, remaining close to the target across the entire noise spectrum. The consistent improvement in CICP implies that the stochastic layers are pushing the approximate posterior much closer to the true posterior.

Surprisingly, the FM reaches exact 0.95 CICP at noise scales (0.01, 0.02 and 0.05). Because FM learns a continuous vector field rather than a series of discrete couplings, it likely provides a smoother initial density for the stochastic layers. When RMALA is applied to this smooth surface, the gradient-based steps can move the samples to the exact posterior boundaries with higher precision than in more rigid architectures like RealNVP and NSF. This makes the RMALA-refined FM an exceptionally reliable tool for parameter identification in complex, noisy dynamical systems.

2 State Estimation

Adapting normalizing flow methods from parameter identification to state estimation introduces several non-trivial challenges. Unlike parameter identification, which targets a fixed set of unknowns, state estimation requires recovering a full sequential trajectory from partial observations — that is, computing the posterior $p(x_{1:T} \mid y_{1:T})$ over a potentially multimodal distribution. Standard filtering approaches such as the Kalman filter assume known, linear-Gaussian dynamics; the Bayesian setting considered here imposes no such restrictions: both the state dynamics and the observation model may be nonlinear and stochastic, and the resulting posterior is generally multimodal. A naive approach — applying the parameter identification method independently at each time step — fails to capture the temporal dependencies coupling successive states, and permits errors to accumulate unchecked across time.

Classically, this is addressed by sequential Monte Carlo methods such as particle filters (Nemeth et al., 2014), which approximate the posterior by propagating a set of weighted particles through time. However, these methods are computationally expensive and suffer from degeneracy issues in high-dimensional state spaces. Variational approaches such as VAEs and Recurrent Neural Networks (RNNs) have been proposed to learn amortized inference networks for state estimation 2, but they often struggle to capture complex temporal dependencies and multimodal posteriors effectively. To address these limitations, this section introduces four methods for state estimation with normalizing flows, ordered by increasing generalisability: Amortized State Flow, Latent State Space Model Flow, NICE Flow State Space Model, and Conditional Stochastic Transpolant. For the first two methods, we employ two normalizing flow architectures: RealNVP and Flow Matching.

All four methods are evaluated on the Lorenz-96 system (Lorenz, 1996), a canonical benchmark for nonlinear data assimilation. The state has dimension $N = 12$, and at each time step only $N/2 = 6$ components are observed, subject to additive Gaussian noise of varying magnitude. The governing equations are

$$\frac{dx_i}{dt} = (x_{i+1} - x_{i-2})\, x_{i-1} - x_i + F, \qquad i = 1, \ldots, N, \tag{IV.29}$$

with periodic boundary conditions $x_{-1} = x_{N-1}$, $x_0 = x_N$, $x_{N+1} = x_1$, and forcing constant $F = 8$, a regime in which the system exhibits sustained chaotic behaviour.

2.1 Amortized State Flow

The most conceptually straightforward approach is to amortize inference across time by applying a normalizing flow independently to each observation segment, thereby reducing the sequential estimation problem to a collection of independent inference tasks. The

primary advantage is that all parameter identification methods introduced in the preceding section transfer directly with minimal modification. To endow the latent variables with

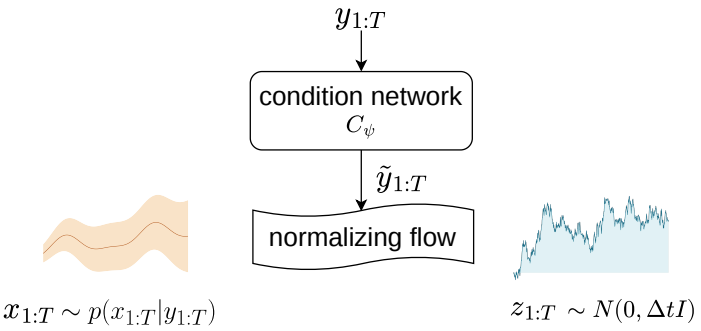


Figure IV.16: Architecture of amortized state estimation with normalizing flows.

temporal structure, we adopt a Wiener process as the base distribution of the normalizing flow. This choice assigns monotonically increasing variance to successive latent variables while preserving the Gaussian character of the distribution, providing a principled inductive bias toward smooth trajectories. The architecture is illustrated in Figure IV.16.

This simplicity, however, comes at a cost. Because each segment is processed as an independent collection of observations, the approach discards the temporal structure of the state trajectory entirely. Long-range dependencies between states are not modelled, and estimation errors can accumulate across segments without any corrective mechanism. Amortized state estimation is therefore most appropriate when the underlying dynamics are relatively smooth and temporal correlations are weak; it is ill-suited to settings with highly nonlinear dynamics or substantial observation noise.

2.2 Latent State Space Model Flow

To overcome the fundamental limitation of treating time steps independently, we impose a linear Gaussian State Space Model directly in the latent space of the normalizing flow, yielding the architecture we term LatentSSMFlow (Figure IV.17). At each time step t, the latent variable z_t evolves from the preceding latent state z_{t-1} via a linear Gaussian transition:

$$z_t = F z_{t-1} + G, \qquad G \sim \mathcal{N}(0,\, \Sigma). \tag{IV.30}$$

The transition matrix F is fixed prior to training to ensure that the latent dynamics remain in a contractive or non-expansive regime, enforced by bounding the spectral radius:

$$\rho(F) = \max_i |\lambda_i| \leq \rho_{max} < 1,$$

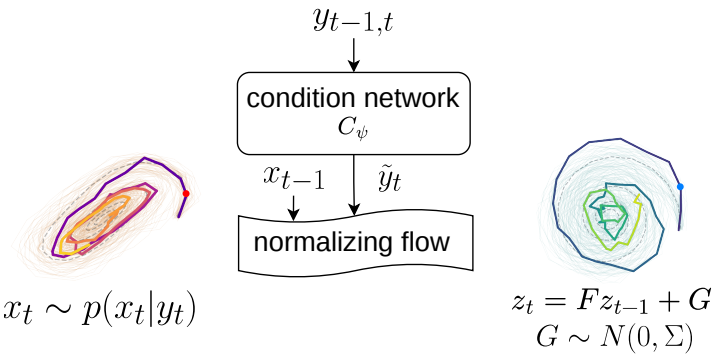


Figure IV.17: Architecture of Latent State Space Model Flow.

where λ_i denote the eigenvalues of F. The process noise covariance Σ is likewise held fixed throughout training to promote stability.

The normalizing flow acts as a learned generative filter: at each step it maps the latent variable z_t to the state x_t, conditioned on the observations $y_{t-1:t}$ and the preceding state x_{t-1}. Observations include the current and the previous step and are first encoded by a conditioning network C_ψ before being passed to the flow. To improve both training efficiency and accuracy over long time horizons, we employ a multiple-shooting strategy, segmenting each trajectory into shorter overlapping intervals and unrolling the flow autoregressively within each segment.

Note that the autoregressive process works differently for RealNVP and Flow Matching. For training RealNVP, each state is transformed to the latent space because of its invertibility:

$$z_t = f_\phi^{-1}(x_t; c_t, x_{t-1}), \quad c_t = C_\psi(y_{t-1:t}). \tag{IV.31}$$

Then, the model is trained via Maximum Likelihood Estimation (MLE). Using the Change of Variables formula, the log-likelihood of the state sequence is:

$$\log p(x_{1:T}|y_{1:T}) = \log p_{SSM}(z_{1:T}) + \sum_{t=1}^{T} \log \left| \det \frac{\partial f_\phi(x_t; c_t, x_{t-1})}{\partial x_t} \right|, \tag{IV.32}$$

where $p_{SSM}(z_{1:T})$ is the likelihood of the latent trajectory under the linear Gaussian SSM. While the dynamics of the latent variable is modeled by the linear Gaussian SSM, the state transition is also learned by the RealNVP.

For Flow Matching, because the architecture is not invertible, the state during training is not updated autoregressively. Instead, only one step transition through conditioning is trained at a time. In the inference process, however, the state is updated autoregressively.

This mismatch between training and inference leads to a significant performance gap.

2.3 NICE Flow State Space Model

To further enhance the dynamics modeling capability, we propose to estimate the state trajectory by modeling the underlying dynamics as a stochastic differential equation. We take the variable $x_t|y_t := z_t$ as the new state of the system. When the state is assumed to evolve continuously in time, the dynamics can be described by a Stochastic Differential Equation (SDE):

$$dz_t = f(z_t, t)dt + g(z_t, t)dW_t, \quad dW_t = \sqrt{dt}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \tag{IV.33}$$

By modeling f and g with neural networks, we can capture complex nonlinear dynamics and state-dependent noise. A naive approach to training such a NeuralSDE by maximizing the likelihood of the observed data causes collapse of the latent state, as the networks can simply learn to ignore the diffusion and fit the drift dynamics only.

One possible solution to enhance the expressiveness of the diffusion term is to apply variational inference, namely LatentSDE (Li et al., 2020b). LatentSDE adopts a prior NeuralSDE and a posterior NeuralSDE with the same paramtereization of the diffusion function and with different drift functions. By sharing the diffusion term, the KL divergence between the two stochastic processes becomes analytically tractable, which allows the model to be trained by maximizing the Evidence Lower Bound (ELBO). Another approach treats NeuralSDE as infinite-dimensional Generative Adversarial Network (GAN) (Kidger et al., 2021). In this framework, a NeuralSDE acts as a generator that transforms Brownian motion into sample paths, while a Neural Controlled Differential Equation (NeuralCDE) serves as a discriminator that operates directly on the continuous-time trajectories. By optimizing a Wasserstein-type objective, this method allows for more flexible noise structures that are otherwise difficult to train using variational methods, effectively matching the entire distribution of generated paths to the true data distribution in path space.

However, both methods require significant computational resources and careful tuning through extensive experimentation. To mitigate this issue, we introduce to leverage normalizing flow directly in the NeuralSDE framework, avoiding to use another differential equation for supporting training of the NeuralSDE. In order to keep the computation efficient, we use a simple normalizing flow architecture, NICE 4.2, to model the diffusion term. Then, the SDE becomes a NICE enhanced state space model:

$$dz_t = f_\theta(z_t, t)dt + g_\phi(dW_t; z_t, t)\sqrt{dt}. \tag{IV.34}$$

The drift function f_θ is a neural network that models the deterministic part of the dynamics.

The NICE architecture g_ϕ takes the Brownian motion increment dW_t as its latent variable, and outputs a state-dependent noise term. Unlike armortized Bayesian inference, the NICE flow is not trained by minimizing the KL divergence, because its ground truth is not available. Instead, NICE flow is sampled from the latent distribution while training. The NICEFlowSSM is trained by maximizing the overall likelihood of the observed data, which requires the SSM to be integrated over time.

The NICE flow captures complex dependencies between the noise and the state, enabling the model to learn rich stochastic dynamics. Another advantage is its simple Jacobian determinant II.57. When the coupling layer is additive, the Jacobian determinant is always 1, which significantly simplifies the likelihood computation and allows for efficient training. Moreover, .

The integration of the SSM is performed by a numerical SDE solver, such as the Euler-Maruyma method, Heun, Midpoint, Milstein, or reversible Heun method. The solvers are supposed to be adjusted according to Equation IV.34. The initial state is given or randomly initialized. So far, in comparison to the previous two methods, the NICEFlowSSM is the most powerful and general approach, which can capture complex, nonlinear, and long-term dynamics, as well as state-dependent noise. However, it is also the most computationally expensive and challenging to train, as it requires integrating the SDE over time and optimizing the neural networks simultaneously.

2.4 Stochastic Interpolant

3 Optimal Control

Model free control with IMFlow

Chapter V

Experiments and Results

1 Overview

2 Parameter Identification

von der pol single track car quadcopter

3 State Estimation

4 Optimal Control

Chapter VI

Conclusion and Future Work

Abstract

Zusammenfassung

Own Contributions

This chapter gives an overview of my contributions in distinction to team efforts.

Own share in data acquisition and data analysis

Own share in Data Acquisition.

Own share in Data Analysis.

Own Publications

In this section, all publications that I was a part of and contributed to during my Ph.D. work are listed. It is subdivided into *First Authorships* and *Co-Authorships*.

First Authorships - Peer Reviewed International Journal Publications

Y. Zhang, L. Mikelsons. **Advanced Modeling and Simulation in Engineering and Science**. *Sensitivity-guided iterative parameter identification and data generation with BayesFlow and PELS-VAE for model calibration.* doi:10.1186/s40323-023-00246-y.

First Authorships - Peer Reviewed International Conference Publications

Y. Zhang, L. Mikelsons. **2023 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)**. *Solving Stochastic Inverse Problems with Stochastic BayesFlow.* doi:10.1109/AIM46323.2023.10196190.

Y. Zhang, L. Mikelsons. **2024 IEEE 63rd Conference on Decision and Control(CDC)**. *Inverse Modeling with Conditional Normalizing Flows for Data-Driven Model Predictive Control*. doi:10.1109/CDC56724.2024.10886862.

Co-Authorships

M.E. Heringhaus, Y. Zhang, A. Zimmermann, L. Mikelsons. **Sensors 2022**. *Towards Reliable Parameter Extraction in MEMS Final Module Testing Using Bayesian Inference*. doi:10.3390/s22145408.

Bibliography

Kweku Abraham and Neil Deo. Deep gaussian process priors for bayesian inference in nonlinear inverse problems, 2023. URL https://arxiv.org/abs/2312.14294.

Ryan P. Adams, Jeffrey Pennington, Matthew J. Johnson, Jamie Smith, Yaniv Ovadia, Brian Patton, and James Saunderson. Estimating the spectral density of large implicit matrices, 2018. URL https://arxiv.org/abs/1802.03451.

Shubhankar Agarwal, Harshit Sikchi, Cole Gulino, Eric Wilkinson, and Shivam Gautam. Imitative planning using conditional normalizing flow, 2022. URL https://arxiv.org/abs/2007.16162.

Ahmed M. Alaa and Mihaela van der Schaar. Deep multi-task gaussian processes for survival analysis with competing risks. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 2326–2334, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Edward Allen. *Modeling with itô stochastic differential equations*, volume 22. Springer, 2007.

Marvin Alles, Nutan Chen, Patrick van der Smagt, and Botond Cseke. Flowq: Energy-guided flow policies for offline reinforcement learning, 2025. URL https://arxiv.org/abs/2505.14139.

Govinda Anantha Padmanabha and Nicholas Zabaras. Solving inverse problems using conditional invertible neural networks. *Journal of Computational Physics*, 433:110194, 2021. ISSN 0021-9991. doi: https://doi.org/10.1016/j.jcp.2021.110194. URL https://www.sciencedirect.com/science/article/pii/S0021999121000899.

Lynton Ardizzone, Jakob Kruse, Sebastian Wirkert, Daniel Rahner, Eric W. Pellegrini, Ralf S. Klessen, Lena Maier-Hein, Carsten Rother, and Ullrich Köthe. Analyzing inverse

problems with invertible neural networks, 2019a. URL https://arxiv.org/abs/1808.04730.

Lynton Ardizzone, Carsten Lüth, Jakob Kruse, Carsten Rother, and Ullrich Köthe. Guided image generation with conditional invertible neural networks, 2019b. URL https://arxiv.org/abs/1907.02392.

Andrea Asperti. Variance loss in variational autoencoders, 2020. URL https://arxiv.org/abs/2002.09860.

Lorenzo Baldassari, Ali Siahkoohi, Josselin Garnier, Knut Solna, and Maarten V. de Hoop. Conditional score-based diffusion models for bayesian inference in infinite dimensions, 2023. URL https://arxiv.org/abs/2305.19147.

A.L. Barker, D.E. Brown, and W.N. Martin. Bayesian estimation and the kalman filter. *Computers & Mathematics with Applications*, 30(10):55–77, 1995. ISSN 0898-1221. doi: https://doi.org/10.1016/0898-1221(95)00156-S. URL https://www.sciencedirect.com/science/article/pii/089812219500156S.

Jens Behrmann, Will Grathwohl, Ricky T. Q. Chen, David Duvenaud, and Jörn-Henrik Jacobsen. Invertible residual networks, 2019. URL https://arxiv.org/abs/1811.00995.

Naoufal El Bekri, Lucas Drumetz, and Franck Vermet. Time changed normalizing flows for accurate sde modeling. In *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, page 6395–6399. IEEE, April 2024. doi: 10.1109/icassp48485.2024.10446131.

Julian Besag. Markov chain monte carlo methods for statistical inference. *Department of Statistics University of Washington, Seattle, USA*, 2004.

Sahil Bhola and Karthik Duraisamy. Estimating global identifiability using conditional mutual information in a bayesian framework. *Scientific Reports*, 13(1):18336, October 2023.

Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2006. ISBN 0387310738.

L. Bittner. L. s. pontryagin, v. g. boltyanskii, r. v. gamkrelidze, e. f. mishechenko, the mathematical theory of optimal processes. viii + 360 s. new york/london 1962. john wiley & sons. preis 90/–. *ZAMM - Journal of Applied Mathematics and Mechanics / Zeitschrift für Angewandte Mathematik und Mechanik*, 43(10–11):514–515, January 1963.

ISSN 1521-4001. doi: 10.1002/zamm.19630431023. URL http://dx.doi.org/10.1002/zamm.19630431023.

David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, April 2017. ISSN 1537-274X. doi: 10.1080/01621459.2017.1285773. URL http://dx.doi.org/10.1080/01621459.2017.1285773.

Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural networks, 2015. URL https://arxiv.org/abs/1505.05424.

Damian Boborzi, Christoph-Nikolas Straehle, Jens S. Buchner, and Lars Mikelsons. Learning normalizing flow policies based on highway demonstrations. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pages 22–29, 2021. doi: 10.1109/ITSC48978.2021.9564456.

Léon Bottou. *On-line learning and stochastic approximations*, page 9–42. Cambridge University Press, USA, 1999. ISBN 0521652634.

Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. In Stefan Riezler and Yoav Goldberg, editors, *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 10–21, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/K16-1002. URL https://aclanthology.org/K16-1002/.

Janaka Brahmanage, Jiajing LING, and Akshat Kumar. Flowpg: Action-constrained policy gradient with normalizing flows. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 20118–20132. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/3fd9fe8ec6d7238bf71784797399bb61-Paper-Conference.pdf.

Alessandro Brusaferri, Matteo Matteucci, Pietro Portolani, and Stefano Spinelli. Nonlinear system identification using a recurrent network in a bayesian framework. In *2019 IEEE 17th International Conference on Industrial Informatics (INDIN)*, volume 1, pages 319–324, 2019. doi: 10.1109/INDIN41052.2019.8972113.

Lucian Buşoniu, Tim de Bruin, Domagoj Tolić, Jens Kober, and Ivana Palunko. Reinforcement learning for control: Performance, stability, and deep approximators. *Annual Reviews in Control*, 46:8–28, 2018. ISSN 1367-5788. doi: https://doi.org/10.1016/j.arco

ntrol.2018.09.005. URL https://www.sciencedirect.com/science/article/pii/S1367578818301184.

Giuseppe C. Calafiore and Lorenzo Fagiano. Robust model predictive control via scenario optimization. *IEEE Transactions on Automatic Control*, 58(1):219–224, January 2013. ISSN 1558-2523. doi: 10.1109/tac.2012.2203054. URL http://dx.doi.org/10.1109/TAC.2012.2203054.

Peter J. Campo and Manfred Morari. Robust model predictive control. In *1987 American Control Conference*, pages 1021–1026, 1987. doi: 10.23919/ACC.1987.4789462.

Wei-Di Chang, Juan Camilo Gamboa Higuera, Scott Fujimoto, David Meger, and Gregory Dudek. Il-flow: Imitation learning from observation using normalizing flows, 2022. URL https://arxiv.org/abs/2205.09251.

Chen-Hao Chao, Chien Feng, Wei-Fang Sun, Cheng-Kuang Lee, Simon See, and Chun-Yi Lee. Maximum entropy reinforcement learning via energy-based normalizing flow. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 56136–56165. Curran Associates, Inc., 2024. doi: 10.52202/079017-1786. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/661c37f3b098bdee53fd7d9c4ef6964a-Paper-Conference.pdf.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations, 2019. URL https://arxiv.org/abs/1806.07366.

Ricky T. Q. Chen, Jens Behrmann, David Duvenaud, and Jörn-Henrik Jacobsen. Residual flows for invertible generative modeling, 2020. URL https://arxiv.org/abs/1906.02735.

Tianyu Chen, Vansh Bansal, and James G. Scott. Conditional diffusions for amortized neural posterior estimation, 2025. URL https://arxiv.org/abs/2410.19105.

Xiongjie Chen, Hao Wen, and Yunpeng Li. Differentiable particle filters through conditional normalizing flow, 2021. URL https://arxiv.org/abs/2107.00488.

Zhe Chen. Bayesian Filtering: From Kalman Filters to Particle Filters, and Beyond. *Statistics*, 182, January 2003. doi: 10/bkztpw. 01234 ZSCC: 0001234.

Minjae Cho, Jonathan P. How, and Chuangchuang Sun. Out-of-distribution adaptation in offline rl: Counterfactual reasoning via causal normalizing flows, 2024. URL https://arxiv.org/abs/2405.03892.

Frank J. Christophersen. *Receding Horizon Control*, pages 26–38. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. ISBN 978-3-540-72701-9. doi: 10.1007/978-3-540-72701-9_3. URL https://doi.org/10.1007/978-3-540-72701-9_3.

Earl A Coddington and Norman Levinson. *Theory of ordinary differential equations*. McGraw-Hill New York, 1955.

Andreas C. Damianou and Neil D. Lawrence. Deep gaussian processes, 2013. URL https://arxiv.org/abs/1211.0358.

Federico De Santi, Massimiliano Razzano, Francesco Fidecaro, Luca Muccillo, Lucia Papalini, and Barbara Patricelli. Deep learning to detect gravitational waves from binary close encounters: Fast parameter estimation using normalizing flows. *Physical Review D*, 109(10), May 2024. ISSN 2470-0029. doi: 10.1103/physrevd.109.102004. URL http://dx.doi.org/10.1103/PhysRevD.109.102004.

A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society. Series B (Methodological)*, 39(1):1–38, 2025/10/08/ 1977. ISSN 00359246. URL http://www.jstor.org/stable/2984875. Full publication date: 1977.

Ruizhi Deng, Marcus A. Brubaker, Greg Mori, and Andreas M. Lehrmann. Continuous latent process flows, 2021a.

Ruizhi Deng, Bo Chang, Marcus A. Brubaker, Greg Mori, and Andreas Lehrmann. Modeling continuous stochastic processes with dynamic normalizing flows, 2021b.

Aadesh Desai, Eshan Gujarathi, Saagar Parikh, Sachin Yadav, Zeel Patel, and Nipun Batra. Deep gaussian processes for air quality inference. In *Proceedings of the 6th Joint International Conference on Data Science & Management of Data (10th ACM IKDD CODS and 28th COMAD)*, CODS-COMAD '23, page 278–279, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450397971. doi: 10.1145/3570991.3571004. URL https://doi.org/10.1145/3570991.3571004.

Moritz Diehl. *Optimization Algorithms for Model Predictive Control*, pages 1–11. Springer London, London, 2013. ISBN 978-1-4471-5102-9. doi: 10.1007/978-1-4471-5102-9_9-1. URL https://doi.org/10.1007/978-1-4471-5102-9_9-1.

Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation, 2015. URL https://arxiv.org/abs/1410.8516.

Jiayuan Dong, Christian Jacobsen, Mehdi Khalloufi, Maryam Akram, Wanjiao Liu, Karthik Duraisamy, and Xun Huan. Variational bayesian optimal experimental design with normalizing flows. *Computer Methods in Applied Mechanics and Engineering*, 433: 117457, 2025. ISSN 0045-7825. doi: https://doi.org/10.1016/j.cma.2024.117457. URL https://www.sciencedirect.com/science/article/pii/S0045782524007126.

Shian Du, Yihong Luo, Wei Chen, Jian Xu, and Delu Zeng. To-flow: Efficient continuous normalizing flows with temporal optimization adjoint with moving speed, 2022. URL https://arxiv.org/abs/2203.10335.

Simon Duane, A.D. Kennedy, Brian J. Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics Letters B*, 195(2):216–222, 1987. ISSN 0370-2693. doi: https://doi.org/10.1016/0370-2693(87)91197-X. URL https://www.sciencedirect.com/science/article/pii/037026938791197X.

J. Durbin and S. J. Koopman. Time series analysis of non-gaussian observations based on state space models from both classical and bayesian perspectives. *Journal of the Royal Statistical Society. Series B (Statistical Methodology)*, 62(1):3–56, 2025/10/09/ 2000. URL http://www.jstor.org/stable/2680676. Full publication date: 2000.

J. Durbin and Siem Jan Koopman. *Time series analysis by state space methods*. Oxford statistical science series; 38. Oxford University Press, Oxford, 2nd ed edition, 2012. ISBN 9780199641178; 019964117X.

Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural spline flows, 2019. URL https://arxiv.org/abs/1906.04032.

Richard Durrett. *Probability: theory and examples*. Duxbury Press, Belmont, CA, second edition, 1996. ISBN 0-534-24318-5.

Jos Elfring, Elena Torta, and René van de Molengraft. Particle filters: A Hands-On tutorial. *Sensors (Basel)*, 21(2), jan 2021.

Márcio Ferreira and Michał Bejger. Conditional variational autoencoder inference of neutron star equation of state from astrophysical observations. *Physical Review D*, 111(2), January 2025. ISSN 2470-0029. doi: 10.1103/physrevd.111.023035. URL http://dx.doi.org/10.1103/PhysRevD.111.023035.

Michel Fliess and Cédric Join. Model-free control. *International Journal of Control*, 86 (12):2228–2252, December 2013. ISSN 1366-5820. doi: 10.1080/00207179.2013.810345. URL http://dx.doi.org/10.1080/00207179.2013.810345.

Andrew Y. K. Foong, Yingzhen Li, José Miguel Hernández-Lobato, and Richard E. Turner. 'in-between' uncertainty in bayesian neural networks, 2019. URL https://arxiv.org/abs/1906.11537.

Andrew Y. K. Foong, David R. Burt, Yingzhen Li, and Richard E. Turner. On the expressiveness of approximate inference in bayesian neural networks, 2020. URL https://arxiv.org/abs/1909.00719.

Meire Fortunato, Charles Blundell, and Oriol Vinyals. Bayesian recurrent neural networks, 2019. URL https://arxiv.org/abs/1704.02798.

Marco Fraccaro, Simon Kamronn, Ulrich Paquet, and Ole Winther. A disentangled recognition and nonlinear dynamics model for unsupervised learning, 2017. URL https://arxiv.org/abs/1710.05741.

Gideon Joseph Freund, Elad Sarafian, and Sarit Kraus. A coupled flow approach to imitation learning. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10357–10372. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/freund23a.html.

Hunter Gabbard, Chris Messenger, Ik Siong Heng, Francesco Tonolini, and Roderick Murray-Smith. Bayesian parameter estimation using conditional variational autoencoders for gravitational-wave astronomy. *Nature Physics*, 18(1):112–117, Jan 2022. ISSN 1745-2481. doi: 10.1038/s41567-021-01425-7. URL https://doi.org/10.1038/s41567-021-01425-7.

F. Gabbiani and S.J. Cox. *Mathematics for Neuroscientists*. Elsevier science & technology books. Elsevier Science, 2010.

Carlos E. García, David M. Prett, and Manfred Morari. Model predictive control: Theory and practice—a survey. *Automatica*, 25(3):335–348, 1989. ISSN 0005-1098. doi: https://doi.org/10.1016/0005-1098(89)90002-2. URL https://www.sciencedirect.com/science/article/pii/0005109889900022.

M. Gavrilović, R. Petrović, and D. Šiljak. Adjoint method in the sensitivity analysis of optimal systems. *Journal of the Franklin Institute*, 276(1):26–38, 1963. ISSN 0016-0032. doi: https://doi.org/10.1016/0016-0032(63)90307-7. URL https://www.sciencedirect.com/science/article/pii/0016003263903077.

Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Akti Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. Chapman & Hall/CRC Texts in Statistical Science Series. CRC, Boca Raton, Florida, third edition, 2013. ISBN 9781439840955 1439840954. URL https://stat.columbia.edu/~gelman/book/.

Samuel Gershman and Noah Goodman. Amortized inference in probabilistic reasoning. In *Proceedings of the annual meeting of the cognitive science society*, volume 36, 2014.

Raj Ghugare and Benjamin Eysenbach. Normalizing flows are capable models for rl, 2025. URL https://arxiv.org/abs/2505.23527.

Laurent Girin, Simon Leglaive, Xiaoyu Bie, Julien Diard, Thomas Hueber, and Xavier Alameda-Pineda. Dynamical variational autoencoders: A comprehensive review. *Foundations and Trends® in Machine Learning*, 15(1–2):1–175, 2021. ISSN 1935-8245. doi: 10.1561/2200000089. URL http://dx.doi.org/10.1561/2200000089.

Ethan Goan and Clinton Fookes. *Bayesian Neural Networks: An Introduction and Survey*, page 45–87. Springer International Publishing, 2020. ISBN 9783030425531. doi: 10.1007/978-3-030-42553-1_3. URL http://dx.doi.org/10.1007/978-3-030-42553-1_3.

N.J. Gordon, D.J. Salmond, and A.F.M. Smith. Novel approach to nonlinear/non-gaussian bayesian state estimation. *IEE Proceedings F (Radar and Signal Processing)*, 140:107–113, 1993. doi: 10.1049/ip-f-2.1993.0015. URL https://digital-library.theiet.org/doi/abs/10.1049/ip-f-2.1993.0015.

Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models, 2018. URL https://arxiv.org/abs/1810.01367.

Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse Problems*, 34(1):014004, December 2017. ISSN 1361-6420. doi: 10.1088/1361-6420/aa9a90. URL http://dx.doi.org/10.1088/1361-6420/aa9a90.

Paul Hagemann, Johannes Hertrich, and Gabriele Steidl. Stochastic normalizing flows for inverse problems: A markov chains viewpoint. *SIAM/ASA Journal on Uncertainty Quantification*, 10(3):1162–1190, September 2022. ISSN 2166-2525. doi: 10.1137/21m1450604. URL http://dx.doi.org/10.1137/21M1450604.

Katsiaryna Haitsiukevich, Onur Poyraz, Pekka Marttinen, and Alexander Ilin. Diffusion models as probabilistic neural operators for recovering unobserved states of dynamical systems, 2024. URL https://arxiv.org/abs/2405.07097.

W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 04 1970. ISSN 0006-3444. doi: 10.1093/biomet/57.1.97. URL https://doi.org/10.1093/biomet/57.1.97.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL https://arxiv.org/abs/1512.03385.

Tor Aksel N. Heirung, Joel A. Paulson, Jared O'Leary, and Ali Mesbah. Stochastic model predictive control — how does it work? *Computers & Chemical Engineering*, 114:158–170, 2018. ISSN 0098-1354. doi: https://doi.org/10.1016/j.compchemeng.2017.10.026. URL https://www.sciencedirect.com/science/article/pii/S0098135417303812. FOCAPO/CPC 2017.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL https://arxiv.org/abs/2006.11239.

Mai P. Ho, Yang Song, Perminder S. Sachdev, Jiyang Jiang, and Wei Wen. An enhanced conditional variational autoencoder-based normative model for neuroimaging analysis. *bioRxiv*, 2025. doi: 10.1101/2025.01.05.631276. URL https://www.biorxiv.org/content/early/2025/01/05/2025.01.05.631276.

Liam Hodgkinson, Chris van der Heide, Fred Roosta, and Michael W. Mahoney. Stochastic continuous normalizing flows: training SDEs as ODEs. In Cassio de Campos and Marloes H. Maathuis, editors, *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence*, volume 161 of *Proceedings of Machine Learning Research*, pages 1130–1140. PMLR, 27–30 Jul 2021.

M.F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 19(2): 433–450, 1990. doi: 10.1080/03610919008812866. URL https://doi.org/10.1080/03610919008812866.

Pavel Izmailov, Sharad Vikram, Matthew D. Hoffman, and Andrew Gordon Wilson. What are bayesian neural network posteriors really like?, 2021. URL https://arxiv.org/abs/2104.14421.

Kalvik Jakkala. Deep gaussian processes: A survey, 2021. URL https://arxiv.org/abs/2106.12135.

Michael I. Jordan, Zoubin Ghahramani, Tommi S. Jaakkola, and Lawrence K. Saul. *An Introduction to Variational Methods for Graphical Models*. Springer Netherlands,

Dordrecht, 1998. ISBN 978-94-011-5014-9. doi: 10.1007/978-94-011-5014-9_5. URL https://doi.org/10.1007/978-94-011-5014-9_5.

R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 03 1960. ISSN 0021-9223. doi: 10.1115/1.3662552. URL https://doi.org/10.1115/1.3662552.

Shahbaz Abdul Khader, Hang Yin, Pietro Falco, and Danica Kragic. Learning stable normalizing-flow control for robotic manipulation. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1644–1650, 2021. doi: 10.1109/ICRA48506.2021.9562071.

Hassan K Khalil and Jessy W Grizzle. *Nonlinear systems*, volume 3. Prentice hall Upper Saddle River, NJ, 2002.

Patrick Kidger, James Foster, Xuechen Li, Harald Oberhauser, and Terry Lyons. Neural sdes as infinite-dimensional gans, 2021. URL https://arxiv.org/abs/2102.03657.

Diederik P. Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions, 2018. URL https://arxiv.org/abs/1807.03039.

Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, 2019. ISSN 1935-8245. doi: 10.1561/2200000056. URL http://dx.doi.org/10.1561/2200000056.

Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022. URL https://arxiv.org/abs/1312.6114.

Ivan Kobyzev, Simon J.D. Prince, and Marcus A. Brubaker. Normalizing flows: An introduction and review of current methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):3964–3979, November 2021. ISSN 1939-3539. doi: 10.1109/tpami.2020.2992934. URL http://dx.doi.org/10.1109/TPAMI.2020.2992934.

Lingkai Kong, Haichuan Wang, Tonghan Wang, Guojun Xiong, and Milind Tambe. Composite flow matching for reinforcement learning with shifted-dynamics data, 2025. URL https://arxiv.org/abs/2505.23062.

Jakob Kruse, Gianluca Detommaso, Ullrich Köthe, and Robert Scheichl. Hint: Hierarchical invertible neural transport for density estimation and bayesian inference, 2021. URL https://arxiv.org/abs/1905.10687.

Šimon Kucharský and Paul Christian Bürkner. Amortized bayesian cognitive modeling with bayesflow. *PsyArXiv*, May 2025. doi: 10.31234/osf.io/34k6q_v1. URL http://dx.doi.org/10.31234/osf.io/34k6q_v1.

S. Kullback and R. A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. ISSN 00034851. URL http://www.jstor.org/stable/2236703.

Mason Lee and Siddharth Diwan. Improving probabilistic model based reinforcement learning control in chaotic environments with conditional normalizing flows, 2023. URL https://api.semanticscholar.org/CorpusID:260067705.

Xin Li, Markus Lange-Hegermann, and Bogdan Raiţă. Gaussian process regression for inverse problems in linear pdes, 2025a. URL https://arxiv.org/abs/2502.04276.

Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations, 2020a.

Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations, 2020b. URL https://arxiv.org/abs/2001.01328.

Yong Li, Sushanta K. Mallick, Nianling Wang, Jun Yu, and Tao Zeng. Deviance information criterion for bayesian model selection: Theoretical justification and applications. *Journal of Econometrics*, page 105978, 2025b. ISSN 0304-4076. doi: https://doi.org/10.1016/j.jeconom.2025.105978. URL https://www.sciencedirect.com/science/article/pii/S0304407625000326.

Jiahe Lin and George Michailidis. Deep learning-based approaches for state space models: A selective review, 2024. URL https://arxiv.org/abs/2412.11211.

Zhidi Lin, Ying Li, Feng Yin, Juan Maroñas, and Alexandre H. Thiéry. Efficient transformed gaussian process state-space models for non-stationary high-dimensional dynamical systems, 2025. URL https://arxiv.org/abs/2503.18309.

Simon Kristoffersson Lind, Jialong Li, Maj Stenmark, and Volker Krüger. Normalizing flows are capable visuomotor policy learning models, 2025. URL https://arxiv.org/abs/2509.21073.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023. URL https://arxiv.org/abs/2210.02747.

Edward N Lorenz. Predictability: A problem partly solved. In *Proc. Seminar on predictability*, volume 1, pages 1–18. Reading, 1996.

Yiping Lu, Aoxiao Zhong, Quanzheng Li, and Bin Dong. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations, 2020. URL https://arxiv.org/abs/1710.10121.

Lei Lv, Yunfei Li, Yu Luo, Fuchun Sun, Tao Kong, Jiafeng Xu, and Xiao Ma. Flow-based policy for online reinforcement learning, 2025. URL https://arxiv.org/abs/2506.12811.

Xiaobai Ma, Jayesh K. Gupta, and Mykel J. Kochenderfer. Normalizing flow model for policy representation in continuous action multi-agent systems. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '20, page 1916–1918, Richland, SC, 2020. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9781450375184.

David J.C MacKay. Bayesian neural networks and density networks. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 354(1):73–80, 1995. ISSN 0168-9002. doi: https://doi.org/10.1016/0168-9002(94)00931-7. URL https://www.sciencedirect.com/science/article/pii/0168900294009317. Proceedings of the Third Workshop on Neutron Scattering Data Analysis.

Charles C. Margossian and David M. Blei. Amortized variational inference: When and why?, 2024. URL https://arxiv.org/abs/2307.11018.

Jorge Martínez-Palomera, Joshua S. Bloom, and Ellianna S. Abrahams. Deep generative modeling of periodic variable stars using physical parameters, 2020. URL https://arxiv.org/abs/2005.07773.

Bogdan Mazoure, Thang Doan, Audrey Durand, Joelle Pineau, and R Devon Hjelm. Leveraging exploration in off-policy algorithms via normalizing flows. In Leslie Pack Kaelbling, Danica Kragic, and Komei Sugiura, editors, *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 430–444. PMLR, 30 Oct–01 Nov 2020. URL https://proceedings.mlr.press/v100/mazoure20a.html.

David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hongsuk Choi, Haiwen Feng, and Angjoo Kanazawa. Flow matching policy gradients, 2025. URL https://arxiv.org/abs/2507.21053.

Arya D. McCarthy, Xian Li, Jiatao Gu, and Ning Dong. Addressing posterior collapse with mutual information for improved variational neural machine translation. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8512–8525, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.753. URL https://aclanthology.org/2020.acl-main.753/.

Christos Merkatas and Simo Särkkä. System identification using autoregressive bayesian neural networks with nonparametric noise models. *J. Time Ser. Anal.*, 44(3):319–330, May 2023.

Ali Mesbah. Stochastic model predictive control: An overview and perspectives for future research. *IEEE Control Systems Magazine*, 36(6):30–44, 2016. doi: 10.1109/MCS.2016.2602087.

Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 06 1953. ISSN 0021-9606. doi: 10.1063/1.1699114. URL https://doi.org/10.1063/1.1699114.

Ibomoiye Domor Mienye and Theo G. Swart. Deep autoencoder neural networks: A comprehensive review and new perspectives. *Archives of Computational Methods in Engineering*, Mar 2025. ISSN 1886-1784. doi: 10.1007/s11831-025-10260-5. URL https://doi.org/10.1007/s11831-025-10260-5.

Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. Model-based reinforcement learning: A survey, 2022. URL https://arxiv.org/abs/2006.16712.

Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL http://probml.github.io/book1.

Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023. URL http://probml.github.io/book2.

Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do bayesian inference, 2024. URL https://arxiv.org/abs/2112.10510.

Mayank Nautiyal, Andrey Shternshis, Andreas Hellander, and Prashant Singh. Variational autoencoders for efficient simulation-based inference, 2024. URL https://arxiv.org/abs/2411.14511.

Radford M. Neal. *Priors for Infinite Networks*, pages 29–53. Springer New York, New York, NY, 1996. ISBN 978-1-4612-0745-0. doi: 10.1007/978-1-4612-0745-0_2. URL https://doi.org/10.1007/978-1-4612-0745-0_2.

Christopher Nemeth, Paul Fearnhead, and Lyudmila Mihaylova. Sequential monte carlo methods for state and parameter estimation in abruptly changing environments. *IEEE Transactions on Signal Processing*, 62(5):1245–1255, March 2014. ISSN 1941-0476. doi: 10.1109/tsp.2013.2296278. URL http://dx.doi.org/10.1109/TSP.2013.2296278.

Bernt Øksendal. *Stochastic Differential Equations*, pages 38–50. Springer Berlin Heidelberg, Berlin, Heidelberg, 1985. ISBN 978-3-662-13050-6. doi: 10.1007/978-3-662-13050-6_5. URL https://doi.org/10.1007/978-3-662-13050-6_5.

Jose L. Paniagua and Jesús A. López. Nonlinear system identification using modified variational autoencoders. *Intelligent Systems with Applications*, 22:200344, 2024. ISSN 2667-3053. doi: https://doi.org/10.1016/j.iswa.2024.200344. URL https://www.sciencedirect.com/science/article/pii/S2667305324000206.

George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation, 2018. URL https://arxiv.org/abs/1705.07057.

George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference, 2021. URL https://arxiv.org/abs/1912.02762.

Young-Jin Park, Piyush M. Tagade, and Han-Lim Choi. Deep gaussian process-based bayesian inference for contaminant source localization. *IEEE Access*, 6:49432–49449, 2018. doi: 10.1109/ACCESS.2018.2867687.

Ivo Pasmans, Yumeng Chen, Tobias Sebastian Finn, Marc Bocquet, and Alberto Carrassi. Ensemble kalman filter in latent space using a variational autoencoder pair, 2025. URL https://arxiv.org/abs/2502.12987.

V. Peterka. Bayesian system identification. *Automatica*, 17(1):41–53, 1981. ISSN 0005-1098. doi: https://doi.org/10.1016/0005-1098(81)90083-2. URL https://www.sciencedirect.com/science/article/pii/0005109881900832.

Mohammad Pivezhandi and Abusayeed Saifullah. A distribution-aware flow-matching for generating unstructured data for few-shot reinforcement learning, 2025. URL https://arxiv.org/abs/2409.14178.

Thomas Power and Dmitry Berenson. Variational inference mpc using normalizing flows and out-of-distribution projection, 2022. URL https://arxiv.org/abs/2205.04667.

Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. Wiley, 1994. ISBN 978-0-47161977-2. doi: 10.1002/9780470316887. URL https://doi.org/10.1002/9780470316887.

Stefan Radev. Deep learning architectures for amortized bayesian inference in cognitive modeling, 2021. URL https://archiv.ub.uni-heidelberg.de/volltextserver/id/eprint/30807.

Stefan T. Radev, Ulf K. Mertens, Andreas Voss, Lynton Ardizzone, and Ullrich Köthe. Bayesflow: Learning complex stochastic models with invertible neural networks, 2020. URL https://arxiv.org/abs/2003.06281.

Hazhir Rahmandad, Ali Akhavan, and Mohammad S Jalali. Incorporating deep learning into system dynamics: Amortized bayesian inference for scalable likelihood-free parameter estimation. *Syst. Dyn. Rev.*, 41(1), January 2025.

Davide Martino Raimondo, Daniel Limon, Mircea Lazar, Lalo Magni, and Eduardo Ferná ndez Camacho. Min-max model predictive control of nonlinear systems: A unifying overview on stability. *European Journal of Control*, 15(1):5–21, 2009. ISSN 0947-3580. doi: https://doi.org/10.3166/ejc.15.5-21. URL https://www.sciencedirect.com/science/article/pii/S0947358009707034.

Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian processes for machine learning*. MIT Press, 2008.

James Rawlings, David Mayne, and Moritz Diehl. *Model predictive control: theory, computation and design*. Nob Hill Publishing, LLC, Santa Barbara, second edition. edition, 2020. ISBN 9780975937754.

Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows, 2016. URL https://arxiv.org/abs/1505.05770.

Janosh Riebesell, Kapil Sachdeva, Juan Emmanuel Johnson, François Rozet, AifuHan, Byeongkeun Ahn, He Wang, Hossein Pourbozorg, Hyounguk Shon, John Franklin Crenshaw, Kristian Hartikainen, Kyle Cranmer, Matthew Skiffington, Maxime Vandegar, Rafael Orozco, Sandeep Nagar, Thorsten Glüsenkamp, Vincent Stimper, and kleinicke. janosh/awesome-normalizing-flows: v1.0.0, 2023.

Finn Rietz, Erik Schaffernicht, Stefan Heinrich, and Johannes A. Stork. Towards interpretable reinforcement learning with constrained normalizing flow policies, 2024. URL https://arxiv.org/abs/2405.01198.

Geoffrey Roeder, Paul K. Grant, Andrew Phillips, Neil Dalchau, and Edward Meeds. Efficient amortised bayesian inference for hierarchical and nonlinear dynamical systems, 2019. URL https://arxiv.org/abs/1905.12090.

Quentin Rouxel, Clemente Donoso, Fei Chen, Serena Ivaldi, and Jean-Baptiste Mouret. Extremum flow matching for offline goal conditioned reinforcement learning, 2025. URL https://arxiv.org/abs/2505.19717.

Lars Ruthotto and Eldad Haber. Deep neural networks motivated by partial differential equations, 2018. URL https://arxiv.org/abs/1804.04272.

J. Schwartz. The formula for change in variables in a multiple integral. *The American Mathematical Monthly*, 61(2):81–85, 1954. ISSN 00029890, 19300972. URL http://www.jstor.org/stable/2307790.

Max Schwenzer, Muzaffer Ay, Thomas Bergs, and Dirk Abel. Review on model predictive control: an engineering perspective. *The International Journal of Advanced Manufacturing Technology*, 117(5):1327–1349, Nov 2021. ISSN 1433-3015. doi: 10.1007/s00170-021-07682-3. URL https://doi.org/10.1007/s00170-021-07682-3.

Kumar Shridhar, Felix Laumann, and Marcus Liwicki. A comprehensive guide to bayesian convolutional neural network with variational inference, 2019. URL https://arxiv.org/abs/1901.02731.

Satish Shukla, Sriram Balasubramanian, and Mirjana Pavlović. A generalized banach fixed point theorem. *Bulletin of the Malaysian Mathematical Sciences Society*, 39(4): 1529–1539, 2016. ISSN 2180-4206. doi: 10.1007/s40840-015-0255-5. URL https://doi.org/10.1007/s40840-015-0255-5.

Daniele Silvestro and Tobias Andermann. Prior choice affects ability of bayesian neural networks to identify unknowns, 2020. URL https://arxiv.org/abs/2005.04987.

Dan Simon. *Optimal state estimation: Kalman, H ∞, and nonlinear approaches*. John Wiley & Sons, 2006.

John Skilling. *The Eigenvalues of Mega-dimensional Matrices*, pages 455–466. Springer Netherlands, Dordrecht, 1989. ISBN 978-94-015-7860-8. doi: 10.1007/978-94-015-7860-8_48. URL https://doi.org/10.1007/978-94-015-7860-8_48.

Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/file/8d55a249e6baa5c06772297520da2051-Paper.pdf.

Shriyank Somvanshi, Md Monzurul Islam, Mahmuda Sultana Mimi, Sazzad Bin Bashar Polock, Gaurab Chhetri, and Subasish Das. From s4 to mamba: A comprehensive survey on structured state space models, 2025. URL https://arxiv.org/abs/2503.18970.

David J. Spiegelhalter, Nicola G. Best, Bradley P. Carlin, and Angelika Van Der Linde. Bayesian measures of model complexity and fit. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 64(4):583–639, 10 2002. ISSN 1369-7412. doi: 10.1111/1467-9868.00353. URL https://doi.org/10.1111/1467-9868.00353.

David J. Spiegelhalter, Nicola G. Best, Bradley P. Carlin, and Angelika van der Linde. The deviance information criterion: 12 years on. *Journal of the Royal Statistical Society. Series B (Statistical Methodology)*, 76(3):485–493, 2014. ISSN 13697412, 14679868. URL http://www.jstor.org/stable/24774528.

Hui Sun, He Wang, and Jibo He. Accelerating bayesian sampling for massive black hole binaries with prior constraints from conditional variational autoencoder. *Physical Review D*, 111(10), May 2025. ISSN 2470-0029. doi: 10.1103/8r8b-hckx. URL http://dx.doi.org/10.1103/8r8b-hckx.

Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In Bruce Porter and Raymond Mooney, editors, *Machine Learning Proceedings 1990*, pages 216–224. Morgan Kaufmann, San Francisco (CA), 1990. ISBN 978-1-55860-141-3. doi: https://doi.org/10.1016/B978-1-55860-141-3.50030-4. URL https://www.sciencedirect.com/science/article/pii/B9781558601413500304.

Simo Särkkä. The use of gaussian processes in system identification, 2019. URL https://arxiv.org/abs/1907.06066.

Timur Takhtaganov and Juliane Müller. Adaptive gaussian process surrogates for bayesian inference, 2018. URL https://arxiv.org/abs/1809.10784.

Denis Tarasov, Alexander Nikulin, Ilya Zisman, Albina Klepach, Nikita Lyubaykin, Andrei Polubarov, Alexander Derevyagin, and Vladislav Kurenkov. Nina: Normalizing flows in action. training vla models with normalizing flows, 2025. URL https://arxiv.org/abs/2508.16845.

Travis Tune, Kristina B. Kooiker, Jennifer Davis, Thomas Daniel, and Farid Moussavi-Harami. Bayesian estimation of muscle mechanisms and therapeutic targets using variational autoencoders. *Biophysical Journal*, 124(1):179–191, 2025. ISSN 0006-3495. doi: https://doi.org/10.1016/j.bpj.2024.11.3310. URL https://www.sciencedirect.com/science/article/pii/S0006349524022380.

Julen Urain, Michele Ginesi, Davide Tateo, and Jan Peters. Imitationflow: Learning deep stable stochastic dynamic systems by normalizing flows. In *2020 IEEE/RSJ*

International Conference on Intelligent Robots and Systems (IROS), pages 5231–5237, 2020. doi: 10.1109/IROS45743.2020.9341035.

Martin J Wainwright and Michael I Jordan. *Graphical Models, Exponential Families, and Variational Inference*. Now Publishers Inc., Hanover, MA, USA, 2008. ISBN 1601981848.

E.A. Wan and R. Van Der Merwe. The unscented kalman filter for nonlinear estimation. In *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (Cat. No.00EX373)*, pages 153–158, 2000. doi: 10.1109/ASSPCC.2000.882463.

Zhenglin Wan, Jingxuan Wu, Xingrui Yu, Chubin Zhang, Mingcong Lei, Bo An, and Ivor Tsang. Fm-irl: Flow-matching for reward modeling and policy regularization in reinforcement learning, 2025. URL https://arxiv.org/abs/2510.09222.

Xintong Wang, Xiaofei Guan, Ling Guo, and Hao Wu. Flow-based bayesian filtering for high-dimensional nonlinear stochastic dynamical systems, 2025. URL https://arxiv.org/abs/2502.16232.

Yixin Wang, David M. Blei, and John P. Cunningham. Posterior collapse and latent variable non-identifiability, 2023. URL https://arxiv.org/abs/2301.00537.

Patrick Nadeem Ward, Ariella Smofsky, and Avishek Joey Bose. Improving exploration in soft-actor-critic with normalizing flows policies, 2019. URL https://arxiv.org/abs/1906.02771.

Junjie Wen, Minjie Zhu, Yichen Zhu, Zhibin Tang, Jinming Li, Zhongyi Zhou, Chengmeng Li, Xiaoyu Liu, Yaxin Peng, Chaomin Shen, and Feifei Feng. Diffusion-vla: Generalizable and interpretable robot foundation model via self-generated reasoning, 2025. URL https://arxiv.org/abs/2412.03293.

Christina Winkler, Daniel Worrall, Emiel Hoogeboom, and Max Welling. Learning likelihoods with conditional normalizing flows, 2023. URL https://arxiv.org/abs/1912.00042.

Hao Wu, Jonas Köhler, and Frank Noé. Stochastic normalizing flows, 2020. URL https://arxiv.org/abs/2002.06707.

Sheng Xu and Guiliang Liu. Uncertainty-aware constraint inference in inverse constrained reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=ILYjDvUM6U.

Takahiro S. Yamamoto and Takahiro Tanaka. Use of conditional variational auto encoder to analyze ringdown gravitational waves, 2 2020.

Shi Yan, Yan Liang, Le Zheng, Mingyang Fan, Xiaoxu Wang, and Binglu Wang. Explainable gated bayesian recurrent neural network for non-markov state estimation, 2024. URL https://arxiv.org/abs/2310.17187.

Liu Yang, Xuhui Meng, and George Em Karniadakis. B-pinns: Bayesian physics-informed neural networks for forward and inverse pde problems with noisy data. *Journal of Computational Physics*, 425:109913, 2021. ISSN 0021-9991. doi: https://doi.org/10.1016/j.jcp.2020.109913. URL https://www.sciencedirect.com/science/article/pii/S0021999120306872.

Hang Yin, Christos K. Verginis, and Danica Kragic. Consensus-based normalizing-flow control: A case study in learning dual-arm coordination. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10417–10424, 2022. doi: 10.1109/IROS47612.2022.9981827.

Jice Zeng, Michael D. Todd, and Zhen Hu. Probabilistic damage detection using a new likelihood-free bayesian inference method. *Journal of Civil Structural Health Monitoring*, 13(2):319–341, Mar 2023. ISSN 2190-5479. doi: 10.1007/s13349-022-00638-5. URL https://doi.org/10.1007/s13349-022-00638-5.

Qinglun Zhang, Zhen Liu, Haoqiang Fan, Guanghui Liu, Bing Zeng, and Shuaicheng Liu. Flowpolicy: Enabling fast and robust 3d flow-based policy via consistency flow matching for robot manipulation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(14):14754–14762, Apr. 2025a. doi: 10.1609/aaai.v39i14.33617. URL https://ojs.aaai.org/index.php/AAAI/article/view/33617.

Shiyuan Zhang, Weitong Zhang, and Quanquan Gu. Energy-weighted flow matching for offline reinforcement learning, 2025b. URL https://arxiv.org/abs/2503.04975.

Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. Reinflow: Fine-tuning flow matching policy with online reinforcement learning, 2025c. URL https://arxiv.org/abs/2505.22094.

Yixian Zhang, Shu'ang Yu, Tonghe Zhang, Mo Guang, Haojia Hui, Kaiwen Long, Yu Wang, Chao Yu, and Wenbo Ding. Sac flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling, 2025d. URL https://arxiv.org/abs/2509.25756.

Zheng Zhang, Jinlin Zhu, Shuyu Zhang, and Furong Gao. Process monitoring using recurrent kalman variational auto-encoder for general complex dynamic processes. *Engineering Applications of Artificial Intelligence*, 123:106424, 2023. ISSN 0952-1976. doi: https://doi.org/10.1016/j.engappai.2023.106424. URL https://www.sciencedirect.com/science/article/pii/S0952197623006085.

Maksim Zhdanov, Lisa Randolph, Thomas Kluge, Motoaki Nakatsutsumi, Christian Gutt, Marina Ganeva, and Nico Hoffmann. Amortized bayesian inference of gisaxs data with normalizing flows, 2022. URL https://arxiv.org/abs/2210.01543.

Shan Zhong, Shutong Ding, He Diao, Xiangyu Wang, Kah Chan Teh, and Bei Peng. Flowcritic: Bridging value estimation with flow matching in reinforcement learning, 2025. URL https://arxiv.org/abs/2510.22686.

Feiqin Zhu, Dmitrii Torbunov, Zhongjing Jiang, Tianqiao Zhao, Amirthagunaraj Yogarathnam, Yihui Ren, and Meng Yue. Diffusion model-based parameter estimation in dynamic power systems, 2025. URL https://arxiv.org/abs/2411.10431.

Zhengbang Zhu, Hanye Zhao, Haoran He, Yichao Zhong, Shenyu Zhang, Haoquan Guo, Tingting Chen, and Weinan Zhang. Diffusion models for reinforcement learning: A survey, 2024. URL https://arxiv.org/abs/2311.01223.

Appendix

Acknowledgement

Eidesstattliche Versicherung

Statutory Declaration

1. Bei der eingereichten Dissertation zu dem Thema ***Deep Bayesian Inference for Control: Normalizing Flow–Based Approaches to System Identification, State Estimation, and Optimal Control*** handelt es sich um meine eigenständig erbrachte Leistung.
   I herewith formally declare that I have written the submitted dissertation ***Deep Bayesian Inference for Control: Normalizing Flow–Based Approaches to System Identification, State Estimation, and Optimal Control*** *independently.*
2. Ich habe nur die angegebenen Quellen und Hilfsmittel benutzt und mich keiner unzulässigen Hilfe Dritter bedient. Insbesondere habe ich wörtlich oder sinngemäß aus anderen Werken übernommene Inhalte als solche kenntlich gemacht.
   I did not use any third party support except for the quoted literature and other sources mentioned in the text. Content from other work, either literally or in content, has been declared as such.
3. Die Arbeit oder Teile davon habe ich bislang nicht an einer Hochschule des In- oder Auslands als Bestandteil einer Prüfungs- oder Qualifikationsleistung vorgelegt.
   The thesis has not been submitted to any examination body in this, or similar, form.
4. Die Richtigkeit der vorstehenden Erklärungen bestätige ich.
   I confirm the correctness of the aforementioned declarations.
5. Die Bedeutung der eidesstattlichen Versicherung und die strafrechtlichen Folgen einer unrichtigen oder unvollständigen eidesstattlichen Versicherung sind mir bekannt. Ich versichere an Eides statt, dass ich nach bestem Wissen die reine Wahrheit erklärt und nichts verschwiegen habe.
   I am aware of the legal consequences of this declaration. To the best of my knowledge I have told the pure truth and not concealed anything.

Augsburg, 30.04.2026 _____

Angabe zu verwendeten KI-basierter Elektronischer Hilfsmittel

Information on the use of AI-based tools

Zur Dokumentation der verwendeten Hilfsmittel ist der schriftlichen Ausarbeitung ein besonderer Anhang hinzugefügt, der eine Liste und Beschreibung aller verwendeter KI-basierter Hilfsmittel enthält. Der besondere Anhang zur Dokumentation der verwendeten Hilfsmittel erfüllt folgende Kriterien:

To document the tools used, a special appendix has been added to the written report, which contains a list and description of all AI-based tools used. The special appendix to document the tools used meets the following criteria:

1. Auflistung der Ziele, für die die KI-basierten Hilfsmittel in der vorliegenden Arbeit eingesetzt wurden.
   List of the goals for which the AI-based tools were used in this work.

2. Dokumentation der Verwendungsweise der KI-basierten Hilfsmittel.
   Documentation of how AI-based tools are used.

3. Nennung der Kapitel und Abschnitte der vorliegenden Arbeit, in denen die KI-basierten Hilfsmittel eingesetzt wurden, um Inhalte zu erzeugen.
   Identification of the chapters and sections of this work in which AI-based tools were used to generate content.

Der Gebrauch dieser Hilfsmittel inklusive Art, Ziel und Umfang des Gebrauchs wurde mit meinem offiziellen Betreuer **Prof.Dr.Ing. Lars Mikelsons** abgesprochen.

*The use of these aids, including the type, purpose and extent of use, has been agreed with my official supervisor **Prof.Dr.Eng. Lars Mikelsons**.*

Mir ist bewusst, dass insbesondere der Versuch einer nicht dokumentierten Nutzung KI- basierter Hilfsmittel als Täuschungsversuch zu werten ist:

I am aware that in particular the attempt to use AI-based tools without documentation is to be regarded as an attempt at deception:

Gem. § 16 Abs. 2 der Promotionsordnung „Dr. med. / dent.“: „Ergibt sich vor Aushändigung der Promotionsurkunde, dass der Kandidat/die Kandidatin bei einer Promotionsleistung getäuscht hat, so können einzelne oder alle Promotionsleistungen für ungültig erklärt werden. In schweren Fällen kann die Zulassung zum Promotionsverfahren zurückgenommen werden.“

According to § 16 paragraph 2 of the doctoral regulations for "Dr. med. / dent.": "If it emerges before the doctoral certificate is issued that the candidate has cheated in a doctoral achievement, individual or all doctoral achievements can be declared invalid. In serious cases, admission to the doctoral process can be withdrawn.

Und § 16 Abs. 2 der Promotionsordnung „Dr. sc. hum.“: „Ergibt sich vor Aushändigung der Promotionsurkunde, dass der Doktorand / Doktorandin bei einer Promotionsleistung getäuscht hat, so kann der Promotionsausschuss diese Promotionsleistung oder alle bisher erbrachten Promotionsleistungen für ungültig erklären. In besonders schweren Fällen kann der Promotionsausschuss die Annahme als Doktorand / Doktorandin endgültig widerrufen.“

And § 16 paragraph 2 of the doctoral regulations “Dr. sc. hum.”: “If it emerges before the doctoral certificate is issued that the doctoral candidate has cheated in a doctoral achievement, the doctoral committee can declare this doctoral achievement or all doctoral achievements to date invalid. In particularly serious cases, the doctoral committee can permanently revoke the acceptance as a doctoral candidate.”

Augsburg, 30.04.2026 _____